



TRUNG TÂM ĐÀO TẠO AI SAO VIỆT

AI - 03

VIBE CODING

DỰNG WEB APP CÙNG AI

Từ ý tưởng đến sản phẩm chạy thật

2026

Dành cho người muốn tự tay
dựng sản phẩm số bằng AI

LỜI NÓI ĐẦU

Bạn có một ý tưởng. Một trang web bán hàng cho cửa hàng của mình, một công cụ nhỏ để quản lý khách, hay một ứng dụng đặt lịch cho phòng khám. Ý tưởng rõ trong đầu, bạn hình dung được gần như từng màn hình: khách vào thấy gì, bấm nút nào, thông tin chạy đi đâu.

Rồi bạn dừng lại ở đúng một chỗ. Bạn không biết viết code.

Từ trước tới nay, giữa một ý tưởng và một sản phẩm chạy được luôn có một bức tường. Muốn qua tường, bạn có vài lối, và lối nào cũng nặng. Thuê người viết phần mềm thì tốn kém, và mỗi lần muốn đổi một dòng chữ hay một cái nút, bạn lại phải chờ họ. Tự đi học lập trình thì mất hàng tháng, và phần lớn người ta bỏ cuộc từ lâu trước khi làm ra được thứ đầu tiên chạy được.

Có một cách khác

Cách này gọi là vibe coding.

Bạn không gõ code. Bạn mô tả bằng tiếng Việt thứ mình muốn, giống như đang nói cho một lập trình viên ngồi cạnh nghe. Một AI Agent nghe mô tả đó, tự viết code, tự tạo file, tự chạy thử. Bạn nhìn kết quả trên màn hình, thấy chỗ nào chưa ưng thì nói lại bằng lời, và nó sửa.

Công việc của bạn chuyển từ "viết code" sang "mô tả cho rõ và kiểm tra cho kỹ". Đó là hai việc bạn làm được ngay hôm nay, không cần học trước tháng nào.

Cuốn sách này đưa bạn tới đâu

Tám bài của cuốn sách bám theo một hành trình có thật: dựng một cửa hàng bán đồ thủ công trên mạng, từ con số không cho tới lúc nó chạy thật và ai cũng vào được bằng một đường link.

Đi hết tám bài, bạn tự làm được những việc sau trên máy của mình:

- Làm một game nhỏ chạy trên trình duyệt để hiểu một trang web được ghép từ gì
- Dựng đầy đủ giao diện một cửa hàng: trang chủ, danh sách sản phẩm, giỏ hàng, trang quản trị
- Hiểu vì sao có lúc phải chuyển sang một công cụ mạnh hơn, và tự tay chuyển
- Đưa dữ liệu thật vào, cho phép khách đặt hàng, cho người quản lý thêm sửa sản phẩm
- Đưa cả cửa hàng lên mạng bằng một đường link gửi được cho bất kỳ ai

Bạn cần chuẩn bị gì

Một máy tính nối được Internet, một tài khoản Google, và tiếng Việt. Công cụ chính là Antigravity, một phần mềm miễn phí cài trên máy, có sẵn AI Agent bên trong. Bài 1 hướng dẫn cài từ đầu.

Bạn sẽ không phải nhớ một câu lệnh máy tính nào. Thứ bạn viết là mô tả công việc, và thứ bạn rèn là hai thói quen: mô tả sao cho AI hiểu đúng, và kiểm tra sao cho mình không nhận nhầm một kết quả hỏng.

Cách dùng bộ tài liệu đi kèm

Mỗi bài đều có phần kiểm tra kết quả với những dấu hiệu cụ thể nhìn thấy được, để bạn tự biết mình làm đúng hay chưa. Mỗi bài cũng có một thư mục mẫu chứa sản phẩm đã hoàn chỉnh của bài đó. Nếu bạn làm theo mà bị kẹt, mở thư mục mẫu ra đối chiếu, hoặc dùng luôn nó để đi tiếp bài sau.

Bức tường "không biết code" đã đứng đó rất lâu. Cuốn sách này không dạy bạn trèo qua nó. Nó chỉ cho bạn thấy cánh cửa vẫn luôn ở đó, và cách mở.

MỤC LỤC

LỜI NÓI ĐẦU.....	2
MỤC LỤC	4
BÀI 1: VIBE CODING LÀ GÌ	1
1.1 Vibe coding là gì	1
1.2 Một phần mềm gồm những gì	1
1.3 Web chạy trên máy mình và web trên mạng	2
1.4 Ngôn ngữ lập trình, HTML, CSS và JavaScript	3
1.5 Cài Antigravity và mở chỗ làm việc	4
1.6 Cách viết một yêu cầu rõ ràng.....	10
1.7 Làm game đầu tay: Bắt Đầu 30 Giây.....	11
1.8 Khi Agent làm chưa đúng	13
Tóm tắt bài	13
BÀI 2: DỰNG CỬA HÀNG BẰNG HTML, CSS VÀ JAVASCRIPT	14
2.1 Một cửa hàng gồm những màn hình nào	14
2.2 Chuẩn bị: tải ảnh và sắp xếp chỗ làm việc	15
2.3 Dựng khung chung và trang chủ	16
2.4 Danh sách sản phẩm.....	17
2.5 Lọc theo danh mục và tìm kiếm.....	18
2.6 Chi tiết một sản phẩm.....	18
2.7 Giỏ hàng.....	18
2.8 Sản phẩm yêu thích.....	19
2.9 Thanh toán và đặt hàng.....	19
2.10 Đăng nhập và đăng ký.....	19
2.11 Khu quản trị cho người bán.....	19
2.12 Nhìn lại: chỗ này bắt đầu đuổi	20
Tóm tắt bài	20
BÀI 3: CHUYỂN CỬA HÀNG SANG NEXT.JS.....	22
3.1 Nhắc lại ba chỗ đuổi	22
3.2 Framework là gì, và Next.js cho bạn thứ gì	22
3.3 Kiểm kê trước khi chuyển.....	23

3.4 Nhờ Agent cài Node.js	23
3.5 Chỗ ở cho bản mới, và bắt AI lên kế hoạch	24
3.6 Thực thi và nghiệm thu.....	25
3.7 Nhìn lại: bạn được gì sau khi chuyển	26
Tóm tắt bài	26
BÀI 4: CẮT GIỮ CÔNG SỨC VÀ TRANG BỊ THÊM NGHỀ CHO AI.....	27
4.1 Công sức của bạn đang mong manh	27
4.2 Việc thứ nhất: tạo tài khoản GitHub.....	28
4.3 Việc thứ hai: nối máy tính với GitHub.....	29
4.4 Việc thứ ba: tạo một kho chứa	35
4.5 Có những thứ không được lên mạng	37
4.6 Đẩy code lên kho.....	37
4.7 Thói quen lưu mốc.....	39
4.8 Skill: trang bị thêm nghề cho AI	39
4.9 Cài skill và dùng thử	39
Tóm tắt bài	40
BÀI 5: ĐƯA DỮ LIỆU THẬT VÀO CỬA HÀNG	41
5.1 Vì sao cần một kho dữ liệu chung	41
5.2 Chặng một: tạo tài khoản và dựng kho.....	42
5.3 Chặng hai: nối ba đường.....	46
5.4 Chặng ba: nhờ AI dựng bảng và đổ dữ liệu	51
5.5 Cho trang web đọc dữ liệu thật.....	52
5.6 Nghiệm thu: đổi sổ cái, cửa hàng đổi theo	53
Tóm tắt bài	53
BÀI 6: CHO CỬA HÀNG VẬN HÀNH THẬT	54
6.1 Từ trưng bày sang vận hành	54
6.2 Khách đặt hàng, đơn vào sổ cái.....	54
6.3 Đăng nhập thật.....	54
6.4 Ai là chủ cửa hàng.....	55
6.5 Khu quản trị ghi thật	55
6.6 Nhìn lại	56
Tóm tắt bài	56

BÀI 7: KHÁM TỔNG QUÁT TRƯỚC KHI RA MẮT	57
7.1 Khách không báo lỗi, khách bỏ đi.....	57
7.2 Kịch bản người mua	57
7.3 Kịch bản người bán	58
7.4 Thử như một người lạ	58
7.5 Đưa danh sách lỗi cho AI sửa	59
7.6 Danh sách sẵn sàng ra mắt.....	59
Tóm tắt bài	60
BÀI 8: ĐƯA CỬA HÀNG LÊN MẠNG.....	61
8.1 Giờ mở cửa đã tới.....	61
8.2 Khóa kho trước, mở cửa sau	61
8.3 Đưa lên Vercel.....	63
8.4 Đi một vòng trên web thật.....	68
8.5 Nhịp làm việc từ nay về sau	68
8.6 Giữ gìn và đi tiếp	68
Tóm tắt bài	69
LỜI KẾT	70

BÀI 1: VIBE CODING LÀ GÌ

Mục tiêu bài học:

- Hiểu vibe coding là gì và nó khác gì với việc tự học viết code
- Biết một phần mềm gồm ba lớp: giao diện, xử lý, và dữ liệu
- Phân biệt được web chạy trên máy mình và web đã đưa lên mạng
- Hiểu ngôn ngữ lập trình là gì, và vai trò của HTML, CSS, JavaScript
- Cài được AntigraVity và mở một thư mục làm việc cho nó
- Viết được một yêu cầu theo bốn phần VIỆC, NGUỒN, KẾT QUẢ, LƯU Ý
- Tự tay làm xong một game nhỏ chạy được trên trình duyệt
- Biết cách nói lại khi Agent làm chưa đúng ý

1.1 Vibe coding là gì

Hãy tưởng tượng bạn cần dựng một quầy bán hàng.

Cách thứ nhất: bạn tự học nghề mộc. Mua gỗ, mua cưa, học cưa cho thẳng, học đóng đinh cho chắc. Vài tháng sau bạn mới có cái quầy đầu tiên, và nó thường xộc xệch.

Cách thứ hai: bên cạnh bạn có một người thợ mộc giỏi. Bạn không cầm cưa. Bạn mô tả cho họ: "làm cho tôi một quầy dài hai mét, mặt gỗ, có ba ngăn kéo bên dưới". Họ làm. Bạn nhìn, thấy ngăn kéo hơi nhỏ, nói "ngăn kéo to gấp đôi". Họ sửa.

Vibe coding là cách thứ hai, nhưng người thợ ở đây là một AI, và thứ nó đóng ra là phần mềm.

Bạn không viết code. Bạn mô tả bằng tiếng Việt thứ mình muốn. AI viết code, tạo file, chạy thử. Bạn nhìn kết quả, thấy chưa ưng thì nói lại, nó sửa. Chữ "vibe" ý nói bạn làm việc theo cảm nhận về kết quả: nhìn màn hình, thấy đúng hay sai, rồi điều chỉnh bằng lời, chứ không chú ý đầu vào từng dòng lệnh.

Điều này nghe như phép màu, nhưng nó có một cái giá công bằng. Người thợ giỏi tới đâu cũng chỉ làm đúng thứ bạn mô tả. Mô tả mơ hồ thì ra sản phẩm mơ hồ. Vì vậy kỹ năng thật của vibe coding không nằm ở code, mà nằm ở hai chỗ: **mô tả cho rõ** và **kiểm tra cho kỹ**. Cả cuốn sách này xoay quanh đúng hai việc đó.

1.2 Một phần mềm gồm những gì

Trước khi dựng, cần biết mình đang dựng cái gì. Một trang web hay một ứng dụng, dù nhỏ, thường có ba lớp. Hãy hình dung một nhà hàng.

Lớp thứ nhất là phòng ăn. Đây là chỗ khách nhìn thấy và chạm vào: bàn ghế, thực đơn, ánh đèn. Trong phần mềm, lớp này gọi là **giao diện** (tiếng Anh viết tắt là FE,

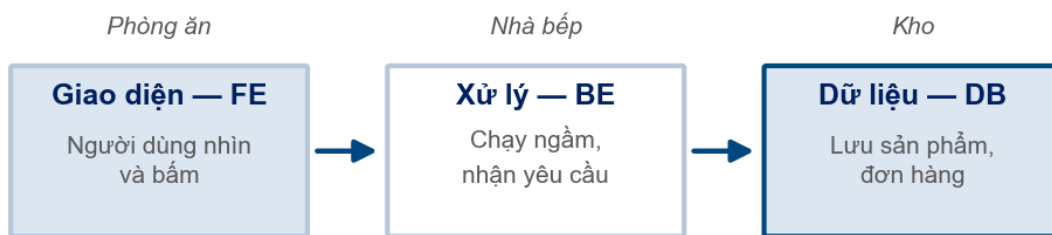


front-end). Là mọi thứ hiện trên màn hình mà người dùng nhìn và bấm: nút, ô nhập, hình ảnh, danh sách sản phẩm.

Lớp thứ hai là nhà bếp. Khách không vào bếp, nhưng mọi món ăn đều nấu ở đó. Trong phần mềm, lớp này gọi là **xử lý** (viết tắt là BE, back-end). Là phần chạy ngầm: khi khách bấm "đặt hàng", chính lớp này nhận yêu cầu, tính tiền, ghi đơn.

Lớp thứ ba là kho. Nơi cất nguyên liệu, cất sổ sách. Trong phần mềm, lớp này gọi là **dữ liệu** (viết tắt là DB, database). Là nơi lưu danh sách sản phẩm, danh sách khách, các đơn hàng đã đặt. Tất máy mở lại, dữ liệu vẫn còn đó.

Một web app thường có ba lớp



Khách chỉ thấy phòng ăn; bếp và kho chạy phía sau

Ba lớp của một web app

Ba lớp này không phải học thuộc. Bạn chỉ cần nhớ chúng để khi đọc các bài sau, bạn biết mình đang làm ở lớp nào. Trong khóa học này, bài 2 và 3 dựng lớp giao diện. Bài 5 và 6 mới dựng tới lớp dữ liệu và lớp xử lý. Cửa hàng đầu tiên của bạn được xây theo đúng thứ tự đó: dựng phòng ăn trước, rồi mới nối bếp và kho.

1.3 Web chạy trên máy mình và web trên mạng

Có một chỗ người mới hay hiểu nhầm, và hiểu sớm sẽ đỡ hụt hẫng về sau.

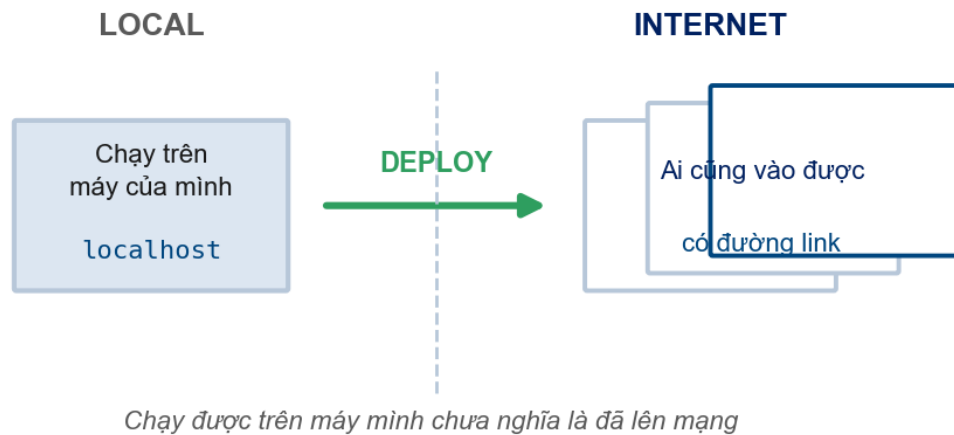
Khi AI dựng xong trang web đầu tiên cho bạn, nó chạy ngay trên máy tính của bạn. Bạn mở trình duyệt, thấy trang hiện ra đầy đủ, bấm nút chạy tốt. Nhìn thì y như một trang web thật trên mạng.

Nhưng nó chưa ở trên mạng. Nó chỉ đang chạy trong máy bạn, giống một món ăn vừa nấu xong còn để trong bếp nhà bạn. Người khác không nếm được. Nếu bạn gửi địa chỉ trang đó cho một người bạn, họ mở sẽ không thấy gì, vì địa chỉ đó chỉ có ý nghĩa trên máy bạn.

Trạng thái này gọi là chạy **cục bộ**, hay quen hơn là chạy **local**. Địa chỉ của nó thường bắt đầu bằng những con số như 127.0.0.1 hoặc chữ localhost. Thấy hai dấu hiệu đó nghĩa là trang đang chạy trong máy bạn, chưa ai ngoài kia vào được.



Để người khác vào được bằng một đường link, bạn phải **đưa web lên mạng**, gọi là **deploy**. Lúc đó trang có một địa chỉ thật, ai gõ vào cũng mở được. Việc này để dành tới bài cuối, khi cửa hàng của bạn đã hoàn chỉnh và đáng để khoe.



Web chạy local và web đã lên mạng

Nhớ một câu: **chạy được trên máy mình chưa có nghĩa là đã lên mạng**. Suốt bảy bài đầu, bạn làm việc với bản local. Điều đó hoàn toàn bình thường và đúng quy trình.

1.4 Ngôn ngữ lập trình, HTML, CSS và JavaScript

Máy tính không hiểu tiếng Việt hay tiếng Anh. Muốn ra lệnh cho nó, người ta dùng một thứ tiếng riêng gọi là **ngôn ngữ lập trình**. Có nhiều ngôn ngữ lập trình khác nhau, giống như thế giới có nhiều thứ tiếng. Trong khóa học này, cái tên bạn sẽ gặp nhiều nhất là **JavaScript**.

Một trang web được ghép từ ba thứ, và chỉ một trong ba là ngôn ngữ lập trình. Hãy hình dung việc xây một căn nhà.

HTML là phần khung nhà. Nó dựng nên bộ xương: chỗ này là tiêu đề, chỗ kia là một đoạn chữ, đây là một cái nút, kia là một tấm ảnh. HTML nói cho trình duyệt biết trang có những gì. Bản thân HTML không phải ngôn ngữ lập trình, nó chỉ là cách đánh dấu "cái này là cái gì".

CSS là phần sơn và nội thất. Cùng một khung nhà, CSS quyết định nó đẹp hay xấu: màu gì, chữ to nhỏ ra sao, nút bo tròn hay vuông, mọi thứ nằm ở đâu trên trang. CSS cũng không phải ngôn ngữ lập trình, nó chỉ lo phần nhìn.

JavaScript là phần điện nước. Đây mới là ngôn ngữ lập trình thật sự. Nó làm cho căn nhà sống dậy: bấm công tắc thì đèn sáng, bấm nút thì có chuyện xảy ra. Khi bạn bấm "thêm vào giỏ" và số trên giỏ hàng tăng lên, đó là JavaScript đang làm việc.

Ba phần ghép nên một trang web



Chỉ JavaScript là ngôn ngữ lập trình; HTML và CSS thì không

HTML dựng khung, CSS làm đẹp, JavaScript tạo tương tác

Ba thứ này đủ để dựng một trang web hoàn chỉnh, và bạn sẽ làm đúng như vậy ở bài 2. Về sau bạn sẽ nghe tới một cái tên nữa là **Next.js**. Xin nói trước để tránh nhầm: Next.js **không phải một ngôn ngữ lập trình**. Nó là một bộ công cụ dựng sẵn, gọi là **framework**, xây bên trên JavaScript, giúp làm những trang web lớn gọn gàng hơn. Bài 3 sẽ giải thích vì sao có lúc cần tới nó. Bây giờ chỉ cần nhớ: JavaScript là ngôn ngữ, Next.js là bộ đồ nghề dựng trên ngôn ngữ đó.

Tin tốt là bạn sẽ không phải tự gõ một dòng HTML, CSS hay JavaScript nào. AI viết hết. Nhưng biết ba cái tên này để làm gì thì bạn sẽ hiểu AI đang làm gì, và ra lệnh cho nó chính xác hơn nhiều.

1.5 Cài Antigravity và mở chỗ làm việc

Công cụ bạn dùng suốt cuốn sách là **Antigravity**, một phần mềm miễn phí của Google, cài trên máy, bên trong có sẵn một AI Agent. Phần này cài từ đầu, từng bước một. Bạn nên dành khoảng mười lăm phút và làm liền một mạch, máy cần nối Internet suốt quá trình.

Chặng 1: Tải bộ cài về máy

Bước 1. Mở trang tải. Mở trình duyệt web bạn hay dùng, thường là Chrome hoặc Microsoft Edge. Trên thanh địa chỉ ở đầu cửa sổ, gõ dòng sau rồi nhấn phím Enter:

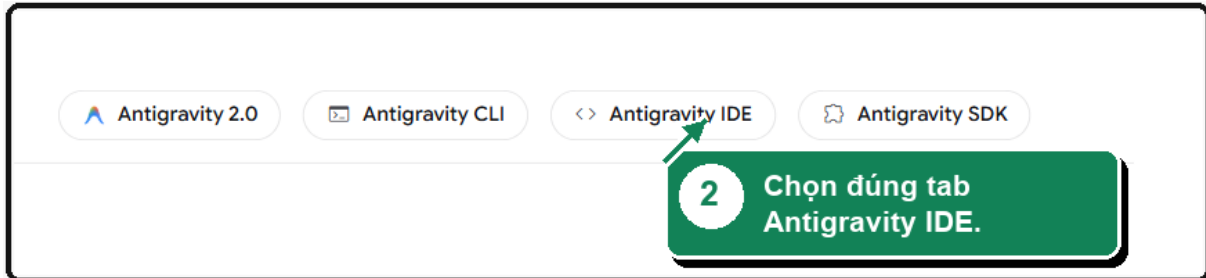
```
antigravity.google/download
```

Bạn sẽ thấy: Trang chủ Antigravity hiện ra, góc trên bên trái có logo hình chữ A nhiều màu.

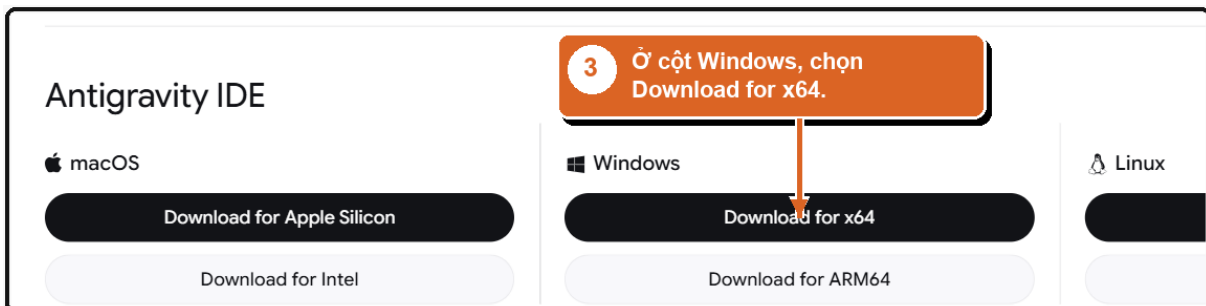


Trang tải Antigravity

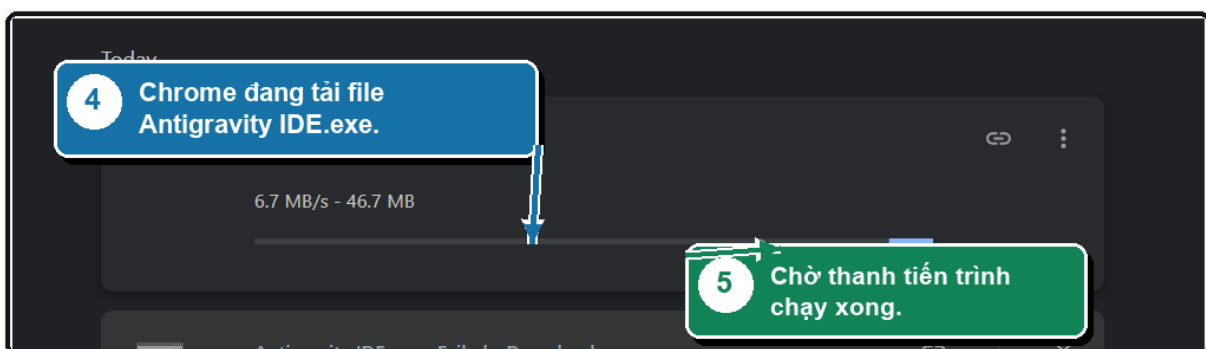
Bước 2. Chọn đúng sản phẩm. Trên trang có mấy ô sản phẩm nằm cạnh nhau. Bấm vào ô **Antigravity IDE**. Các ô còn lại dành cho mục đích khác, chọn nhầm thì phần mềm cài ra sẽ không giống hướng dẫn trong sách.

*Bốn sản phẩm, chỉ chọn Antigravity IDE*

Bước 3. Chọn bản Windows. Kéo trang xuống một chút, bạn thấy ba cột macOS, Windows, Linux. Trong cột **Windows** ở giữa, bấm nút **Download for x64**. Nếu bạn dùng máy Mac thì bấm ở cột macOS bên trái.

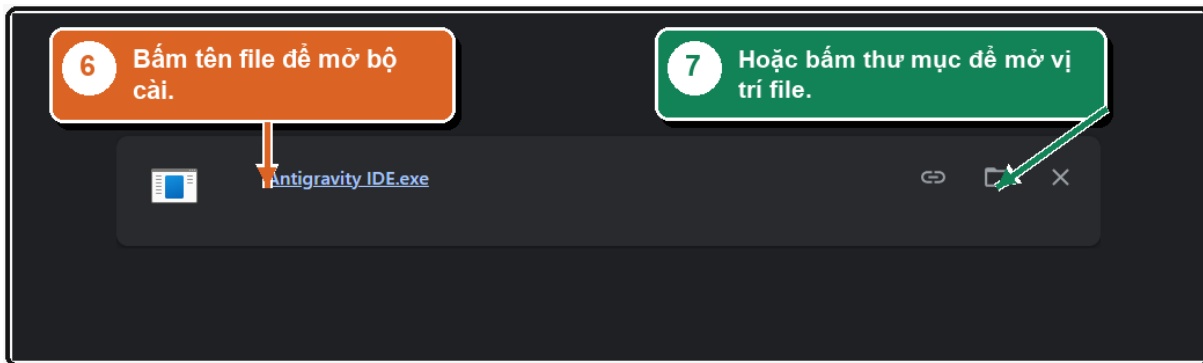
*Cột Windows, nút Download for x64*

Bước 4. Chờ tải xong. Trình duyệt tải về một file tên **Antigravity IDE.exe**, nặng vài trăm megabyte, mất một tới vài phút. Đừng tắt cửa sổ trình duyệt trong lúc này.

*Trình duyệt đang tải bộ cài*

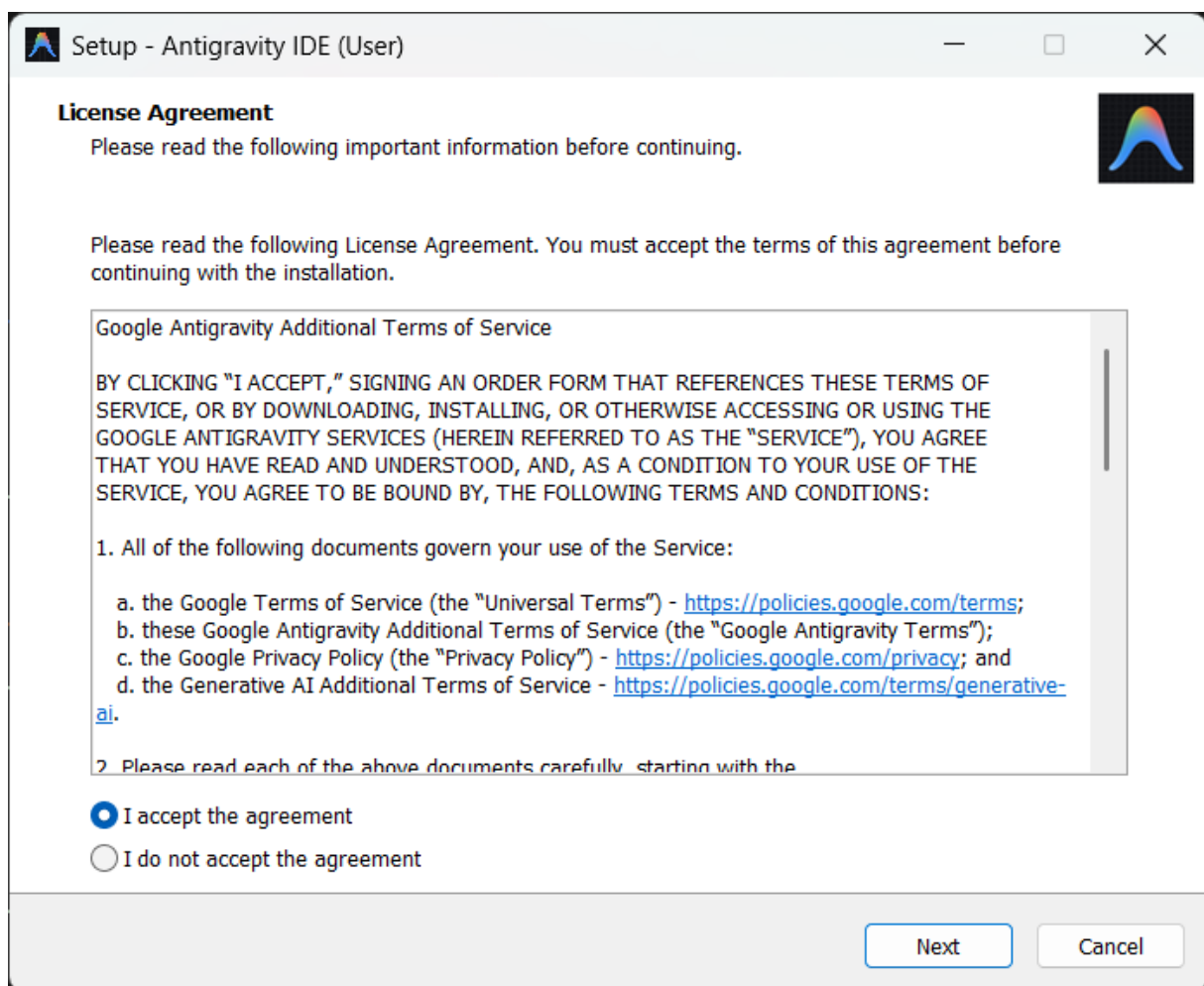
Chặng 2: Chạy bộ cài

Bước 5. Mở file vừa tải. Bấm vào tên file **Antigravity IDE.exe** trong khung thông báo tải của trình duyệt. Nếu Windows hiện hộp thoại hỏi có cho phép ứng dụng thay đổi thiết bị không, bấm **Yes**. Đây là câu hỏi bảo mật bình thường với mọi phần mềm cài mới.



Bấm tên file để mở bộ cài

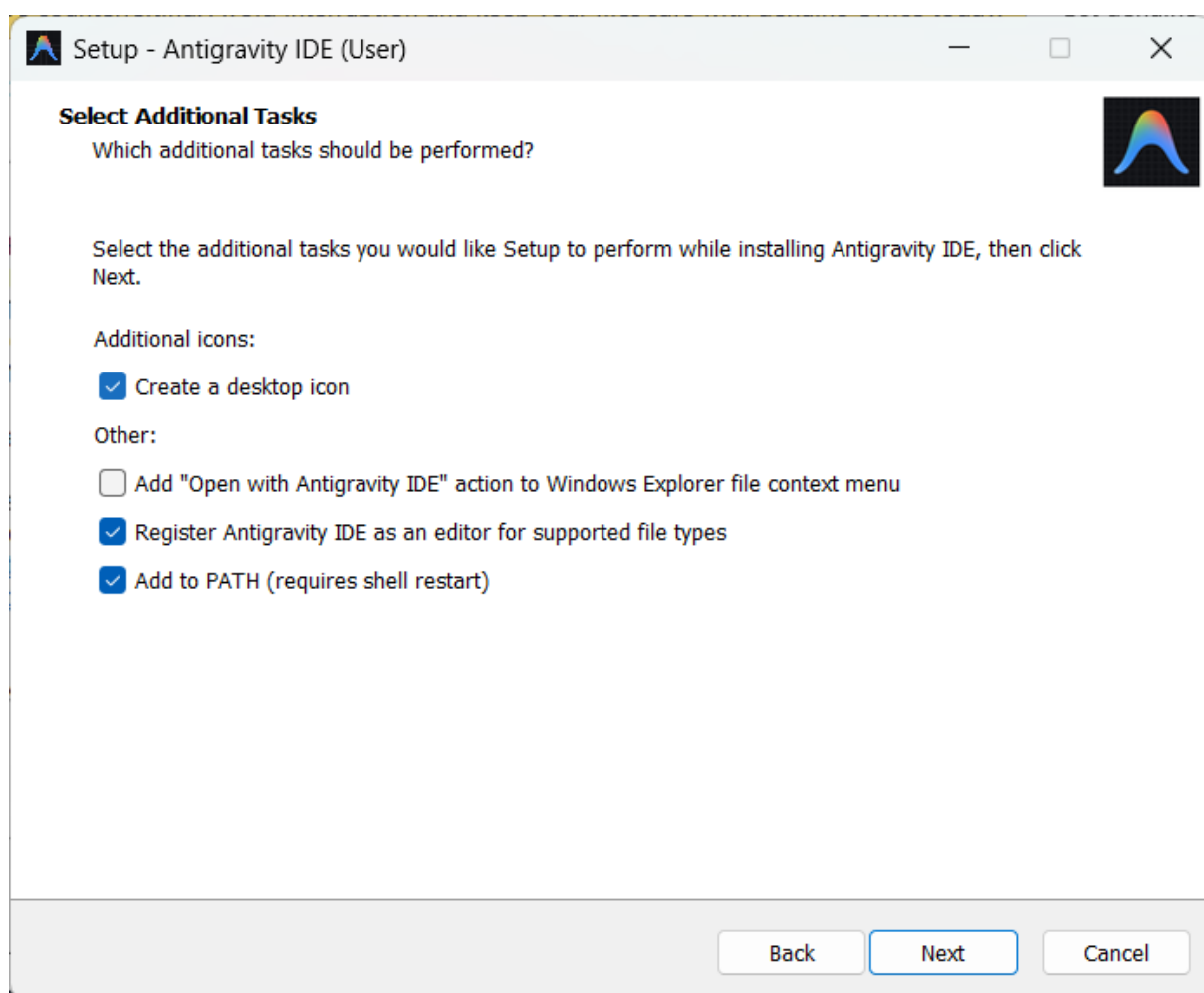
Bước 6. Đồng ý điều khoản. Màn hình đầu tiên tên **License Agreement** chứa điều khoản bằng tiếng Anh. Bấm vào vòng tròn trước dòng **I accept the agreement**, rồi bấm **Next** ở góc dưới bên phải. Chừng nào chưa chọn dòng này thì nút Next còn mờ, bấm không được.



Màn hình License Agreement

Bước 7. Đi qua các màn cài đặt. Vài màn tiếp theo hỏi nơi cài và tên thư mục, bạn cứ để nguyên lựa chọn có sẵn và bấm **Next**. Tới màn **Select Additional Tasks** có mấy ô đánh dấu, bạn để nguyên các ô đã tích sẵn rồi bấm **Next**. Màn cuối bấm **Install**, chờ thanh chạy đầy, rồi bấm **Finish**.

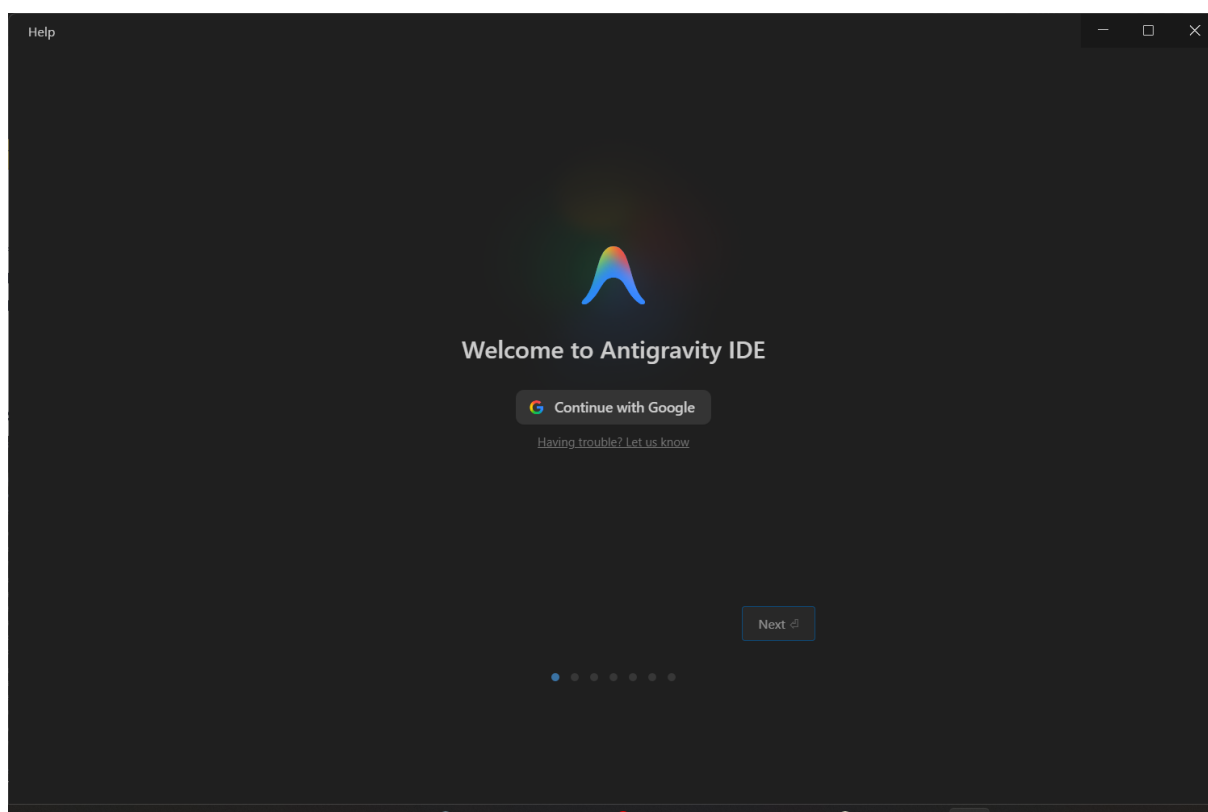




Màn hình Select Additional Tasks

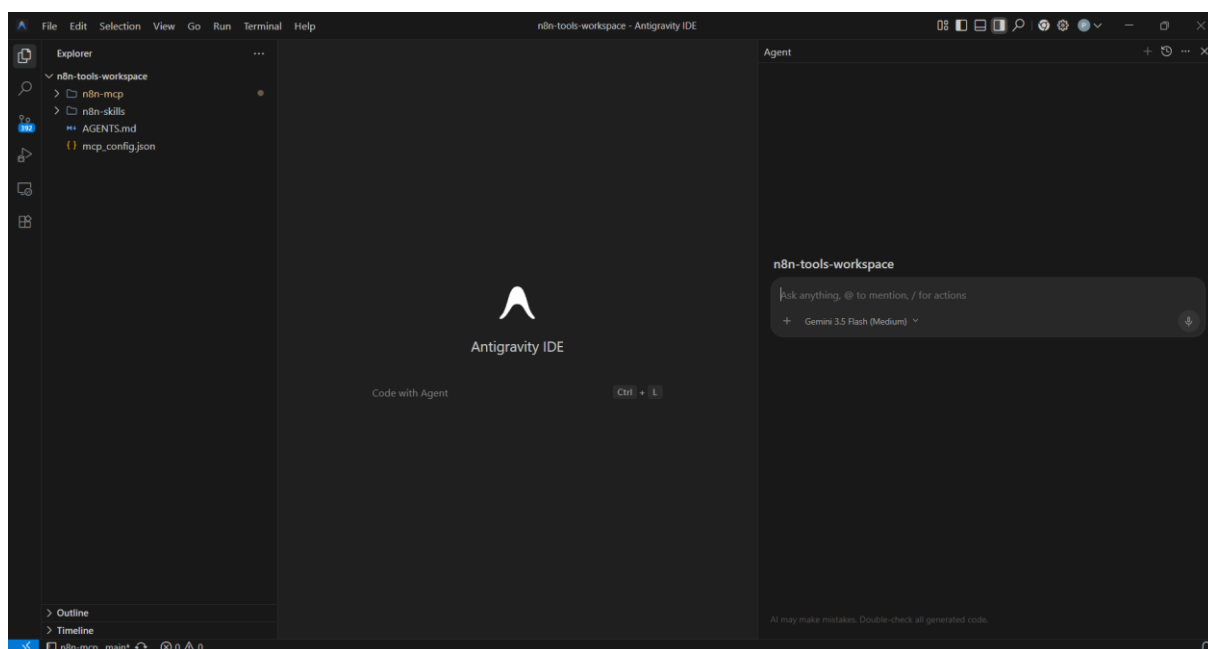
Chặng 3: Đăng nhập

Bước 8. Đăng nhập bằng Google. Antigravity mở lên với màn hình nền tối. Bấm nút **Continue with Google**, chọn tài khoản Google bạn muốn dùng, rồi bấm nút đồng ý cấp quyền. Hãy chọn tài khoản cẩn thận, vì nó gắn với toàn bộ việc về sau. Sau khi đăng nhập, Antigravity dẫn qua vài màn giới thiệu, bạn bấm **Next** cho tới khi giao diện làm việc chính hiện ra.



Màn hình đăng nhập lần đầu

Bạn sẽ thấy: Màn hình chính chia ba phần. Khung bên trái tên **Explorer** là nơi hiện danh sách file. Khung bên phải tên **Agent** là nơi bạn gõ yêu cầu, trông giống một khung chat.



Giao diện làm việc chính

Tạo và mở thư mục làm việc

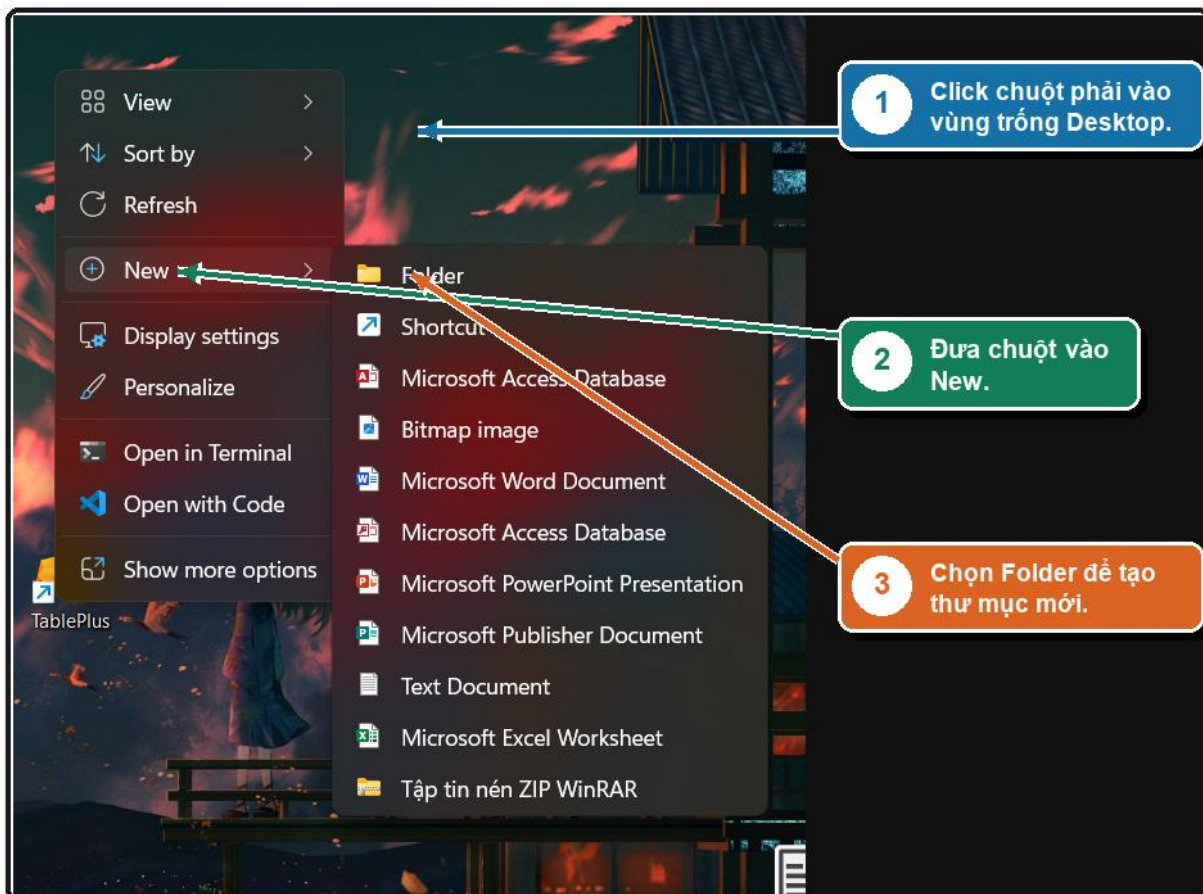


Antigravity chỉ nhìn thấy những file nằm trong thư mục bạn chỉ định. Vì vậy hãy tạo cho nó một chỗ riêng.

Bước 9. Tạo thư mục trên Desktop. Thu nhỏ các cửa sổ để nhìn thấy màn hình Desktop. Bấm chuột phải vào một khoảng trống, rê chuột vào dòng **New**, rồi bấm **Folder**. Một thư mục mới hiện ra đang chờ đặt tên. Gõ tên sau rồi nhấn Enter:

VIBE-CODING

Tên viết hoa, không dấu, dùng dấu gạch ngang thay dấu cách. Đây là thói quen tốt cho mọi thư mục làm việc với AI, vì tên có dấu tiếng Việt hoặc có dấu cách đôi khi gây rắc rối.



Tạo thư mục mới trên Desktop

Bước 10. Chỉ thư mục đó cho Antigravity. Quay lại Antigravity. Trên thanh menu sát mép trên cửa sổ, bấm chữ **File**, rồi chọn **Open Folder**. Một cửa sổ chọn thư mục hiện ra. Nhìn cột bên trái bấm **Desktop**, tìm thư mục **VIBE-CODING**, bấm chọn nó một lần, rồi bấm **Select Folder**.

Bạn sẽ thấy: Khung Explorer bên trái hiện tên **VIBE-CODING**. Từ giờ mọi thứ AI tạo ra sẽ nằm trong thư mục này.



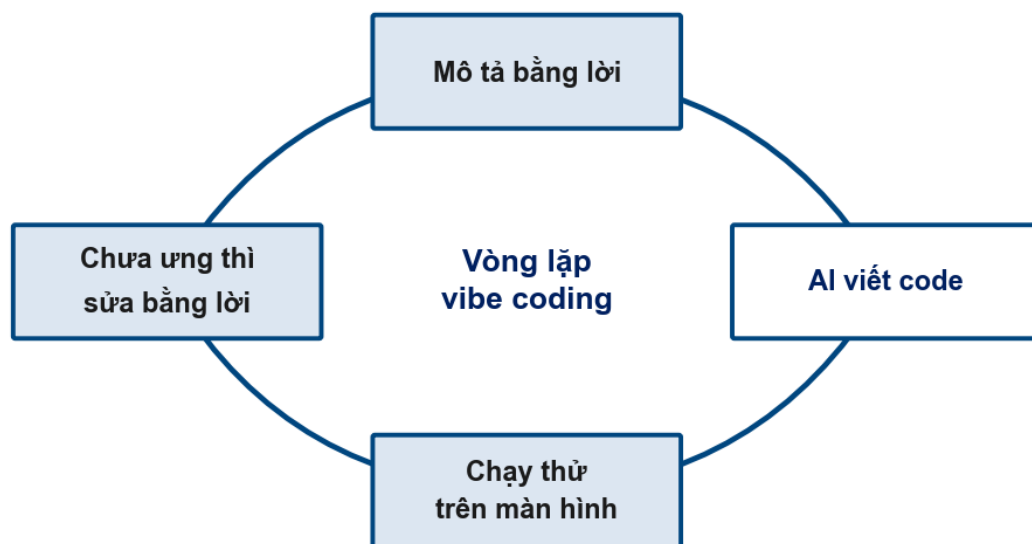
1.6 Cách viết một yêu cầu rõ ràng

Đây là kỹ năng quan trọng nhất của cả khóa học. Mọi thứ còn lại chỉ là áp dụng nó vào từng loại việc.

Người thợ giỏi tới đâu cũng chỉ làm đúng thứ bạn mô tả. Để mô tả cho đủ, hãy tự hỏi bốn câu, và viết câu trả lời thành bốn dòng có nhãn.

Nhãn	Trả lời câu hỏi	Nếu thiếu thì sao
VIỆC	Cần làm cái gì	AI làm một thứ gần giống nhưng không phải thứ bạn cần
NGUỒN	Dựa trên cái gì để làm	AI tự bịa ra dữ liệu, tự chọn công cụ theo ý nó
KẾT QUẢ	Xong thì nhìn thấy gì	AI trả lời bằng chữ, không tạo ra sản phẩm chạy được
LƯU Ý	Có ràng buộc riêng nào	AI tự quyết theo cách của nó, thường không giống ý bạn

Trong bốn nhãn, nhãn **KẾT QUẢ** đáng nói riêng. Với vibe coding, kết quả không phải một đoạn văn, mà là một thứ **nhìn thấy được và chạm được**: một trang mở lên trong trình duyệt, một cái nút bấm vào thì số tăng, một danh sách hiện đúng sản phẩm. Khi viết KẾT QUẢ, hãy tả cái dấu hiệu bạn sẽ nhìn để biết việc đã xong. Đó cũng chính là thứ bạn dùng để kiểm tra ở cuối.



Vòng lặp của vibe coding: mô tả, AI tạo, chạy thử, sửa, lưu lại

Bốn nhãn này theo bạn suốt cuốn sách. Ngay dưới đây, bạn dùng chúng lần đầu để làm một game.



1.7 Làm game đầu tay: Bắt Đầu 30 Giây

Việc đầu tiên nên nhỏ, vui, và thấy kết quả ngay. Bạn sẽ làm một game: những chấm sáng hiện lên khắp màn hình, bấm trúng thì được điểm, bấm nhầm quả bom thì mất một mạng, chơi trong ba mươi giây.



Game Bắt Đầu 30 Giây chạy trên trình duyệt

Trước khi gõ câu đúng, hãy xem chuyện gì xảy ra với một câu viết vội. Đây là cách gần như ai cũng gõ trong lần đầu.

Yêu cầu chưa tốt

Làm cho tôi một game bằng code.

Câu này có bốn chỗ hỏng, và mỗi chỗ dẫn tới một kiểu hỏng:

Chỗ hỏng	AI sẽ làm gì
Không nói game gì	Đoán đại một game bất kỳ, thường không phải thứ bạn hình dung
Không nói chạy ở đâu	Có thể tạo ra file không mở lên chơi được
Không nói luật chơi	Tự bịa luật, cách tính điểm không giống ý bạn
Không nói dấu hiệu xong	Bạn không có gì để đối chiếu xem nó làm đúng chưa

Bây giờ viết lại cho đủ. Việc này làm hai bước, và bước đầu chính là chỗ khác biệt của vibe coding thật.

Yêu cầu tốt



Bước 1. Đưa ảnh mẫu cho Agent. Kéo thẳng ảnh game mẫu vào khung Agent, thả vào ô nhập chữ. Ảnh sẽ hiện thành một ô nhỏ ngay trong ô nhập. Đưa ảnh giúp Agent nhìn thấy game trông thế nào, thay vì phải tự đoán.

Bước 2. Gõ yêu cầu ngay sau ảnh. Gõ tiếp đoạn dưới đây, bạn có thể chép nguyên văn:

```
VIỆC: Làm một game bắt điểm theo phong cách trong ảnh tôi vừa gửi, chạy trên trình duyệt
NGUỒN: Chỉ dùng HTML, CSS và JavaScript, không dùng thư viện ngoài
KẾT QUẢ: Một thư mục có file index.html mở lên chơi được, có nút Bắt đầu, đồng hồ đếm ngược 30 giây, điểm số và số mạng hiện rõ trên đầu màn hình
LƯU Ý: Chấm xanh bấm trúng thì cộng một điểm, bom bấm nhầm thì trừ một mạng; hết 30 giây hoặc hết 3 mạng thì dừng và hiện điểm cuối. Giao diện gọn gàng, chữ tiếng Việt.
```

Gõ xong, nhấn phím Enter. Với một ảnh lẻ như game này, kéo thẳng vào khung chat là gọn nhất. Ở bài sau, khi cần trở tới ảnh nằm sẵn trong dự án, bạn sẽ dùng thêm dấu @.

Agent sẽ làm gì

Khung bên phải bắt đầu chạy. Bạn sẽ thấy Agent lần lượt báo những việc nó làm: tạo file index.html cho khung, style.css cho giao diện, script.js cho phần chơi. Nó có thể tự mở trang lên để chạy thử. Toàn bộ mất vài chục giây tới một hai phút. Trong lúc đó cứ để yên, đừng gõ thêm.

Chơi thử và kiểm tra

Khi Agent báo xong, mở game lên chơi. Nếu Agent đã tự mở sẵn một cửa sổ xem trước thì bạn chơi luôn ở đó. Nếu chưa, nhìn khung Explorer bên trái, bấm phải vào file index.html, chọn dòng mở bằng trình duyệt (thường ghi **Open with Live Server** hoặc **Reveal in File Explorer** rồi bấm đúp để mở).

Đối chiếu năm dấu hiệu sau, đây chính là phần KẾT QUẢ và LƯU Ý bạn đã viết:

Dấu hiệu	Đúng là
Nút Bắt đầu	Bấm vào thì game khởi động
Đồng hồ	Đếm ngược từ 30 về 0
Điểm	Bấm trúng chấm xanh thì tăng
Mạng	Bấm nhầm quả bom thì giảm một
Kết thúc	Hết giờ hoặc hết mạng thì dừng, hiện điểm cuối

Nếu cả năm đều đúng, bạn vừa làm xong phần mềm đầu tiên của mình mà không viết một dòng code nào.



1.8 Khi Agent làm chưa đúng

Lần đầu, kết quả lệch ở đâu đó là chuyện thường. Điều quan trọng là biết cách nói lại. Bạn không cần bắt đầu lại từ đầu, cứ gõ tiếp vào ô nhập, nói rõ chỗ nào chưa đúng, Agent nhớ việc vừa làm và sửa tiếp trên đó.

Game không mở lên được. Nói lại cho rõ nơi chạy:

Trang không mở được. Hãy cho tôi một file index.html mở thẳng bằng trình duyệt là chơi được, không cần cài thêm gì.

Đồng hồ không chạy, hoặc điểm không tăng. Chỉ đúng chỗ hỏng:

Đồng hồ đứng yên không đếm. Hãy sửa để khi bấm Bắt đầu thì đồng hồ đếm ngược từ 30 về 0.

Agent làm thừa những thứ bạn không xin. Ví dụ nó tự thêm bảng xếp hạng, âm thanh, nhiều màn. Nói rõ giới hạn:

Bỏ hết phần thừa. Chỉ giữ đúng một màn chơi với nút Bắt đầu, đồng hồ, điểm và mạng.

Một thói quen nên tập từ bây giờ: sau mỗi lần Agent báo xong, hãy tự tay chạy thử và nhìn kết quả, đừng tin lời báo cáo mà bỏ qua bước nhìn. Ở những bài sau, khi sản phẩm là cả một cửa hàng, thói quen nhìn kỹ này là thứ giữ cho bạn không đưa lên mạng một thứ đang hỏng.

Tóm tắt bài

- Vibe coding là mô tả bằng lời cho AI viết code; kỹ năng thật nằm ở mô tả cho rõ và kiểm tra cho kỹ
- Một phần mềm có ba lớp: giao diện (FE), xử lý (BE), dữ liệu (DB)
- Web chạy trên máy mình gọi là local, địa chỉ có localhost; muốn người khác vào được phải deploy lên mạng
- HTML dựng khung, CSS làm đẹp, JavaScript tạo tương tác; chỉ JavaScript là ngôn ngữ lập trình, còn Next.js là framework
- Antigravity chỉ nhìn thấy file trong thư mục bạn mở qua menu File và Open Folder
- Một yêu cầu rõ gồm bốn phần VIỆC, NGUỒN, KẾT QUẢ, LƯU Ý; phần KẾT QUẢ tả dấu hiệu nhìn thấy được để sau này đối chiếu
- Kết quả chưa đúng thì gõ tiếp để sửa, không cần làm lại từ đầu; luôn tự chạy thử trước khi tin là xong



BÀI 2: DỰNG CỬA HÀNG BẰNG HTML, CSS VÀ JAVASCRIPT

Mục tiêu bài học:

- Biết một cửa hàng trực tuyến gồm những màn hình nào, dựng theo thứ tự nào
- Đưa ảnh mẫu giao diện cho AI bằng dấu @ rồi mô tả để nó dựng theo
- Viết prompt để giao diện không bị nhật nhẽo kiểu AI (tránh AI slop)
- Dựng lần lượt: khung chung, danh sách, lọc, tìm, chi tiết, giỏ hàng, yêu thích, thanh toán
- Dựng được đăng nhập giả và khu quản trị cho người bán
- Biết bổ sung tính năng và chỉnh giao diện bằng những câu prompt ngắn
- Nhận ra chỗ cách làm này bắt đầu đuối, để hiểu vì sao bài sau đổi công cụ

2.1 Một cửa hàng gồm những màn hình nào

Ở bài trước bạn làm một game gói gọn trong một màn. Cửa hàng thì khác, nó có nhiều màn hình nối nhau. Nếu xin AI làm hết trong một câu, bạn sẽ nhận về một mớ rối và không biết chỗ nào hỏng. Vì vậy ta dựng từng màn một, theo thứ tự, xong màn này mới sang màn sau.

Thứ tự dựng có một nguyên tắc: làm phần khách nhìn thấy trước, phần quản trị sau. Khách vào cửa hàng thì xem sản phẩm, bỏ vào giỏ, đặt mua. Người bán thì đăng nhập vào khu riêng để thêm sửa sản phẩm và xem đơn. Dựng phần khách trước vì nó cho bạn thấy kết quả đẹp sớm, có động lực đi tiếp.

Đây là những màn hình bạn sẽ dựng, theo đúng thứ tự:

Nhóm	Màn hình
Khách	Khung chung và trang chủ
Khách	Danh sách sản phẩm
Khách	Lọc theo danh mục và tìm kiếm
Khách	Chi tiết một sản phẩm
Khách	Giỏ hàng
Khách	Sản phẩm yêu thích
Khách	Thanh toán, đặt hàng
Khách	Đăng nhập, đăng ký
Quản trị	Trang tổng quan
Quản trị	Quản lý sản phẩm
Quản trị	Quản lý đơn hàng



2.2 Chuẩn bị: tải ảnh và sắp xếp chỗ làm việc

Cửa hàng bạn sắp dựng tên **Mini Shop**, bán đồ thủ công và trang trí. Để dựng nó, bạn cần một bộ ảnh, tải về ở đường link sau:

Link tải ảnh: [Bộ ảnh Mini Shop](#)

Mở link, bạn thấy hai thư mục. Tải cả hai về máy: bấm chuột phải vào tên thư mục rồi chọn **Tải xuống**, Drive sẽ nén lại thành một file rồi bạn giải nén ra.

- **assets:** ảnh các mặt hàng thật (giỏ mây, bình gốm, đèn tre, sofa...) và ảnh banner. Đây là hàng hóa sẽ hiện trên web.
- **giao_dien_web:** vài ảnh chụp mẫu các trang (trang chủ, danh sách, chi tiết, khu quản trị). Đây là ảnh bạn đưa cho AI để nó dựng giao diện trông giống vậy.

Tạo một thư mục mới trên Desktop tên **MINI-SHOP**, rồi chép cả hai thư mục vừa giải nén vào trong đó. Quay lại Antigravity, vào **File** rồi **Open Folder**, chọn thư mục **MINI-SHOP**.

Vì sao ảnh phải xếp đúng chỗ

Mở thư mục **assets** ra, bạn thấy ảnh không nằm lộn xộn mà xếp theo tầng:

```
assets/  
├── images/  
│   ├── banner/           (ảnh banner đầu trang)  
│   └── products/         (ảnh sản phẩm, chia theo danh mục)  
│       ├── do-thu-cong/  
│       ├── do-my-nghe/  
│       └── noi-that-gia-dung/
```

Vì sao phải xếp gọn như vậy thay vì đổ hết ảnh vào một chỗ? Ba lý do:

- **Bạn tìm nhanh.** Cần ảnh đèn thì vào thẳng nhóm đồ mỹ nghệ, không phải lục giữa hàng chục tấm.
- **AI hiểu đúng.** Khi bạn bảo "dùng ảnh trong thư mục products", AI biết chính xác lấy ở đâu.
- **Web không vỡ ảnh**, và đây là lý do quan trọng nhất, cần nói kỹ.

Ảnh được đưa vào web như thế nào

Một trang web không "chứa" ảnh bên trong. Nó chỉ **trỏ tới đường dẫn** của ảnh, giống như bạn ghi địa chỉ nhà chứ không bê cả căn nhà theo. Trong code, mỗi sản phẩm có một dòng đại ý "ảnh của tôi nằm ở assets/images/products/do-thu-cong/gio-may-dan.jpg". Khi mở trang, trình duyệt đi theo địa chỉ đó lấy ảnh về hiện.

Hệ quả: nếu ảnh nằm sai chỗ so với địa chỉ code ghi, ô ảnh sẽ trống hoặc hiện dấu vỡ. Vì vậy **giữ nguyên cấu trúc thư mục assets như khi tải về** là cách an toàn nhất, để code và ảnh khớp địa chỉ với nhau. Bạn không phải tự viết các địa chỉ này, AI làm. Nhưng hiểu cơ chế thì khi thấy ô ảnh trống, bạn biết ngay là "ảnh đặt sai chỗ" và bảo AI sửa.

Đưa ảnh mẫu cho AI bằng dấu @



Các ảnh trong **giao_dien_web** thì không bỏ vào code, mà bạn đưa cho AI xem để nó dựng giao diện theo. Cách đưa: trong ô nhập của khung Agent, gõ dấu @. Một danh sách các file trong dự án hiện ra ngay. Bạn chọn đúng ảnh của trang đang làm, ví dụ dựng trang chủ thì chọn `mini-shop-user-home-reference.png`. Tên ảnh được chèn vào ô nhập, và AI biết bạn đang trỏ tới ảnh đó. Mỗi màn dưới đây sẽ nói rõ chọn ảnh nào.

Một điều nên nhớ trước khi bắt đầu

AI hiếm khi làm đúng ý hoàn toàn ngay lần đầu, và điều đó bình thường. Cách làm việc ở đây không phải "ra lệnh một câu rồi xong", mà là **dựng thô rồi chỉnh dần bằng lời**. Bạn dựng một trang, nhìn, rồi gõ tiếp những câu ngắn để sửa: "nút to hơn", "thêm khoảng cách giữa các sản phẩm", "làm cho xem được trên điện thoại". Mỗi màn dưới đây cho bạn một câu để bắt đầu, phần chỉnh sau là của bạn.

Điểm cần kiểm tra trước khi dựng

- Thư mục **MINI-SHOP** đã mở trong Antigravity, bên trong nhìn thấy cả **assets** và **giao_dien_web**.
- Cấu trúc bên trong **assets** giữ nguyên như khi tải, không xáo trộn.
- Gõ thử dấu @ trong khung Agent, thấy danh sách file hiện ra.

2.3 Dựng khung chung và trang chủ

Đây là màn quan trọng nhất, vì nó đặt ra bộ mặt cho toàn cửa hàng: màu sắc, kiểu chữ, kiểu nút. Dựng đúng một lần ở đây thì mọi màn sau chỉ việc noi theo.

Trước khi gõ câu đúng, hãy xem một câu viết vội sẽ hỏng thế nào.

Yêu cầu chưa tốt

Làm cho tôi trang chủ một shop bán hàng cho đẹp.

Câu này bỏ trống bốn chỗ, và AI sẽ tự lấp bằng phỏng đoán của nó:

Chỗ hỏng	AI sẽ làm gì
Không đưa ảnh mẫu	Tự chế một giao diện bất kỳ, thường không giống thứ bạn hình dung
Không nói dùng ảnh sản phẩm nào	Chèn ảnh vu vơ hoặc để ô trống
"Cho đẹp" nghĩa là gì	Mỗi lần một kiểu, không lần nào giống ý bạn
Không nói cần những phần gì	Thiếu banner, thiếu chân trang, hoặc thừa thứ bạn không cần

Cách chữa là **đưa ảnh mẫu cho AI trước, rồi mới mô tả**. Làm hai bước.

Yêu cầu tốt

Bước 1. Đưa ảnh mẫu. Trong ô nhập của khung Agent, gõ dấu @ rồi chọn ảnh `mini-shop-user-home-reference.png` (hoặc kéo ảnh vào khung Agent).



Bước 2. Gõ yêu cầu:

@mini-shop-user-home-reference.png

VIỆC: Dựng khung chung và trang chủ cho một shop bán đồ thủ công, trang trí, theo bố cục trong ảnh

NGUỒN: Ảnh mẫu tôi vừa gửi; ảnh sản phẩm trong thư mục assets của dự án

KẾT QUẢ: Trang chủ mở trên trình duyệt: thanh menu trên cùng, banner lớn, lưới vài sản phẩm nổi bật, chân trang

LƯU Ý: Bám tinh thần thiết kế trong ảnh mẫu – khoảng cách thoáng và đều, một màu nhấn

nhất quán, bo góc và bóng đổ nhẹ; dùng ảnh sản phẩm thật trong assets, không dùng ô xám giả;

không emoji trang trí, không nền gradient tím mặc định. Responsive xem được trên điện thoại;

giá dạng 290.000đ; tách thanh menu và chân trang để các trang sau dùng lại

Kiểm tra: Mở trang chủ. Có thanh menu với tên cửa hàng, banner ở đầu, và một lưới sản phẩm hiện ảnh thật cùng giá có chữ đ. Chưa ưng chỗ nào thì gõ tiếp một câu, ví dụ "banner cao hơn một chút" hoặc "đổi tông màu nút sang xanh dương".

Giữ giao diện khỏi bị nhạt

Có một điều đặc biệt về phần giao diện: khi bạn để AI tự do, nó hay cho ra những trang na ná nhau và nhạt nhẽo — nền gradient tím, khoảng cách lộn xộn, ô ảnh xám để trống, emoji rải khắp nơi. Nhìn là biết máy làm. Người trong nghề gọi hiện tượng này là **AI slop**, tạm hiểu là "đồ nhạt do AI đổ ra hàng loạt".

Phần giao diện dễ dính AI slop nhất, nên đáng để chặn ngay từ prompt. Cách chặn nằm gọn trong ba việc, và việc quan trọng nhất bạn đã làm rồi:

1. **Luôn đưa ảnh mẫu đẹp và bảo AI bám theo tinh thần của nó.** Một ảnh mẫu tử tế kéo AI ra khỏi lối mòn nhạt nhẽo. Đây là lý do mỗi màn ta đều đưa ảnh trước khi mô tả.
2. **Nói rõ vài tiêu chí chất lượng**, chính là dòng LƯU Ý ở trên: khoảng cách thoáng và đều, một màu nhấn nhất quán, dùng ảnh sản phẩm thật, chữ dễ đọc.
3. **Cấm vài thứ hay gây nhạt:** không nền gradient tím mặc định, không emoji trang trí, không ô ảnh xám giả.

Dòng LƯU Ý về chất lượng này nên có ở mọi màn, không riêng trang chủ. Ở các màn sau, bạn không phải chép lại cả đoạn — chỉ cần thêm "giữ đúng phong cách và chất lượng như các trang đã dựng" là đủ, vì AI đã thấy các trang trước. Và nhớ: giao diện đẹp là kết quả của vài vòng nhìn rồi chỉnh ("khoảng cách giữa các thẻ đều hơn", "chữ tên sản phẩm to hơn chút"), không phải một câu lệnh duy nhất.

2.4 Danh sách sản phẩm

Trang chủ chỉ khoe vài sản phẩm nổi bật. Giờ dựng một trang riêng liệt kê tất cả mặt hàng.

Bước 1. Gõ @ chọn ảnh mini-shop-product-list-reference.png.

Bước 2:



@mini-shop-product-list-reference.png

VIỆC: Dựng trang danh sách sản phẩm theo bố cục trong ảnh, hiện tất cả sản phẩm

NGUỒN: Ảnh sản phẩm trong thư mục assets; kiểu thẻ giống lưới ở trang chủ

KẾT QUẢ: Trang hiện lưới các thẻ sản phẩm, mỗi thẻ có ảnh, tên, giá, nút Xem chi tiết

LƯU Ý: Dùng lại thanh menu và chân trang đã có; giá dạng VND như 290.000đ

Kiểm tra: Các sản phẩm hiện thành lưới, ảnh không méo, giá đúng dạng tiền Việt. Ví dụ Giỏ mây đan 290.000đ, Sofa phòng khách vài triệu đồng.

2.5 Lọc theo danh mục và tìm kiếm

Vài món thì còn dễ nhìn, nhưng cửa hàng thật có hàng trăm. Thêm cho khách cách lọc và tìm.

Màn này không có ảnh mẫu riêng, nên mô tả bằng lời và bảo AI làm theo phong cách các trang đã dựng.

VIỆC: Thêm bộ lọc theo danh mục và ô tìm kiếm vào trang danh sách sản phẩm

NGUỒN: Các danh mục sẵn có trong dữ liệu; theo phong cách các trang đã dựng

KẾT QUẢ: Bấm một danh mục thì chỉ hiện sản phẩm nhóm đó; gõ tên vào ô tìm thì lọc dần theo chữ;

bấm Tất cả thì hiện lại đủ mọi sản phẩm

LƯU Ý: Không tải lại trang khi lọc, chỉ ẩn hiện sản phẩm cho mượt

Kiểm tra: Bấm một danh mục thì chỉ còn nhóm đó. Gõ "đèn" vào ô tìm thì chỉ còn các sản phẩm đèn. Bấm Tất cả thấy lại đủ.

2.6 Chi tiết một sản phẩm

Khách bấm vào một sản phẩm thì cần một trang riêng xem kỹ.

Bước 1. Gõ @ chọn ảnh mini-shop-product-detail-reference.png.

Bước 2:

@mini-shop-product-detail-reference.png

VIỆC: Dựng trang chi tiết sản phẩm theo bố cục trong ảnh

NGUỒN: Sản phẩm được chọn từ danh sách, ảnh trong thư mục assets

KẾT QUẢ: Trang hiện ảnh lớn, tên, giá, mô tả, nút Thêm vào giỏ và nút Yêu thích

LƯU Ý: Bấm một thẻ ở trang danh sách thì mở đúng trang chi tiết của sản phẩm đó

Kiểm tra: Bấm một sản phẩm ở danh sách, trang mở ra đúng ảnh và giá của sản phẩm đó, không phải món khác.

2.7 Giỏ hàng

Đây là màn đầu tiên có trí nhớ: thêm món vào giỏ, sang trang khác quay lại giỏ vẫn còn. Màn này không có ảnh mẫu, mô tả bằng lời.

VIỆC: Làm giỏ hàng cho cửa hàng

NGUỒN: Nút Thêm vào giỏ ở trang chi tiết và các thẻ sản phẩm; theo phong cách trang đã dựng

KẾT QUẢ: Bấm Thêm vào giỏ thì số trên biểu tượng giỏ tăng; mở trang giỏ thấy danh sách món,

tăng giảm số lượng được, có nút xóa và tổng tiền



LƯU Ý: Giỏ hàng phải nhớ được khi chuyển trang, dùng bộ nhớ trình duyệt để lưu tạm

Kiểm tra: Thêm hai món, số trên giỏ thành 2. Sang trang chủ rồi quay lại giỏ, hai món vẫn còn. Tăng số lượng thì tổng tiền tăng theo.

2.8 Sản phẩm yêu thích

Giống giỏ hàng nhưng để khách lưu món thích xem sau.

VIỆC: Thêm chức năng yêu thích

NGUỒN: Nút hình trái tim trên mỗi thẻ và trang chi tiết; theo phong cách trang đã dựng

KẾT QUẢ: Bấm trái tim thì nó tô đậm và món được lưu; có trang Yêu thích liệt kê món đã lưu;

bấm lại trái tim thì bỏ khỏi danh sách

LƯU Ý: Danh sách yêu thích cũng nhớ khi chuyển trang như giỏ hàng

Kiểm tra: Thả tim ba món, trang Yêu thích hiện đúng ba. Bỏ tim một món, còn hai.

2.9 Thanh toán và đặt hàng

Khách chọn xong thì đặt hàng. Ở bài này chưa có thanh toán thật, chỉ ghi nhận đơn.

VIỆC: Làm trang thanh toán và đặt hàng

NGUỒN: Các món đang có trong giỏ hàng; theo phong cách trang đã dựng

KẾT QUẢ: Trang có form họ tên, số điện thoại, địa chỉ, phần tóm tắt đơn và tổng tiền;

bấm Đặt hàng thì hiện thông báo đặt thành công và làm trống giỏ

LƯU Ý: Chưa cần thanh toán tiền thật, chỉ ghi nhận đơn và báo thành công

Kiểm tra: Điền form, bấm Đặt hàng, hiện thông báo cảm ơn, giỏ hàng về 0.

2.10 Đăng nhập và đăng ký

Cửa hàng cần phân biệt khách với người bán. Ở bài này ta làm đăng nhập **giả**: chưa nói máy chủ thật, chỉ dựng sẵn màn hình và luồng đi. Đăng nhập thật để dành tới bài 6.

VIỆC: Làm trang đăng nhập và đăng ký giả

NGUỒN: Theo phong cách và màu của cửa hàng đã dựng

KẾT QUẢ: Có trang đăng nhập và đăng ký; đăng nhập tài khoản thường thì vào trang khách,

tài khoản admin thì vào khu quản trị

LƯU Ý: Đây là đăng nhập giả chưa nói máy chủ, chỉ cần luồng đi đúng

Kiểm tra: Đăng nhập tài khoản thường thì về trang chủ. Đăng nhập tài khoản admin thì mở khu quản trị ở màn sau.

2.11 Khu quản trị cho người bán

Phần khách đã xong. Giờ dựng khu riêng cho người bán, gồm ba màn. Có hai ảnh mẫu cho khu này, đưa lần lượt cho AI.



Bước 1. Gõ @ chọn ảnh mini-shop-admin-dashboard-reference.png (và có thể kèm luôn mini-shop-admin-management-reference.png).

Bước 2:

@mini-shop-admin-dashboard-reference.png @mini-shop-admin-management-reference.png
VIỆC: Dựng khu quản trị theo bố cục hai ảnh mẫu này
NGUỒN: Ảnh sản phẩm và đơn hàng đã có
KẾT QUẢ: Màn Tổng quan có vài con số thống kê; màn Quản lý sản phẩm là bảng các sản phẩm
có nút thêm sửa xóa; màn Quản lý đơn hàng liệt kê đơn đã đặt
LƯU Ý: Theo bố cục ảnh nhưng dùng dữ liệu đồ thủ công, giá VND; chưa cần lưu thật,
sửa xong hiện ngay trên màn là được

Kiểm tra: Vào khu quản trị thấy có mấy con số thống kê và một bảng sản phẩm. Thêm thử một sản phẩm thì nó hiện thêm vào bảng.

2.12 Nhìn lại: chỗ này bắt đầu đuối

Dừng lại nhìn thứ vừa dựng. Bạn có một cửa hàng đủ màn, chạy được, đẹp. Đó là thành quả thật.

Nhưng nếu để ý, bạn sẽ thấy vài chỗ bắt đầu nặng nề.

Thanh menu và chân trang bị lặp. Mỗi trang mới, AI phải chép lại nguyên khối menu và chân trang. Muốn đổi một chữ trên menu, bạn phải sửa ở mọi trang. Cửa hàng càng nhiều trang, việc này càng khổ.

Trí nhớ chỉ là tạm bợ. Giỏ hàng và yêu thích lưu trong bộ nhớ trình duyệt, tức là trong máy của đúng người khách đó. Đổi máy khác mở lên là mất. Người bán cũng không nhìn thấy đơn của khách, vì đơn chỉ nằm trong máy khách.

Sửa một chỗ dễ vỡ chỗ khác. Khi mọi thứ nằm rải rác trong các file HTML lặp nhau, một thay đổi nhỏ dễ kéo theo lỗi ở nơi bạn không ngờ.

Những cái đuối này không phải do bạn làm sai. Chúng là giới hạn tự nhiên của cách dựng bằng HTML, CSS, JavaScript thuần cho một trang web lớn. Bài 3 sẽ chỉ cho bạn một công cụ dựng gọn hơn để gỡ đúng những cái đuối này, và bạn sẽ tự tay chuyển cửa hàng sang đó.

Tóm tắt bài

- Cửa hàng gồm nhiều màn hình, dựng từng màn một theo thứ tự, phần khách trước phần quản trị sau
- Cách dựng mỗi màn: đưa ảnh mẫu cho AI bằng dấu @, rồi mô tả theo bốn phần VIỆC, NGUỒN, KẾT QUẢ, LƯU Ý
- Màn nào không có ảnh mẫu thì mô tả bằng lời và bảo AI làm theo phong cách các trang đã dựng
- Không cần AI làm đúng từng chi tiết ngay; dựng thô rồi chỉnh dần bằng những câu prompt ngắn



- Giao diện dễ bị nhạc kiểu AI (AI slop); tránh bằng cách luôn đưa ảnh mẫu đẹp, nêu tiêu chí chất lượng, và cấm vài thứ gây nhạc (gradient tím, emoji trang trí, ô ảnh giả)
- Giỏ hàng và yêu thích nhớ được nhờ lưu tạm trong bộ nhớ trình duyệt, nhưng đó chỉ là cách tạm
- Cách dựng bằng HTML, CSS, JavaScript thuần bắt đầu đuoối khi cửa hàng lớn dần, đó là lý do bài 3 đổi công cụ



BÀI 3: CHUYỂN CỬA HÀNG SANG NEXT.JS

Mục tiêu bài học:

- Hiểu vì sao HTML, CSS, JavaScript thuần đuối dần khi web lớn lên
- Biết framework là gì, Next.js là gì, và khái niệm **component** giải quyết điều gì
- Kiểm kê được cửa hàng bằng một prompt rà soát trước khi chuyển
- Biết nhờ Agent tự cài công cụ còn thiếu trên máy, không phải cài tay
- Tổ chức thư mục dự án gọn gàng: bản mới nằm riêng, không trộn với bản cũ
- Biết bắt AI lên kế hoạch cho việc lớn, đọc duyệt kế hoạch rồi mới cho làm
- Chuyển xong cửa hàng sang Next.js và nghiệm thu bằng đúng bằng kiểm kê

3.1 Nhắc lại ba chỗ đuối

Cuối bài trước, bạn đã có một cửa hàng chạy được nhưng bắt đầu thấy nặng nề ở ba chỗ. Nhắc lại để thấy rõ cái ta sắp gỡ:

- **Thanh menu và chân trang bị lặp** ở mọi trang. Đổi một chữ trên menu phải sửa khắp nơi.
- **Trí nhớ chỉ là tạm bợ**, giỏ hàng nằm trong máy từng khách, người bán không thấy đơn.
- **Sửa một chỗ dễ vỡ chỗ khác**, vì mọi thứ nằm rải trong các file lặp nhau.

Bài này gỡ chỗ thứ nhất và thứ ba bằng một công cụ dựng gọn hơn. Chỗ thứ hai, chuyện dữ liệu, để dành cho bài 5.

3.2 Framework là gì, và Next.js cho bạn thứ gì

Ở bài 1 bạn đã gặp cái tên Next.js một lần, kèm lời hứa sẽ giải thích sau. Giờ là lúc đó.

Hãy nhớ lại việc dựng nhà. HTML, CSS, JavaScript giống như gạch, xi măng, dây điện rồi. Bạn dựng được nhà từ vật liệu thô đó, nhưng mỗi bức tường phải xây lại từ đầu. **Framework** là một bộ đồ nghề dựng sẵn: có khung nhà đúc sẵn, có những mảng tường lắp ghép, có đường điện đi sẵn. Bạn ghép nhanh hơn nhiều, và các mảng ghép với nhau cho khớp.

Next.js là một framework như vậy, xây bên trên JavaScript. Nó cho cửa hàng của bạn nhiều thứ, nhưng thứ quan trọng nhất, thứ gỡ đúng cái đuối của bạn, gọi là **component**.

Component: dựng một lần, dùng nhiều nơi

Một component là một mảng giao diện bạn dựng một lần rồi dùng lại ở mọi nơi cần. Thanh menu là một component. Chân trang là một component. Thẻ sản phẩm cũng là một component.



Khác biệt nằm ở đây. Ở bản cũ, mỗi trang chứa một bản sao của thanh menu. Ở Next.js, mọi trang cùng gọi tới **một** component thanh menu. Sửa component đó một lần, mọi trang đổi theo ngay. Cái khổ "đổi menu phải sửa khắp nơi" biến mất.

Bạn không phải học cách viết component. AI viết. Nhưng hiểu ý niệm này thì bạn ra lệnh đúng: thay vì nói "chép menu vào trang này", bạn nói "dùng lại component menu đã có".

3.3 Kiểm kê trước khi chuyển

Trước một việc lớn như dọn nhà, người cẩn thận đi một vòng đếm lại đồ đạc. Chuyển cửa hàng cũng vậy: kiểm kê trước, để không chuyển thiếu.

Có một điều cần biết ở đây, và biết rồi bạn sẽ tránh được nhiều bức mình về sau: **AI không nhớ các cuộc trò chuyện cũ**. Hôm nay mở chat mới, nó không nhớ ảnh bạn đưa tuần trước, không nhớ những gì hai bên đã bàn ở bài 2. Thứ nó luôn đọc được là những gì **đang nằm trong thư mục làm việc**: toàn bộ code và ảnh của cửa hàng. Nói cách khác, trí nhớ thật của dự án là file, không phải cuộc chat. Vì vậy ở bài này ta không cần đưa lại ảnh mẫu nào cả — ta trở thẳng vào code cũ, vì code chính là bản mô tả đầy đủ nhất của cửa hàng.

Mở lại thư mục **MINI-SHOP** trong Antigravity (menu **File**, chọn **Open Folder** nếu chưa mở sẵn), mở cuộc trò chuyện mới, rồi giao việc kiểm kê:

VIỆC: Rà soát cửa hàng trong thư mục này xem đã đủ các phần chưa, chưa cần sửa gì
NGUỒN: Toàn bộ code HTML, CSS, JavaScript đang có của dự án
KẾT QUẢ: Một bảng đối chiếu theo danh sách sau, mỗi mục ghi rõ có hay thiếu: trang chủ, danh sách sản phẩm, lọc và tìm kiếm, chi tiết sản phẩm, giỏ hàng, yêu thích, thanh toán, đăng nhập giả, admin tổng quan, admin sản phẩm, admin đơn hàng
LƯU Ý: Chỉ kiểm tra và báo cáo, không tự ý thêm hay sửa code

Danh sách mười một mục trong prompt chính là bảng màn hình của bài 2. Nó sẽ được dùng lại lần nữa ở cuối bài, khi nghiệm thu bản mới.

Kết quả rà soát rẽ hai nhánh:

- **Đủ cả mười một mục**: sang phần tiếp theo.
- **Thiếu mục nào đó** (chuyện bình thường nếu bạn làm bài 2 chưa trọn): bảo AI làm bù đúng theo nếp bài 2 — mô tả bốn phần, và nếu màn đó có ảnh mẫu trong thư mục `giao_dien_web` thì gõ **@** chọn ảnh. Làm xong màn thiếu, chạy lại prompt kiểm kê cho tới khi đủ.

3.4 Nhờ Agent cài Node.js

Next.js cần một công cụ nền tên **Node.js** để chạy. Bạn không cần hiểu Node.js là gì, và cũng không cần tự cài. Nguyên tắc từ giờ về sau: **việc gì Agent tự làm được trên máy, cứ để Agent làm**. Cài phần mềm nền là một việc như vậy.

Gõ vào khung Agent:



Kiểm tra máy tôi đã có Node.js chưa. Chưa có thì cài giúp tôi luôn, xong báo lại phiên bản đã cài.

Nếu máy đã có, Agent báo phiên bản và bạn đi tiếp ngay. Nếu chưa, Agent tự tải và cài.

Có thể xuất hiện: Windows hiện hộp thoại hỏi có cho phép ứng dụng thay đổi thiết bị không. Bấm **Yes**. Đây là bước duy nhất bạn phải tự bấm, vì hộp xin quyền này Windows chỉ cho con người bấm.

Kiểm tra: Agent báo lại một số phiên bản, ví dụ v22. Thấy số phiên bản là xong.

3.5 Chỗ ở cho bản mới, và bắt AI lên kế hoạch

Bản mới nằm riêng một thư mục

Trước khi cho AI bắt tay, phải trả lời một câu: code Next.js sẽ nằm ở đâu?

Câu trả lời dứt khoát: **trong một thư mục con mới, tên `mini-shop-next`, nằm ngay trong MINI-SHOP**. Toàn bộ code mới gói gọn trong đó, không một file nào trộn vào chỗ cũ. Thư mục lúc này trông như sau:

```
MINI-SHOP/  
├─ index.html, style.css, script.js... (bản cũ, giữ nguyên làm mẫu đối chiếu)  
├─ assets/ (ảnh của web, hai bản dùng chung)  
├─ giao_dien_web/ (ảnh mẫu giao diện)  
└─ mini-shop-next/ (bản Next.js mới, toàn bộ code mới nằm ở đây)
```

Tổ chức như vậy vì ba lẽ. Một, cũ và mới không trộn lẫn, nhìn tên thư mục biết ngay file thuộc bản nào. Hai, bản cũ còn nguyên bên cạnh để đối chiếu từng màn khi nghiệm thu. Ba, lỡ bản mới hỏng nặng, xóa nguyên thư mục mini-shop-next làm lại là xong, không sợ mất gì. Đây là thói quen tổ chức dự án nên giữ suốt về sau: **mỗi bản, mỗi thứ, một chỗ riêng có tên rõ ràng**.

Việc lớn thì bắt AI lên kế hoạch trước

Ở bài 1, một việc nhỏ cần một prompt. Ở bài 2, mỗi màn một prompt. Việc lần này lớn hơn hẳn: chuyển cả một cửa hàng mười một hạng mục. Với việc cỡ này có một cách làm mới: **bắt AI lên kế hoạch trước, mình đọc duyệt, rồi mới cho làm**. Giống như thuê thợ sửa cả căn nhà, không ai để thợ đập tường ngay, phải xem bản kế hoạch đã.

Trước khi gõ câu đúng, xem câu viết vội hổng ra sao.

Yêu cầu chưa tốt

Chuyển web của tôi sang Next.js.

Chỗ hỏng	AI sẽ làm gì
Không kiểm kê trước	Chuyển thiếu màn mà không ai phát hiện
Không nói code mới nằm đâu	File mới cũ trộn một chỗ, rồi không gỡ nổi
Không bắt lên kế hoạch	Lao vào code ngay, việc quá lớn, hổng giữa chừng khó lần lại



Chỗ hỏng	AI sẽ làm gì
Không nói nguồn là code cũ	Tự chế lại theo ý nó, khác hẳn bản bạn đã ứng

Kiểm kê thì bạn làm rồi. Ba chỗ còn lại, gói vào một prompt:

Yêu cầu tốt

VIỆC: Lên kế hoạch chuyển cửa hàng này sang Next.js, chưa code vội
NGUỒN: Toàn bộ code HTML, CSS, JavaScript đang có trong thư mục này
KẾT QUẢ: Một bản kế hoạch liệt kê các bước làm, danh sách trang sẽ chuyển, và những component dùng chung sẽ tách ra
LƯU Ý: Toàn bộ code mới nằm gọn trong thư mục con mini-shop-next, không đụng code cũ;
giữ nguyên giao diện như bản cũ; dùng lại thư mục assets; lên xong kế hoạch thì dừng chờ tôi duyệt

Đọc kế hoạch trước khi gật đầu

Agent trả về một bản kế hoạch nhiều bước. Bạn không cần hiểu hết chữ nghĩa kỹ thuật trong đó. Chỉ cần soi ba điều:

1. **Đủ màn chưa** — kế hoạch có nhắc đủ mười một mục trong bảng kiểm kê không.
2. **Có tách component không** — có dòng nào nói tách thanh menu, chân trang, thẻ sản phẩm thành phần dùng chung không.
3. **Có giữ đồ cũ không** — có dùng lại thư mục assets, có giữ giỏ hàng và yêu thích không.

Thiếu điều nào, gõ một câu bổ sung, ví dụ: "Bổ sung vào kế hoạch: tách thanh menu và chân trang thành component dùng chung." Kế hoạch ổn rồi mới sang bước chạy.

3.6 Thực thi và nghiệm thu

Cho AI chạy theo kế hoạch bằng một câu ngắn:

Kế hoạch ổn. Làm theo đúng kế hoạch, xong bước nào báo bước đó.

Đây là lượt chạy dài nhất từ đầu sách tới giờ. Agent sẽ tạo thư mục mini-shop-next, tự cài các gói cần thiết, rồi dựng dần từng trang theo kế hoạch — có thể mất mười tới hai mươi phút. Cứ để nó chạy, đừng ngắt giữa chừng. Nếu nó dừng lại hỏi, đọc câu hỏi rồi trả lời cho nó làm tiếp. Xong việc, nó mở bản xem trước ở địa chỉ dạng localhost:3000.

Nghiệm thu bằng đúng bảng kiểm kê

Mở bản mới lên và đi lại đúng danh sách mười một mục đã dùng ở đầu bài: trang chủ, danh sách, lọc và tìm, chi tiết, giỏ hàng, yêu thích, thanh toán, đăng nhập giả, ba màn admin. Mỗi mục bấm thử vài cái: thêm món vào giỏ xem số có tăng, thả tim xem trang yêu thích có nhận, đặt thử một đơn.



Muốn chắc tay hơn, mở bản cũ và bản mới cạnh nhau, so từng màn. Hai bản phải trông như nhau; khác biệt nằm ở bên trong code, không nằm trên màn hình.

Có màn nào hỏng hoặc thiếu, chỉ đích danh cho Agent: "Trang giỏ hàng chưa cộng tổng tiền, sửa lại." Xong hết danh sách, cuộc chuyển nhà kết thúc.

3.7 Nhìn lại: bạn được gì sau khi chuyển

Đứng lùi một bước nhìn thành quả.

Cửa hàng trông vẫn như cũ với khách, nhưng bên trong đã khác hẳn. Thanh menu giờ là **một** component. Thử bảo Agent "đổi tên cửa hàng trên menu thành Mini Shop Decor", bạn sẽ thấy nó sửa đúng một chỗ và mọi trang cùng đổi theo. Cái khổ "sửa khắp nơi" đã hết. Thử xong, bảo nó đổi lại tên cũ.

Bạn cũng vừa học được cách làm việc mới với AI, đáng giá hơn cả bản Next.js: **việc lớn thì kiểm kê trước, bắt lên kế hoạch, duyệt rồi mới cho chạy, và nghiệm thu bằng đúng danh sách đã kiểm kê.** Cách này dùng lại được cho mọi việc lớn về sau, ở bất kỳ công cụ nào.

Còn một chỗ đuối chưa gỡ: dữ liệu vẫn là tạm bợ. Giỏ hàng vẫn nằm trong máy từng khách, sản phẩm vẫn ghi cứng trong code chứ chưa nằm ở một kho chung. Bài 4 lo chuyện cất giữ và chia sẻ công việc cho an toàn, rồi bài 5 mới đưa dữ liệu thật vào.

Tóm tắt bài

- Trí nhớ thật của dự án là file trong thư mục làm việc, không phải cuộc chat; giao việc lớn thì trở AI vào code, không cần đưa lại ảnh
- Trước việc lớn phải kiểm kê: một prompt rà soát theo danh sách cố định, thiếu gì bổ sung trước
- Việc gì Agent tự làm được trên máy thì để Agent làm, kể cả cài Node.js; bạn chỉ bấm hộp xin quyền của Windows
- Code bản mới nằm gọn trong thư mục con riêng, không trộn với bản cũ; mỗi thứ một chỗ có tên rõ ràng
- Việc lớn thì bắt AI lên kế hoạch, đọc duyệt ba điều (đủ màn, tách component, giữ đồ cũ) rồi mới cho chạy
- Nghiệm thu bằng đúng bảng kiểm kê ban đầu, so bản mới với bản cũ từng màn
- Component giúp sửa một chỗ, mọi trang đổi theo; chuyển dữ liệu thật để dành bài 5



BÀI 4: CẤT GIỮ CÔNG SỨC VÀ TRANG BỊ THÊM NGHỀ CHO AI

Mục tiêu bài học:

- Hiểu GitHub là gì và vì sao dự án nào cũng nên nằm trên đó
- Tạo được tài khoản GitHub
- Nối được máy tính với GitHub, và tự kiểm tra đã nối thành công
- Tạo được một kho chứa trên GitHub, hiểu từng ô cấu hình
- Biết thứ gì không được phép đưa lên mạng
- Đẩy được code lên kho bằng cách chép lệnh GitHub đưa cho AI
- Tập được thói quen lưu mốc sau mỗi buổi làm việc
- Hiểu skill là gì và cài được hai bộ skill bằng prompt

4.1 Công sức của bạn đang mong manh

Hãy làm một phép thử tưởng tượng. Tối nay máy tính của bạn hỏng ổ cứng. Sáng mai, cửa hàng bạn dựng suốt ba bài vừa qua còn lại gì?

Câu trả lời là không còn gì. Toàn bộ code nằm trong đúng một chiếc máy. Mất máy là mất trắng, và không ai muốn dựng lại từ đầu lần thứ hai.

Còn một nỗi lo thứ hai kín đáo hơn. Càng về sau, mỗi lần bạn nhờ AI sửa lớn, luôn có một xác suất nhỏ là nó làm hỏng thứ đang chạy tốt. Nếu không có cách quay về "bản hôm qua còn ngon", một lần sửa hỏng có thể phá nhiều ngày công sức.

Cả hai nỗi lo có chung một lời giải, và đó là thứ dân làm phần mềm cả thế giới đều dùng: **Git** và **GitHub**.

Hãy hình dung một cuốn **nhật ký công trình**. Mỗi khi ngôi nhà xây thêm được một phần ra hồn, người ta chụp ảnh, ghi ngày tháng và vài dòng mô tả vào nhật ký. Muốn biết tuần trước nhà trông ra sao, giở lại trang cũ. **Git** chính là cuốn nhật ký đó cho code: mỗi lần bạn "lưu mốc", nó chụp lại toàn bộ dự án kèm dòng mô tả, và bạn có thể quay về bất kỳ mốc nào.

GitHub là nơi cất cuốn nhật ký đó **trên mạng**. Máy bạn hỏng, cuốn nhật ký vẫn còn nguyên trên GitHub, tải về là tiếp tục.

Với cửa hàng của bạn, GitHub mang lại bốn thứ:

Lợi ích	Nghĩa là
Không mất trắng	Máy hỏng, code vẫn nằm trên mạng
Quay lại được	Sửa hỏng thì lùi về mốc gần nhất còn tốt
Chia sẻ được	Mời người khác xem hoặc cùng làm
Cửa ngõ lên mạng	Bài 8, dịch vụ đưa web lên mạng sẽ lấy code từ đây



Ba việc, theo đúng thứ tự này

Đưa được code lên GitHub cần ba việc, và thứ tự không đảo được:

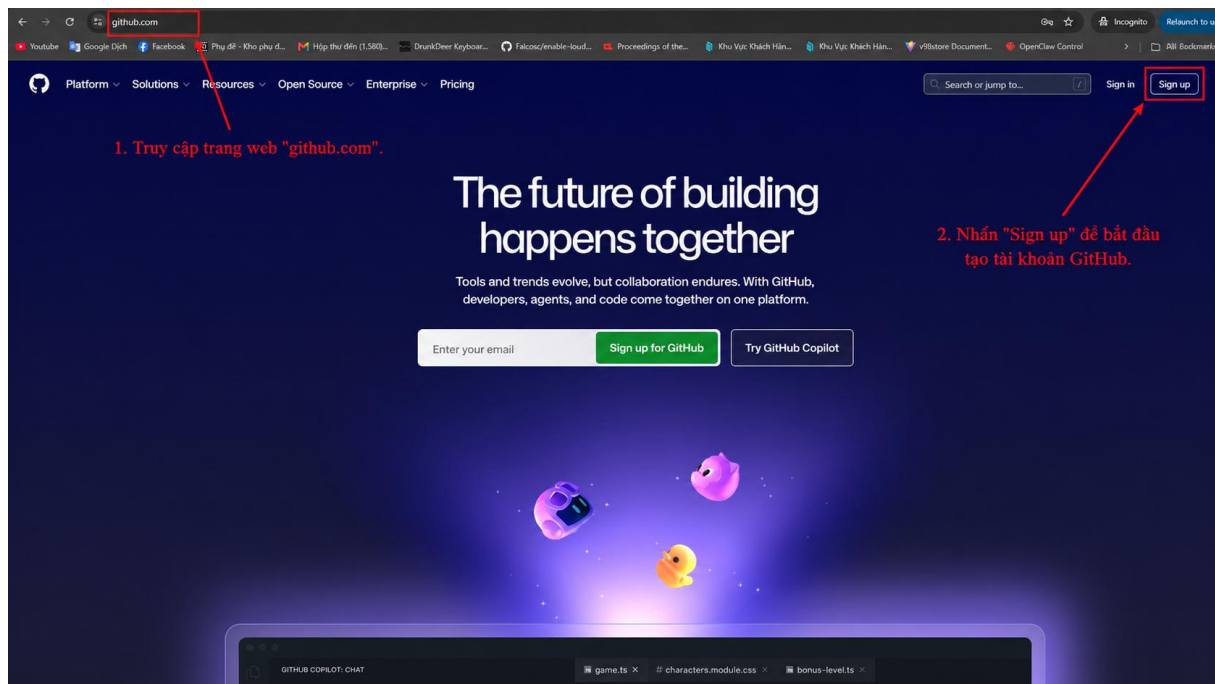
1. **Có tài khoản** trên GitHub.
2. **Nối máy tính với tài khoản đó**, để máy được phép gửi code đi.
3. **Tạo một kho chứa** rồi đẩy code vào.

Nhiều người vấp vì làm việc thứ ba trước việc thứ hai: tạo kho xong hơn hờ đẩy code, máy chưa được phép nên bị chặn, loay hoay không hiểu vì sao. Ta làm đúng thứ tự, mọi thứ trôi một mạch.

4.2 Việc thứ nhất: tạo tài khoản GitHub

Việc này làm tay, vì các dịch vụ chỉ cho con người tự đăng ký.

Mở trình duyệt, vào trang **github.com**, bấm nút **Sign up** ở góc trên bên phải.



Trang chủ GitHub, bấm Sign up

Màn đăng ký hiện ra. Điền email, đặt mật khẩu, chọn tên tài khoản. Tên tài khoản nên ngắn gọn, viết liền không dấu, vì nó sẽ xuất hiện trong địa chỉ kho code của bạn. Xong bấm **Create account**.



The screenshot shows the GitHub sign-up page. The browser's address bar shows 'www.github.com'. The page has a dark header with 'Incognito' and 'Relaunch to update' buttons. The main content area is white. At the top right, it says 'Already have an account? [Sign in](#)'. The main heading is 'Sign up for GitHub'. Below it, there are three input fields: 'Email' (containing 'pauljacker308@gmail.com'), 'Username' (containing 'pauljacker308-stack' with a green checkmark), and 'Your Country/Region' (a dropdown menu showing 'Bangladesh'). Below the country field, there is a checkbox for 'Email preferences' with the text 'Receive occasional product updates and announcements'. At the bottom, there is a large green button labeled 'Create account >'. Three red arrows point to these elements with Vietnamese text: 1. 'Kiểm tra email và nhập Username còn trống hoặc đúng định dạng.' (Check email and enter Username, empty or in correct format), 2. 'Chọn Country/Region phù hợp.' (Choose appropriate Country/Region), and 3. 'Nhấn "Create account" để tạo tài khoản.' (Click "Create account" to create account).

edings of the... Khu Vực Khách Hân... Khu Vực Khách Hân... v98store Document... OpenClaw Control >> All Bookmarks

Already have an account? [Sign in](#)

Sign up for GitHub

Email

pauljacker308@gmail.com

[Use a different Google account](#)

Username

pauljacker308-stack

Your Country/Region

Bangladesh

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Email preferences

☐ Receive occasional product updates and announcements

Create account >

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

1. Kiểm tra email và nhập Username còn trống hoặc đúng định dạng.

2. Chọn Country/Region phù hợp.

3. Nhấn "Create account" để tạo tài khoản.

Điền thông tin rồi bấm *Create account*

GitHub gửi một mã xác nhận vào email, bạn mở email chép mã dán vào là xong.

Ghi tên tài khoản và mật khẩu vào chỗ bạn vẫn cất mật khẩu quan trọng. Tài khoản này còn dùng cho hai dịch vụ nữa ở bài 5 và bài 8.

4.3 Việc thứ hai: nối máy tính với GitHub

Máy tính của bạn và tài khoản GitHub vừa tạo hiện chưa quen nhau. Phải giới thiệu hai bên một lần, gọi là **đăng nhập GitHub trên máy**. Làm một lần duy nhất, các lần sau máy tự nhớ.

Việc này Agent làm hộ gần hết. Mở Antigravity ở thư mục dự án, gõ:

VIỆC: Nối máy tôi với tài khoản GitHub

NGUỒN: Tài khoản GitHub tôi vừa tạo

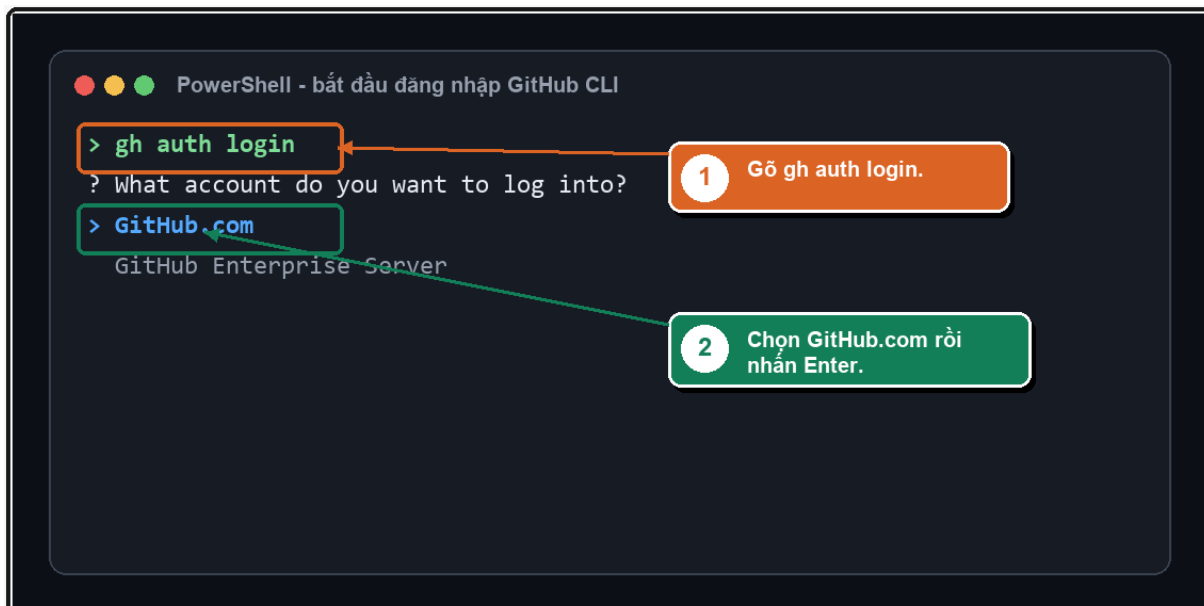
KẾT QUẢ: Máy đăng nhập được vào GitHub, sẵn sàng đẩy code lên



LƯU Ý: Máy chưa có Git hay công cụ dòng lệnh của GitHub thì cài giúp tôi luôn; tới đoạn cần tôi chọn hay bấm gì thì dừng lại chỉ rõ cho tôi

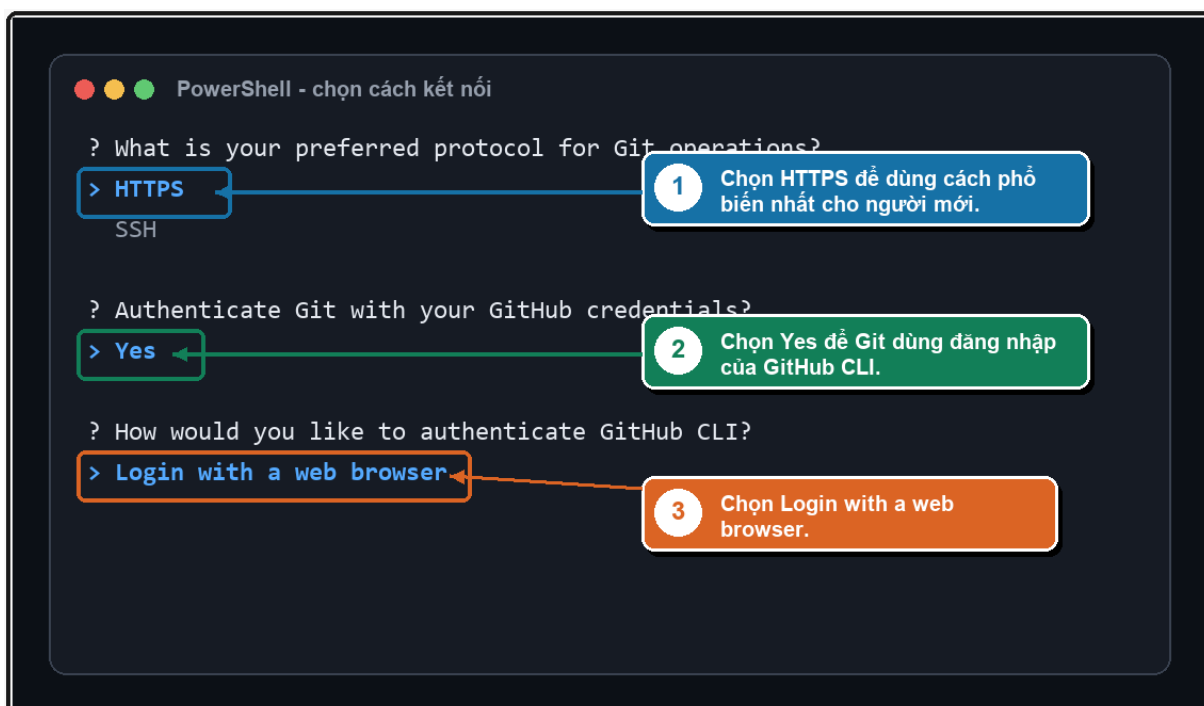
Agent cài những thứ còn thiếu (Windows hỏi xin quyền thì bấm **Yes**), rồi chạy lệnh đăng nhập. Từ đây bạn tham gia bốn bước.

Bước 1. Chọn nơi đăng nhập. Màn hình dòng lệnh hỏi bạn đăng nhập vào đâu, chọn dòng **GitHub.com** rồi nhấn **Enter**.



Chọn GitHub.com rồi nhấn Enter

Bước 2. Trả lời ba câu hỏi. Đây là chỗ quan trọng nhất của cả mục này, chọn sai thì lát nữa đầy code sẽ bị chặn. Dùng phím mũi tên lên xuống để chọn, nhấn Enter để xác nhận.



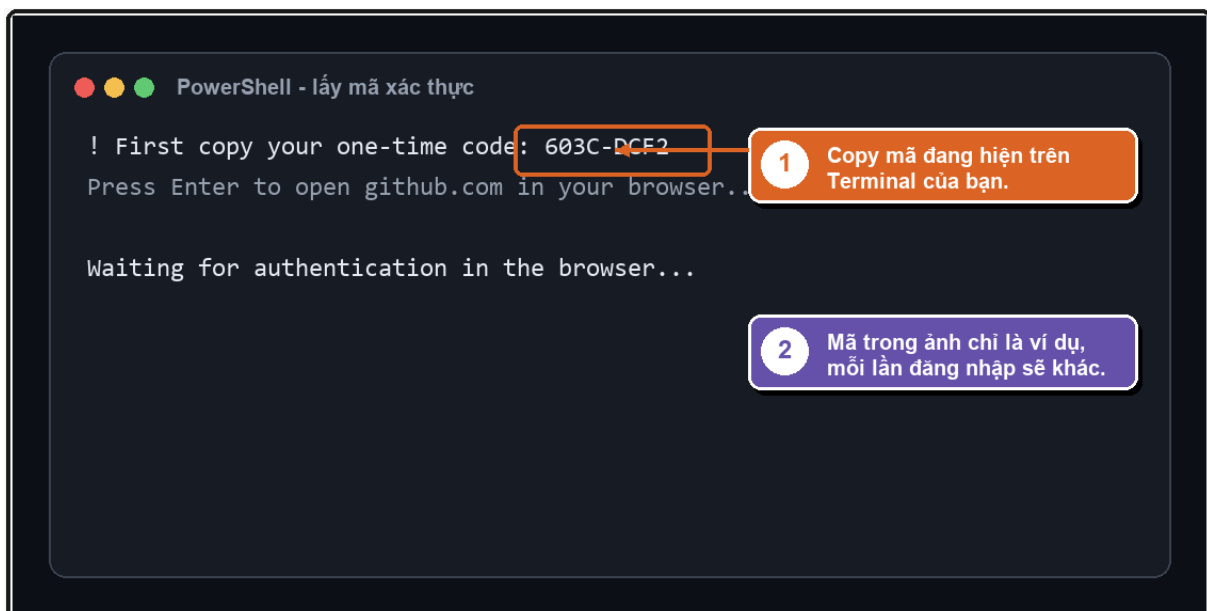
Ba câu hỏi và ba lựa chọn đúng



Câu hỏi	Chọn	Vì sao
Preferred protocol for Git operations	HTTPS	Cách phổ biến và đơn giản nhất, không phải cài thêm gì
Authenticate Git with your GitHub credentials	Yes	Cho phép máy dùng luôn thông tin đăng nhập này để đẩy code
How would you like to authenticate	Login with a web browser	Đăng nhập qua trình duyệt cho nhẹ đầu, khỏi tạo khóa thủ công

Câu thứ hai chọn **Yes** chính là thứ khiến bước đẩy code ở mục 4.6 chạy thẳng một mạch. Nhớ chọn đúng.

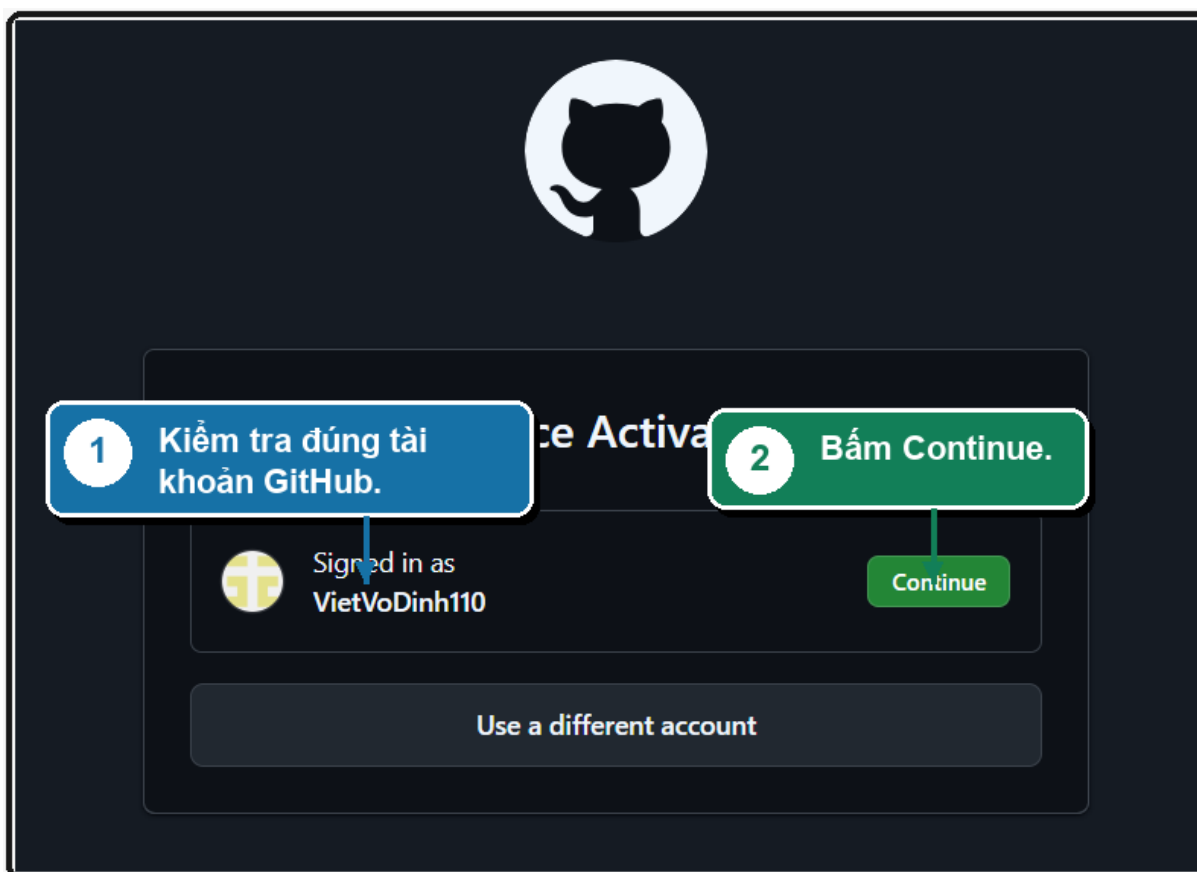
Bước 3. Lấy mã và mở trình duyệt. Màn hình dòng lệnh hiện một **mã tám ký tự** dạng 603C-DCF2. Chép mã đó lại, rồi nhấn **Enter** để trình duyệt tự mở.



Mã tám ký tự hiện trên màn hình dòng lệnh

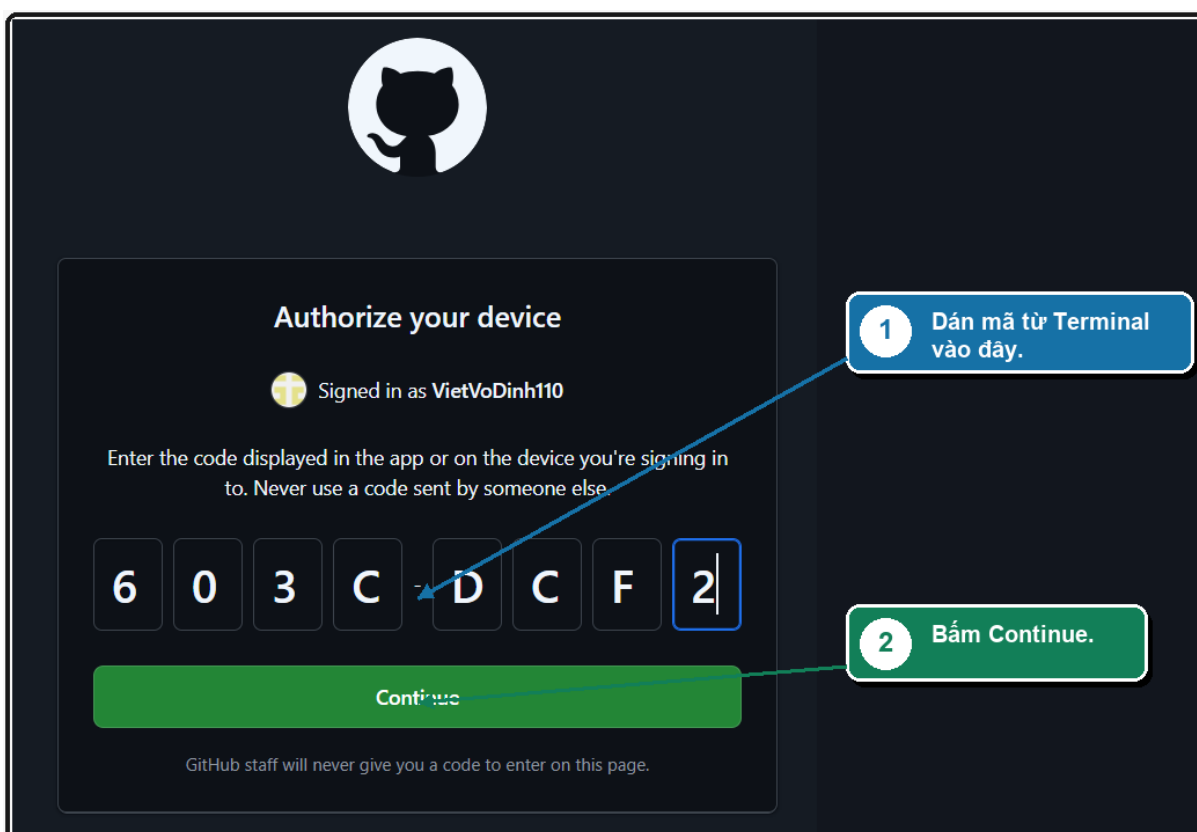
Mã này mỗi lần một khác, đừng dùng mã trong ảnh.

Bước 4. Xác nhận trên trình duyệt. Trình duyệt mở trang GitHub. Đầu tiên nó hỏi bạn đúng tài khoản chưa, kiểm tên rồi bấm **Continue**.



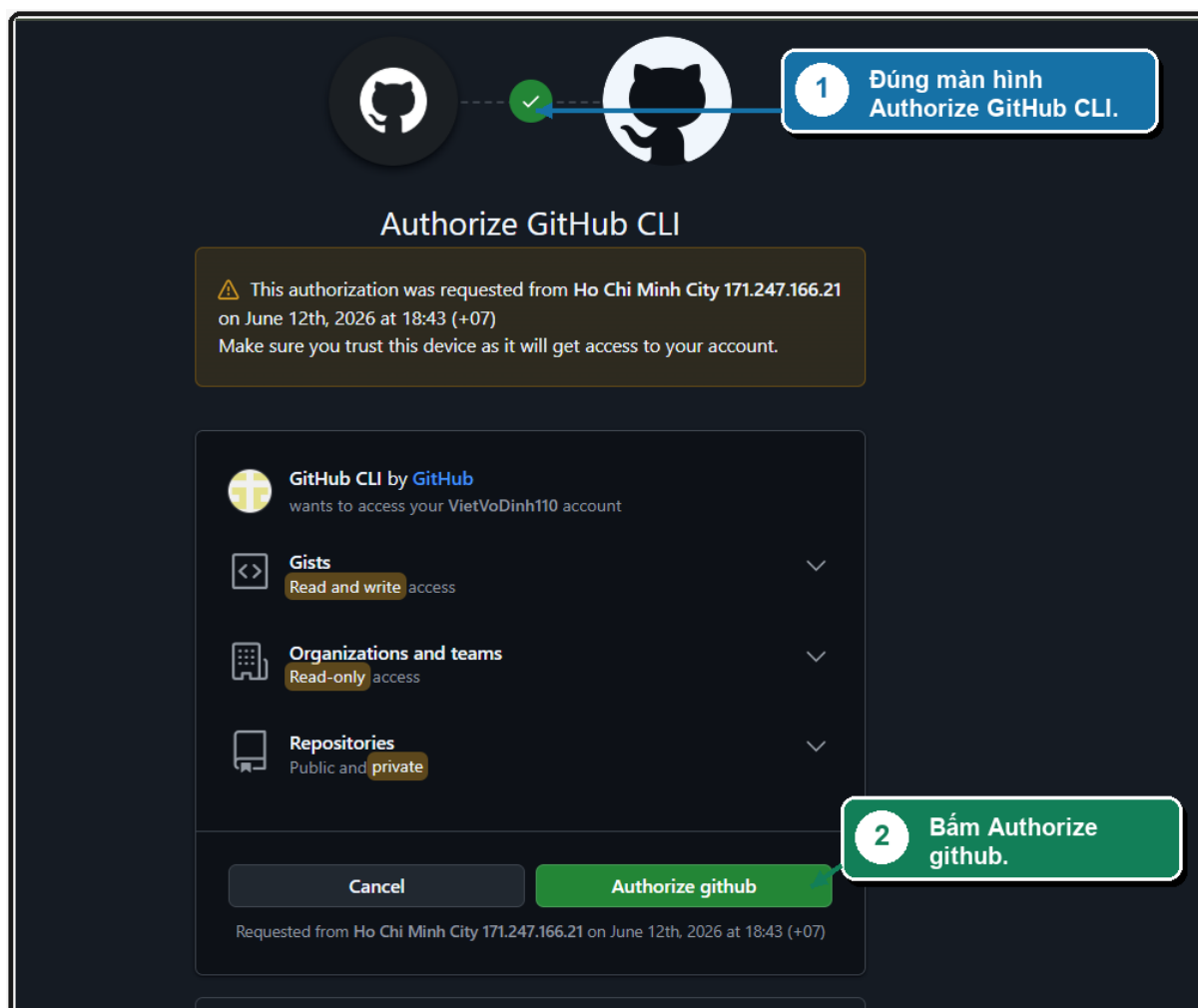
Kiểm đúng tài khoản rồi bấm Continue

Tiếp theo là ô nhập mã. Dán tám ký tự vừa chép vào rồi bấm **Continue**.



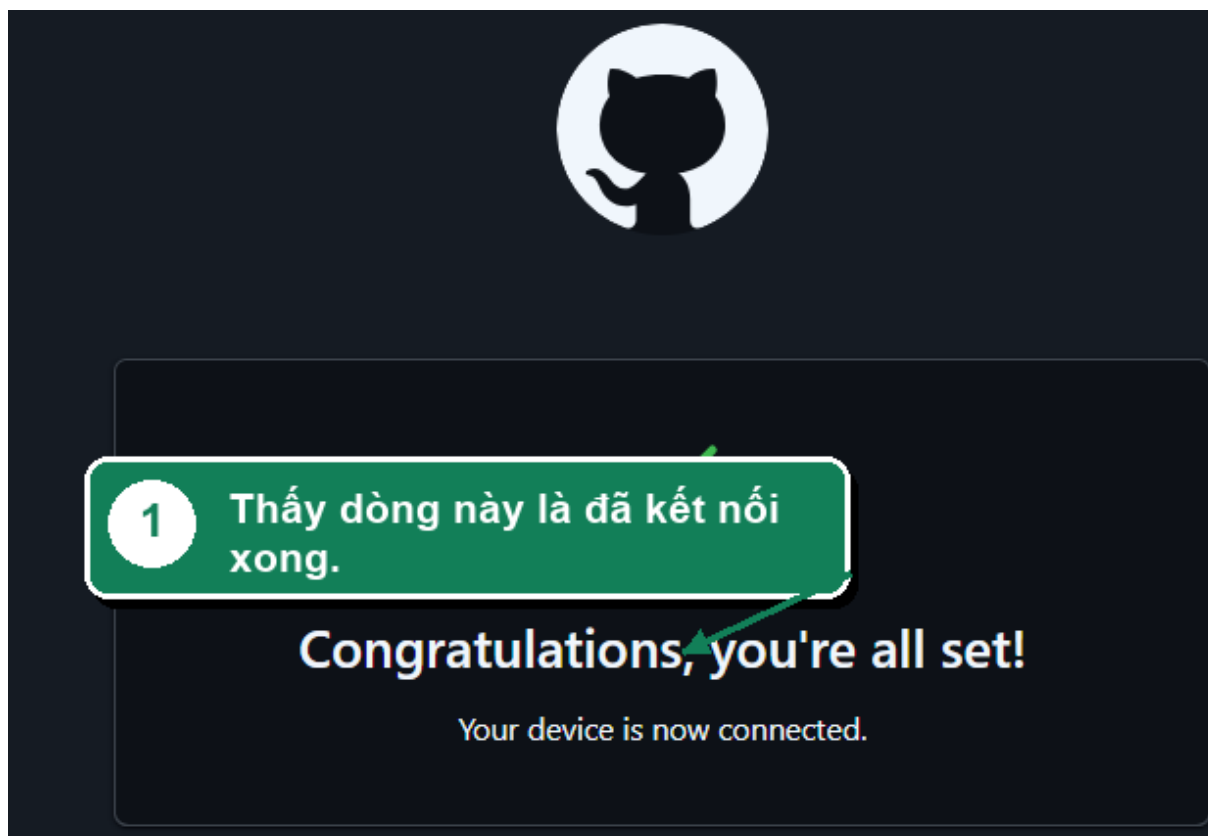
Dán mã vừa chép rồi bấm Continue

Màn cuối liệt kê những quyền mà máy bạn xin. Kéo xuống bấm nút xanh **Authorize github**.



Bấm Authorize github để cho phép

Bạn sẽ thấy: Trang báo **Congratulations, you're all set!** kèm dòng "Your device is now connected". Máy tính và tài khoản GitHub đã quen nhau.

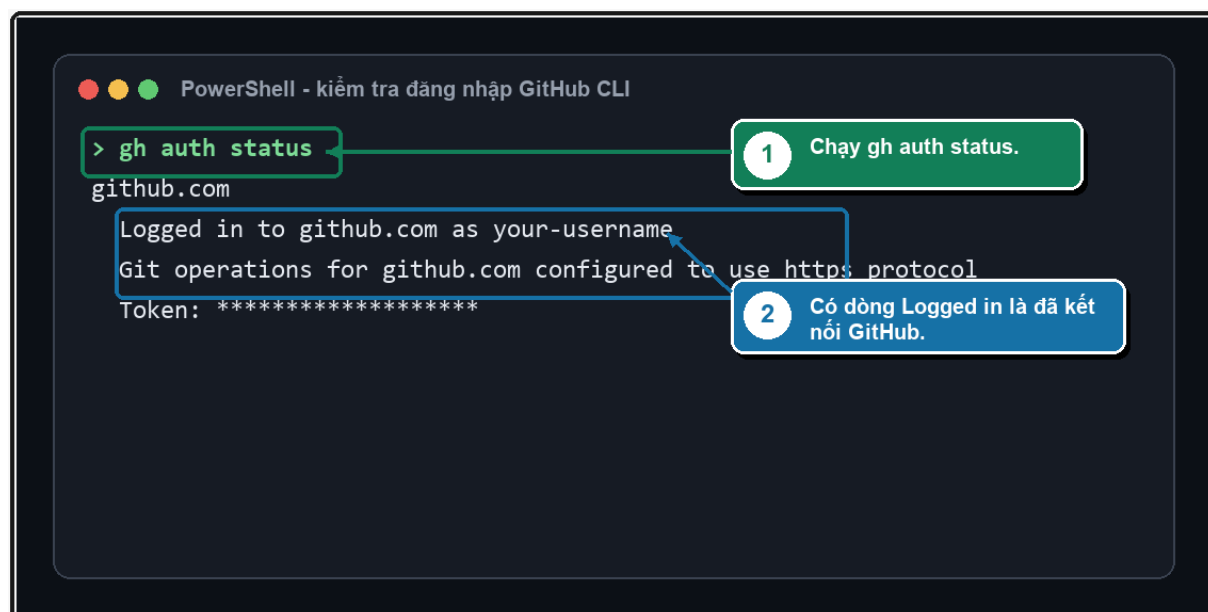


Đã kết nối thành công

Kiểm tra cho chắc. Đừng vội tin, quay về Antigravity và bảo Agent kiểm:

Kiểm tra giúp tôi máy đã đăng nhập GitHub chưa.

Kết quả đúng có hai dòng đáng nhìn: **Logged in to github.com as** kèm tên tài khoản của bạn, và **Git operations for github.com configured to use https protocol**.



Hai dòng xác nhận đã nối xong

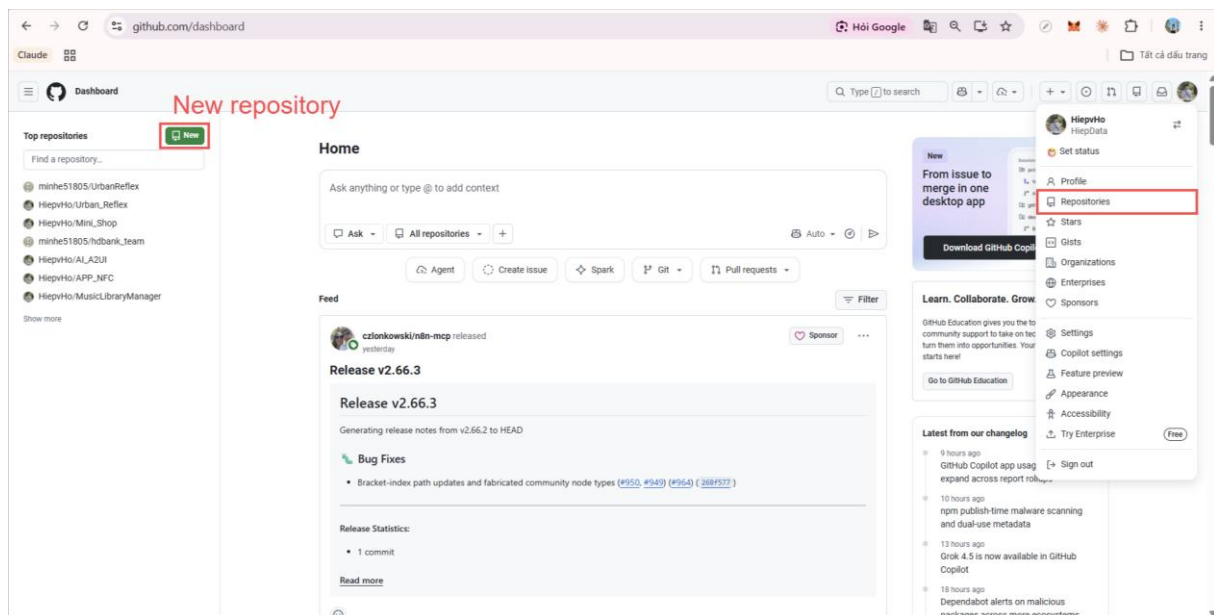


Thấy hai dòng đó là xong việc thứ hai. Giờ mới sang tạo kho.

4.4 Việc thứ ba: tạo một kho chứa

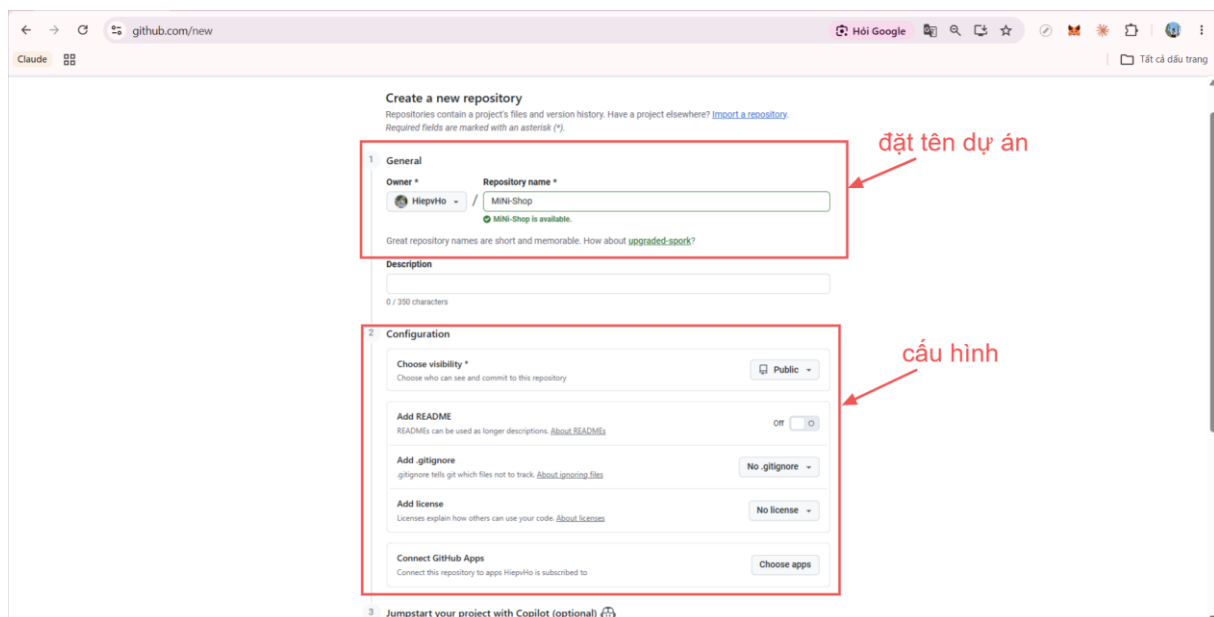
Một **kho** (repository, hay gọi tắt là repo) là chỗ chứa riêng cho một dự án. Cửa hàng của bạn sẽ có một kho mang tên nó.

Bước 1. Bấm tạo kho mới. Ở trang chủ github.com, cột bên trái có nút **New** màu xanh lá. Bấm vào đó.



Bấm nút New để tạo kho mới

Bước 2. Đặt tên và chọn cấu hình. Trang tạo kho hiện ra với vài ô. Nhìn thì nhiều nhưng chỉ có hai chỗ đáng quan tâm.



Đặt tên kho và chọn cấu hình



Ô **Repository name** là tên kho, gõ **mini-shop**. Tên viết liền, không dấu, dùng gạch ngang thay dấu cách. GitHub sẽ báo một dòng xanh nếu tên còn trống chỗ.

Bên dưới là phần cấu hình, gồm bốn ô:

Ô	Nghĩa	Chọn gì
Choose visibility	Public là ai cũng xem được, Private là chỉ mình bạn	Tùy bạn, xem giải thích ngay dưới
Add README	Tạo sẵn một file giới thiệu dự án	Để tắt, dự án của bạn đã có file riêng
Add .gitignore	Tạo sẵn danh sách những thứ không đẩy lên	Để No .gitignore , dự án của bạn đã có sẵn
Add license	Giấy phép cho người khác dùng lại code	Để No license

Về chọn Public hay Private, cả hai đều miễn phí và đều đưa web lên mạng được ở bài 8:

- **Public**: ai có đường link cũng xem được code. Tiện khi bạn muốn khoe sản phẩm hoặc gửi cho người khác xem cách mình làm.
- **Private**: chỉ mình bạn thấy. Yên tâm hơn khi dự án còn dở dang.

Chọn kiểu nào cũng được, và đổi qua lại bất cứ lúc nào trong phần cài đặt của kho.

Xong thì kéo xuống cuối trang, bấm nút **Create repository**.

Bước 3. Trang hướng dẫn hiện ra. GitHub đưa bạn tới một trang có sẵn vài khối lệnh. Đừng lo, bạn không phải hiểu chúng.

The screenshot shows the GitHub repository creation page. It includes sections for 'Start coding with Codespaces', 'Add collaborators to this repository', and 'Quick setup'. The 'Quick setup' section provides instructions for setting up the repository on the command line, with two examples: one for creating a new repository and one for pushing an existing repository. Both examples are enclosed in red boxes with copy icons.

Quick setup – if you've done this kind of thing before

Set up in Desktop or HTTPS SSH `git@github.com:HiiepVo/Mini-Shop.git`

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# Mini-Shop" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:HiiepVo/Mini-Shop.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin git@github.com:HiiepVo/Mini-Shop.git
git branch -M main
git push -u origin main
```

Trang hướng dẫn với hai khối lệnh và nút chép



Nhìn xuống hai khối lệnh trong khung đỏ, mỗi khối có một **nút chép** ở góc phải. Đây chính là thứ bạn sẽ đưa cho AI ở mục 4.6. Cứ để nguyên trang này đó, chưa cần bấm gì.

4.5 Có những thứ không được lên mạng

Trước khi đẩy code đi, phải biết một chuyện, vì lỡ rồi thì khó thu hồi.

Trong một dự án luôn có vài thứ **không được** đưa lên kho:

- **File chứa khóa bí mật.** Bài 5 bạn sẽ có một file tên bắt đầu bằng `.env`, bên trong là chìa khóa vào kho dữ liệu của bạn. Ai cầm được chìa khóa đó thì đọc và sửa được dữ liệu cửa hàng.
- **Thư mục rác máy tự sinh.** Chúng nặng hàng trăm megabyte, máy nào cũng tự tạo lại được, đẩy lên chỉ tổ chật.

Git có một danh sách tên là `.gitignore`, hiểu là "danh sách ở nhà". Tên nào ghi trong đó thì Git bỏ qua, không bao giờ đẩy lên. Dự án Next.js sinh ra ở bài 3 đã có sẵn danh sách này, và các công cụ hiện nay đủ khôn để tự xếp file `.env` vào đó.

Dù vậy, giữ đúng nếp của cuốn sách: **không tin lời hứa, tự nhìn cho chắc**. Lát nữa đẩy code xong, bạn sẽ mở kho trên GitHub kiểm bằng mắt.

4.6 Đẩy code lên kho

Ba việc chuẩn bị đã xong: có tài khoản, máy đã nối, kho đã tạo. Giờ đẩy code, và cách làm đơn giản đến bất ngờ: **chép cả hai khối lệnh ở trang GitHub, dán vào khung Agent, kèm một câu dặn.**

Trước khi gõ câu đúng, xem một câu viết vội hồng ra sao.

Yêu cầu chưa tốt

Lưu code của tôi lên mạng.

Chỗ hồng	AI sẽ làm gì
Không đưa lệnh GitHub vừa cho	Đoán địa chỉ kho, đoán sai là đẩy vào chỗ không tồn tại
Không nói thư mục nào	Đẩy nhầm thư mục cha, bài 8 đưa web lên mạng sẽ hồng
Không nói chọn khối lệnh nào	Chạy nhầm khối, báo lỗi giữa chừng
Không nhắc kiểm file bí mật	Bạn không biết chắc file <code>.env</code> có bị đẩy đi hay không

Yêu cầu tốt

Ở trang GitHub, bấm nút chép của khối thứ nhất, sang Antigravity dán vào ô nhập. Quay lại bấm nút chép khối thứ hai, dán tiếp xuống dưới. Rồi gõ thêm phần yêu cầu:

VIỆC: Đẩy dự án mini-shop-next lên kho GitHub tôi vừa tạo
NGUỒN: Hai khối lệnh GitHub hướng dẫn, tôi dán ở cuối tin nhắn



KẾT QUẢ: Mở kho trên github.com thấy đủ code của dự án

LƯU Ý: Chạy trong thư mục mini-shop-next, không phải thư mục cha; tự chọn khối lệnh đúng với tình trạng dự án; kiểm tra file .env không bị đẩy lên <dán hai khối lệnh vào đây>

Hai điều trong phần LƯU Ý đáng để ý, vì mỗi điều bịt một cái bẫy quen thuộc:

- **Chạy trong `mini-shop-next`**, vì đó mới là dự án web. Đẩy nhầm thư mục cha thì bài 8 dịch vụ đưa web lên mạng sẽ không nhận ra đây là dự án Next.js.
- **Tự chọn khối lệnh đúng**: khối thứ nhất dành cho thư mục chưa từng dùng Git, khối thứ hai dành cho thư mục đã dùng rồi. Bạn không cần biết dự án mình thuộc loại nào, AI xem là biết.

Lần này Agent chạy một mạch không dừng lại hỏi gì, vì máy đã nối với GitHub từ mục 4.3.

Kiểm tra: Mở trang kho trên github.com, tải lại trang. Danh sách file của dự án hiện ra thay cho trang hướng dẫn.

The screenshot shows the GitHub repository page for 'website-ban-hang'. The repository is public and has 1 branch and 0 tags. The file list shows the following files and folders: public, src, .gitignore, README.md, eslint.config.js, index.html, package-lock.json, package.json, and vite.config.js. The README file is selected, showing the title 'React + Vite' and the content: 'This template provides a minimal setup to get React working in Vite with HMR and some ESLint rules. Currently, two official plugins are available: @vitejs/plugin-react (Uses Babel for Fast Refresh) and @vitejs/plugin-react-swc (Uses SWC for Fast Refresh)'. On the right side, there are sections for Release, Packages, Contributors, and Languages. The Languages section shows JavaScript at 92.1% and HTML at 7.9%.

1. Đây là repository GitHub của dự án sau khi đã push code frontend lên.

2. Các thư mục và file như public, src, package.json... đang nằm trực tiếp ở thư mục gốc của repo.

3. Bước tiếp theo có thể là sắp xếp lại source code vào thư mục frontend để project gọn hơn.

Kho đã có đủ code của dự án

Rà mắt một lượt: **không được thấy file nào tên `.env`**. Nếu lỡ thấy, gõ ngay cho Agent: "File .env đang nằm trên GitHub, gỡ nó ra khỏi kho và chặn lại." Không thấy là ổn, công sức của bạn đã nằm an toàn trên mạng.



4.7 Thói quen lưu mốc

Nhật ký chỉ quý khi được ghi đều. Từ giờ, kết thúc mỗi buổi làm việc, hoặc ngay trước một lần sửa lớn, gõ một câu:

Lưu mốc lên GitHub, ghi chú ngắn gọn những gì vừa làm.

Agent gom các thay đổi, ghi một dòng mô tả, đẩy lên. Mỗi mốc như một trang nhật ký có ảnh chụp: sau này lỡ hỏng, bạn nói "quay về mốc gần nhất còn chạy tốt" là cứu được. Cả cuốn sách còn lại sẽ nhắc bạn lưu mốc ở những chỗ quan trọng, nhưng tốt nhất là để nó thành phản xạ.

4.8 Skill: trang bị thêm nghề cho AI

Chuyển sang chuyện thứ hai của bài này.

AI trong Antigravity giống một người thợ đa năng mới tới làm. Rất giỏi, nhưng chưa thuộc nề nếp riêng của bạn. Vì vậy suốt bài 2 và 3, cứ tới phần giao diện là bạn phải dẫn đi dẫn lại trong LƯU Ý: khoảng cách cho thoáng, một màu nhất quán, không trang trí lòe loẹt. Quên dặn là nó lại làm nhặt.

Skill sinh ra để khỏi phải dẫn lại. Một skill là một **cẩm nang nghề** soạn sẵn, cài vào dự án cho AI đọc. Cài rồi, mỗi lần làm việc liên quan, AI tự giờ cẩm nang ra làm theo, như người thợ đã thuộc lòng sổ tay của xưởng. Lời dặn từ chỗ nằm trong trí nhớ của bạn chuyển sang nằm cố định trong dự án.

Cuốn sách này dùng hai bộ:

- **frontend-design**, cẩm nang thiết kế giao diện: cách chọn màu, cỡ chữ, khoảng cách, bố cục cho ra sản phẩm nhìn chuyên nghiệp. Đây là tấm khiên dài hạn chống kiểu giao diện nhặt của AI.
- **superpowers**, cẩm nang nề nếp làm việc: gặp việc lớn thì lên kế hoạch trước, làm xong thì tự soát lại. Chính là những thói quen bạn học ở bài 3, giờ AI cũng thuộc.

4.9 Cài skill và dùng thử

Cài bằng prompt, đúng nguyên tắc quen thuộc:

Cài hai bộ skill frontend-design và superpowers vào dự án này giúp tôi.
Cài xong liệt kê những skill đã có để tôi biết.

Agent tự tải và cài, xong liệt kê danh sách skill. Skill cài một lần cho dự án, các buổi sau dùng luôn.

Giờ cho người thợ mới học nghề trở tài. Nhờ nó rà lại trang chủ:

VIỆC: Dùng skill frontend-design, rà lại trang chủ và cải thiện những chỗ chưa đẹp
NGUỒN: Trang chủ hiện có của mini-shop-next
KẾT QUẢ: Trang chủ sau khi sửa nhìn chuyên nghiệp hơn: khoảng cách đều, cỡ chữ hợp lý, màu nhất quán
LƯU Ý: Giữ nguyên bố cục và các tính năng, chỉ tinh chỉnh phần nhìn



Kiểm tra: Mở trang chủ trước và sau khi sửa, đặt cạnh nhau. Thường bạn sẽ thấy khác biệt ở những chỗ nhỏ: chữ dễ đọc hơn, các khối thẳng hàng hơn, khoảng thờ đều hơn. Ưng thì lưu mốc; chưa ưng chỗ nào thì nói tiếp cho nó chỉnh.

Tóm tắt bài

- Code nằm trong một máy là công sức mong manh; GitHub là cuốn nhật ký công trình cất trên mạng
- Ba việc theo đúng thứ tự: tạo tài khoản, nối máy với GitHub, rồi mới tạo kho và đẩy code
- Khi nối máy, ba lựa chọn đúng là HTTPS, Yes, và Login with a web browser; chọn Yes để sau này đẩy code chạy thẳng
- Nối xong phải tự kiểm: thấy dòng Logged in to github.com kèm tên tài khoản mới là được
- Tạo kho thì đặt tên, chọn Public hay Private tùy ý, các ô còn lại để nguyên
- File bí mật như .env nằm trong danh sách .gitignore nên không bị đẩy lên; đẩy xong vẫn tự mở kho nhìn cho chắc
- Đẩy code bằng cách chép hai khối lệnh GitHub đưa, dán vào Agent kèm câu dặn
- Skill là cảm nang nghề cài vào dự án cho AI, khỏi phải dọn đi dọn lại



BÀI 5: ĐƯA DỮ LIỆU THẬT VÀO CỬA HÀNG

Mục tiêu bài học:

- Hiểu vì sao cửa hàng cần một kho dữ liệu chung, và Supabase là gì
- Tự tạo được tài khoản và dự án Supabase
- Hiểu ba đường kết nối Framework, Direct, MCP mỗi đường làm gì
- Nối xong cả ba đường trước khi bắt tay vào việc
- Nhờ AI dựng bảng và đổ dữ liệu, không phải chép dán lệnh
- Cho trang web đọc dữ liệu thật: đổi trên kho, web đổi theo

5.1 Vì sao cần một kho dữ liệu chung

Cửa hàng của bạn đang chạy đẹp, nhưng dữ liệu của nó có một điểm yếu chết người mà bạn đã biết từ bài 2: **mọi thứ đều tạm bợ**.

Danh sách sản phẩm đang ghi cứng trong code. Muốn đổi giá một món, phải sửa code. Giỏ hàng và đơn đặt nằm trong trình duyệt của từng khách; khách đóng máy là mất, còn người bán thì tuyệt nhiên không nhìn thấy đơn nào.

Hãy hình dung một chuỗi cửa hàng có nhiều quầy. Nếu mỗi quầy giữ một cuốn sổ riêng thì giá cả lệch nhau, đơn hàng thất lạc. Cách đúng là đặt **một cuốn sổ cái ở văn phòng trung tâm**: mọi quầy cùng đọc một bảng giá, mọi đơn hàng cùng ghi về một chỗ.

Cuốn sổ cái đó, trong phần mềm, gọi là **cơ sở dữ liệu** (database, chữ DB bạn gặp ở bài 1). Nó chính là lớp "kho" trong ba lớp của một phần mềm, và là mảnh còn thiếu cuối cùng của cửa hàng.

Supabase là dịch vụ cho thuê kho dữ liệu trên mạng. Bạn không phải mua máy chủ hay cài đặt gì, chỉ cần tạo tài khoản là có một kho riêng, mức khởi đầu miễn phí và quá đủ cho cửa hàng này. Supabase còn kèm sẵn hệ thống đăng nhập và quản lý người dùng, thứ bạn sẽ dùng ở bài 6.

Bài này đi theo ba chặng

Giống bài 4, thứ tự ở đây không đảo được:

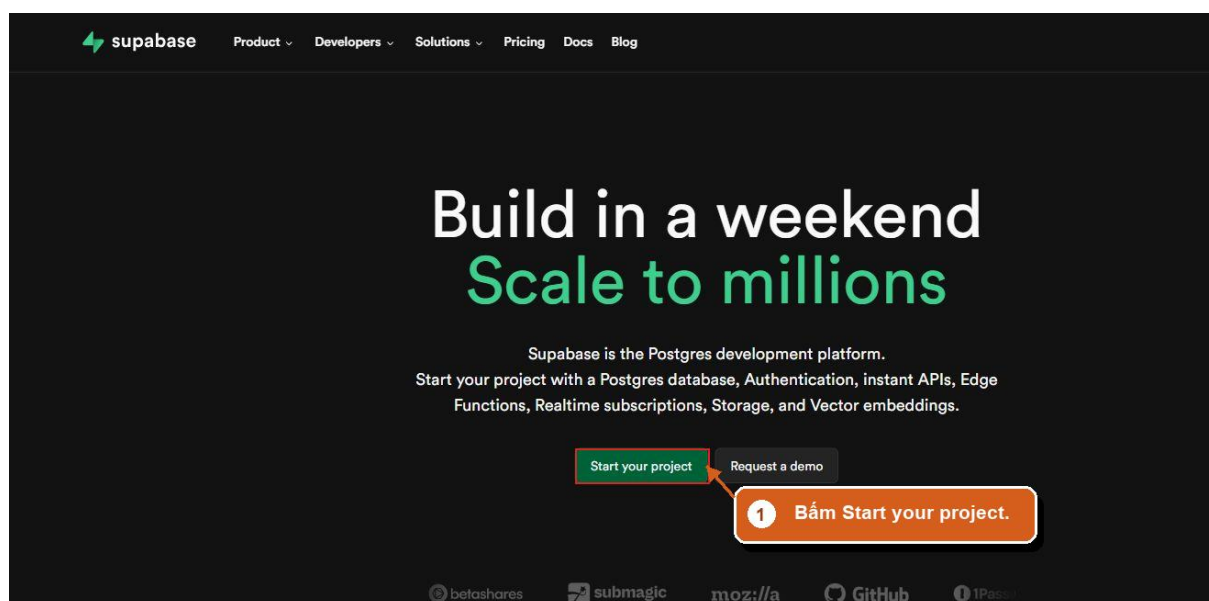
1. **Tạo tài khoản và dựng một cái kho** trên Supabase.
2. **Nối ba đường** giữa kho với máy bạn và với AI.
3. **Bắt tay vào việc**: dựng bảng, đổ dữ liệu, cho web đọc.

Ta làm xong hết chặng nối rồi mới sang chặng làm việc. Cách này gom mọi thao tác tay vào một chỗ, xong là chỉ còn gõ yêu cầu.



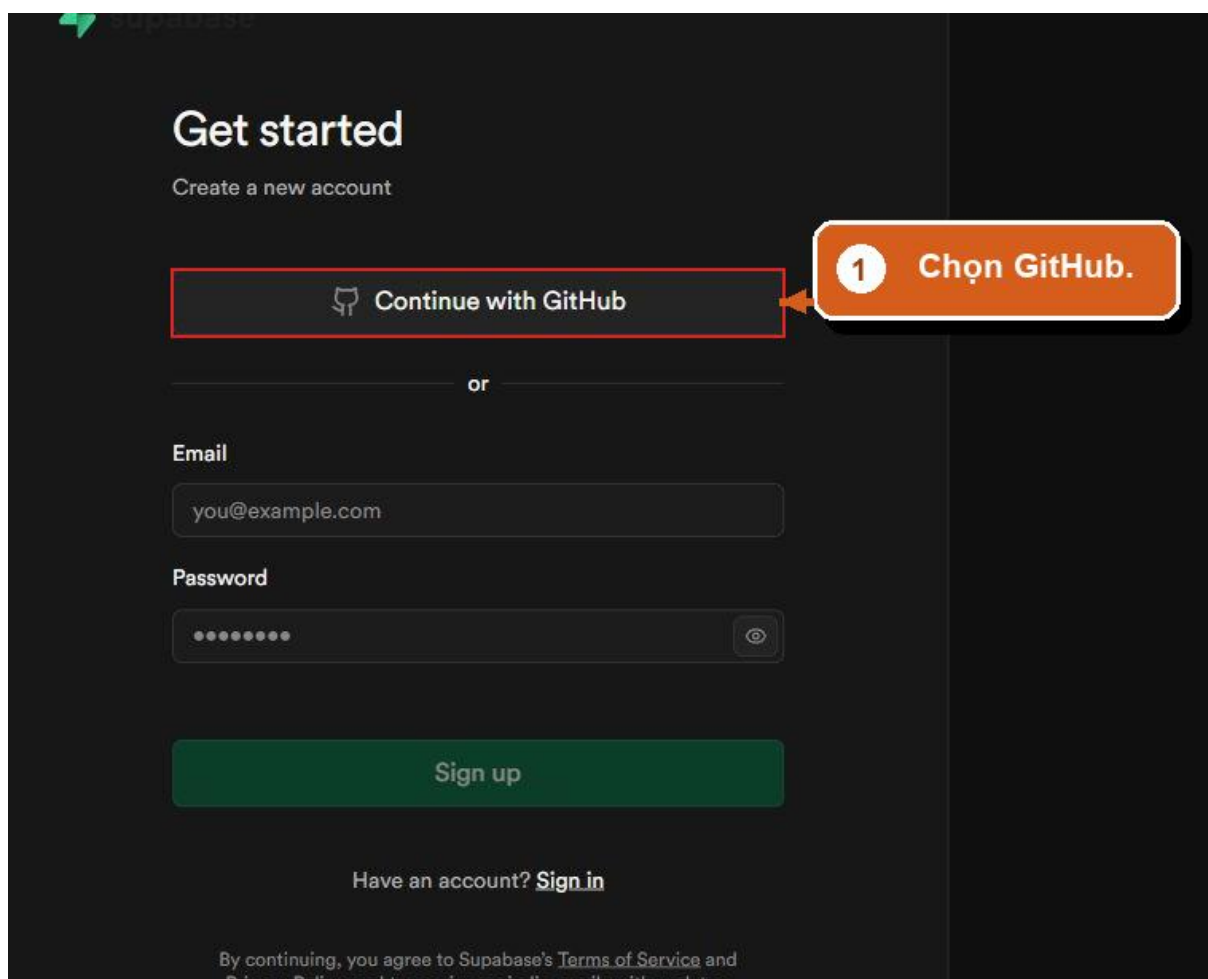
5.2 Chặng một: tạo tài khoản và dựng kho

Bước 1. Vào trang Supabase. Mở trình duyệt, vào **supabase.com**, bấm nút **Start your project**.



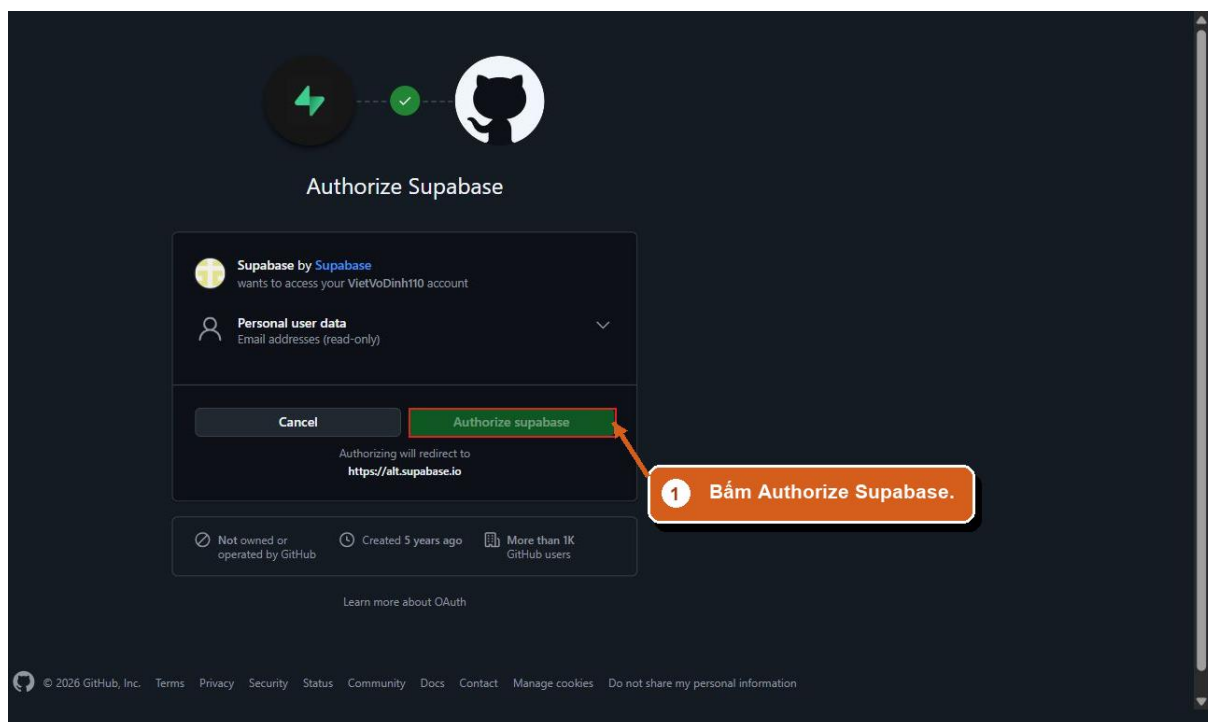
Trang chủ Supabase, bấm Start your project

Bước 2. Đăng nhập bằng GitHub. Trang đăng nhập hiện ra, chọn **Continue with GitHub**. Đây là lúc tài khoản GitHub của bài 4 phát huy: khởi tạo thêm tài khoản mới, và về sau ba dịch vụ dùng chung một lỗi vào.



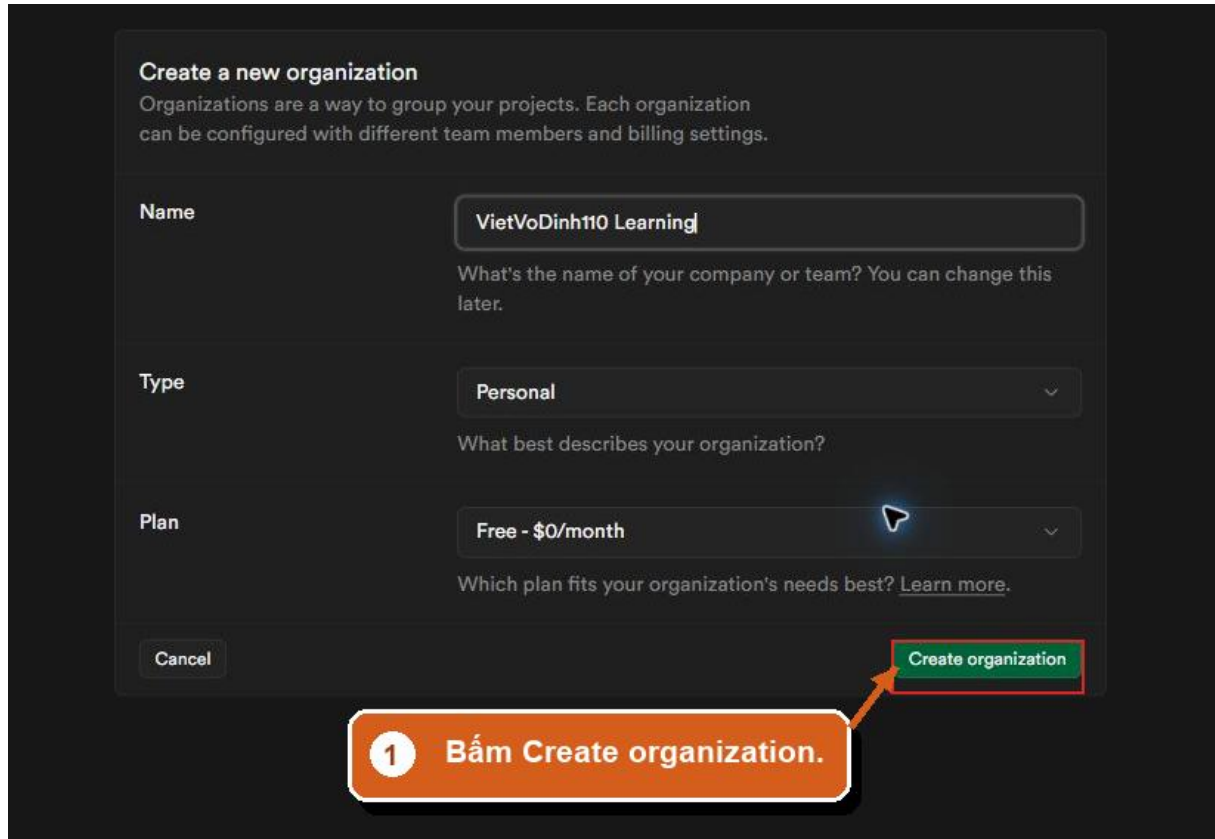
Chọn Continue with GitHub

GitHub hỏi có cho phép Supabase không. Kiểm tra đúng tên tài khoản của mình rồi bấm **Authorize supabase**.



Cho phép Supabase dùng tài khoản GitHub

Bước 3. Tạo tổ chức. Lần đầu vào, Supabase bảo tạo một "organization", hiểu là cái tủ đựng các dự án của bạn. Đặt tên tùy ý, để nguyên hai lựa chọn **Personal** và **Free**, bấm **Create organization**.



Create a new organization
Organizations are a way to group your projects. Each organization can be configured with different team members and billing settings.

Name
VietVoDinh110 Learning
What's the name of your company or team? You can change this later.

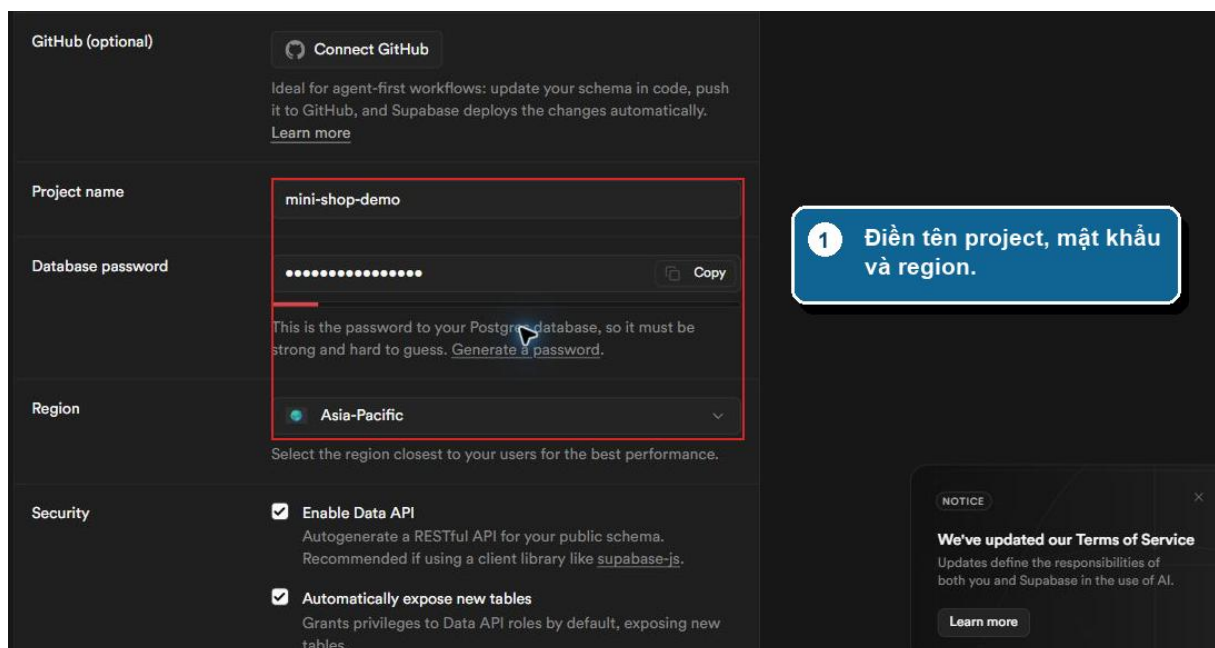
Type
Personal
What best describes your organization?

Plan
Free - \$0/month
Which plan fits your organization's needs best? [Learn more.](#)

1 Bấm Create organization.

Tạo organization cho tài khoản

Bước 4. Điền thông tin dự án. Màn tạo dự án hiện ra. Có ba ô phải điền và một ô phải kiểm.



GitHub (optional)
[Connect GitHub](#)
Ideal for agent-first workflows: update your schema in code, push it to GitHub, and Supabase deploys the changes automatically. [Learn more](#)

Project name
mini-shop-demo

Database password
..... [Copy](#)
This is the password to your PostgreSQL database, so it must be strong and hard to guess. [Generate a password.](#)

Region
Asia-Pacific
Select the region closest to your users for the best performance.

Security
☒ **Enable Data API**
Autogenerate a RESTful API for your public schema. Recommended if using a client library like `supabase-js`.
☒ **Automatically expose new tables**
Grants privileges to Data API roles by default, exposing new tables.

1 Điền tên project, mật khẩu và region.

NOTICE
We've updated our Terms of Service
Updates define the responsibilities of both you and Supabase in the use of AI.
[Learn more](#)



Điền tên dự án, mật khẩu và khu vực

Ô	Điền gì
Project name	mini-shop
Database password	Một mật khẩu mạnh, bấm Copy rồi ghi ngay vào chỗ cất mật khẩu của bạn
Region	Chọn khu vực gần Việt Nam, ví dụ Southeast Asia (Singapore)

Mật khẩu kho dữ liệu đáng nhắc riêng: đây là chìa khóa vạn năng của kho, và **Supabase chỉ cho bạn thấy đúng một lần**. Ghi lại ngay, lát nữa ở chạng nôi bạn sẽ cần tới nó.

Bước 5. Kiểm ô Data API. Kéo xuống phần **Security**, kiểm ô **Enable Data API** đang được bật.

Giữ ô Enable Data API được bật

Ô này cho phép trang web nói chuyện với kho. Tắt nó thì kho vẫn chạy nhưng web không đọc được gì, và bạn sẽ ngồi dò lỗi rất lâu mà không hiểu vì sao. Giữ bật.

Bước 6. Tạo. Bấm nút **Create new project** rồi chờ khoảng một hai phút để Supabase dựng kho.



Project name: mini-shop-demo

Database password: [masked] [Copy](#)

This password is strong. [Generate a password.](#)

Region: Asia-Pacific

Select the region closest to your users for the best performance.

Security:

- ☒ **Enable Data API**
Autogenerate a RESTful API for your public schema.
Recommended if using a client library like [supabase-js](#).
- ☒ **Automatically expose new tables**
Grants privileges to Data API roles by default, exposing new tables.
We recommend disabling this to control access manually.
- ☐ **Enable automatic RLS**
Create an event trigger that automatically enables Row Level Security on all new tables in the public schema.

ADVANCED

1 BẤM Create new project.

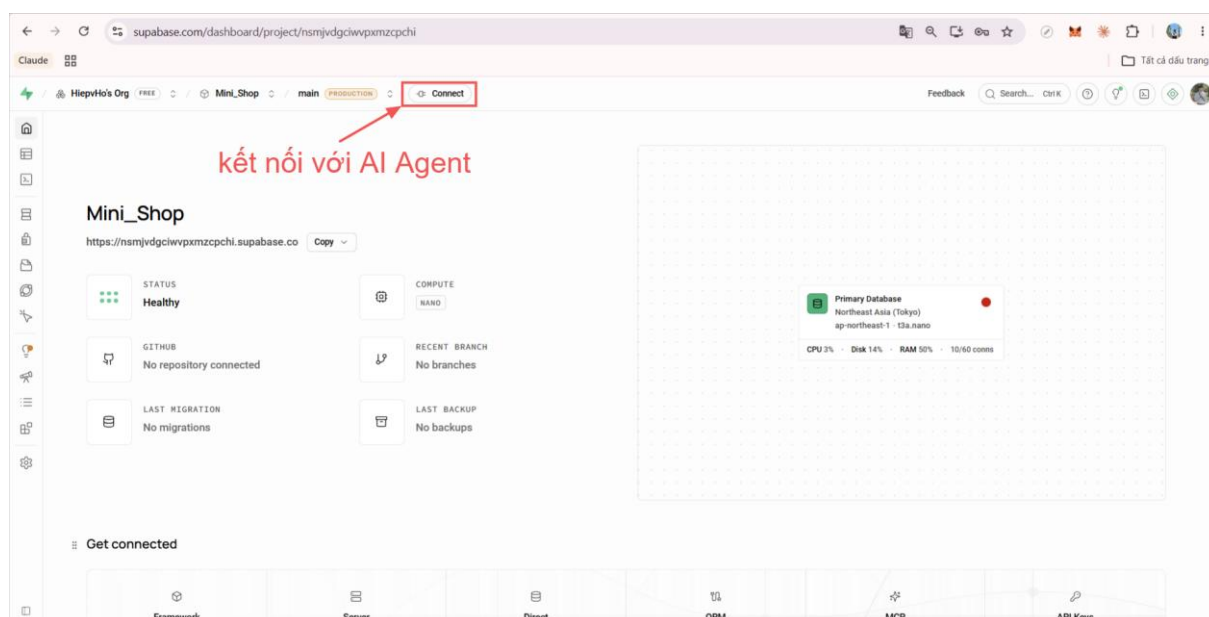
[Cancel](#) [Create new project](#)

Bấm Create new project

5.3 Chặng hai: nối ba đường

Kho đã có nhưng đang trống, và chưa ai nói chuyện được với nó. Giờ bắt các đường nối.

Ở trang dự án, trên thanh trên cùng có nút **Connect**. Bấm vào đó.



Bấm nút Connect ở thanh trên cùng

Một hộp hiện ra với năm thẻ: Framework, Server, Direct, ORM, MCP. Khóa học này dùng ba thẻ, và ta đi lần lượt từ trái sang phải cho khỏi lạc. Mỗi thẻ là một đường nối cho một mục đích khác nhau:

Thẻ	Nối ai với ai	Để làm gì
Framework	Trang web với kho	Cho web đọc và ghi dữ liệu
Direct	Máy bạn với kho, qua một chuỗi có mật khẩu	Đường dự phòng cho những việc nặng
MCP	AI Agent với kho	Để AI tự dựng bảng, đọc, sửa dữ liệu

Hai thẻ còn lại là Server và ORM, dành cho những cách làm khác, khóa này không dùng tới.

Đường thứ nhất: Framework, nối trang web

Bấm thẻ **Framework**. Ở **Framework** chọn **Next.js**, ô **Variant** chọn **App Router** — đây chính là loại dự án bạn dựng ở bài 3. Rồi bấm **Copy prompt** ở góc phải.



Connect to your project

Choose how you want to use Supabase

Framework

Use a client library

Server

Build APIs

Direct

Connection string

ORM

Third-party library

MCP

Connect your agent

Framework

Next.js

Variant

App Router

Shadcn

Install Supabase UI components with shadcn.

Follow these steps

1

Install packages

Run this command to install the required dependencies.

2

Add files

Add env variables, create Supabase client helpers, and set up middleware to keep sessions refreshed.

Copy prompt

.env.localpage.tsxutils/supabase/server.tsutils/supabase/client.ts

NEXT_PUBLIC_SUPABASE_URL=https://nsmjvdgciwvpxmzci

NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishab

Thẻ Framework: chọn Next.js và App Router

Chỗ này cần một lời giải thích, vì chữ "framework" xuất hiện hai lần với hai nghĩa. Ở bài 3, framework là bộ đồ nghề dựng web, và bạn chọn Next.js. Ở đây, thẻ Framework chỉ đơn giản hỏi "*web của bạn dựng bằng bộ đồ nghề nào*", để Supabase đưa đúng hướng dẫn cho loại đó. Chọn sai thì hướng dẫn không khớp với dự án.

Đoạn Supabase vừa chép cho bạn đã có sẵn địa chỉ kho và chìa khóa của dự án này, nên khỏi phải đi chép từng chuỗi. Về Antigravity, dán vào khung Agent kèm một câu dẫn quan trọng:

<dán đoạn Copy prompt của thẻ Framework vào đây>

Chỉ cài đặt và tạo file kết nối thôi, chưa sửa trang nào cả. Kho còn trống, lát nữa có dữ liệu tôi sẽ nhờ bạn sửa các trang sau.

Câu dẫn cuối là bắt buộc. Nếu không có nó, AI sẽ hăng hái sửa luôn trang danh sách sản phẩm để đọc từ kho, mà kho thì chưa có bảng nào, kết quả là trang trắng và một tràng báo lỗi.

Kiểm tra: Trong khung Explorer bên trái xuất hiện một file tên bắt đầu bằng `.env`. Đó là nơi cất chìa khóa, và cũng chính là file mà bài 4 đã dặn không bao giờ được đẩy lên GitHub.

Đường thứ hai: Direct, đường thẳng tới kho

Bấm thẻ **Direct**, rồi bấm **Copy prompt**.

Connect to your project
Choose how you want to use Supabase

Framework
Use a client library

Server
Build APIs

Direct
Connection string

ORM
Third-party library

MCP
Connect your agent

Connection Method

☒ **Direct connection**
Ideal for applications with persistent and long-lived connections such as those running on virtual machines or long-standing containers.

☐ **Transaction pooler**
Ideal for stateless applications like serverless functions where each interaction with Postgres is brief and isolated.

☐ **Session pooler**
Only recommended as an alternative to direct connection when connecting via an IPv4 network.

Type
URI

Follow these steps

Copy prompt

Direct connections use IPv6 by default
Enable the dedicated IPv4 address add-on to connect from IPv4-only networks. [Learn more](#)

Enable IPv4 add-on

Thẻ Direct: chuỗi kết nối trực tiếp tới kho

Đây là đường nói thẳng từ máy bạn tới kho, không đi qua trung gian nào. Nó mạnh nhất trong ba đường, nên cũng cần cẩn thận nhất: trong chuỗi kết nối có một chỗ để trống dành cho **mật khẩu kho dữ liệu**, chính là mật khẩu bạn ghi lại ở bước 4 mục trước.

Dán vào khung Agent, rồi thêm một dòng ghi rõ mật khẩu:

<dán đoạn Copy prompt của thẻ Direct vào đây>

Mật khẩu kho dữ liệu của tôi là: <gõ mật khẩu vào đây>

Hai điều phải nhớ về chuỗi này, vì nó là chìa khóa vạn năng của kho:



- **Không bao giờ để nó lọt vào code hay lên GitHub.** Nó cho toàn quyền với dữ liệu của bạn.
- Dán mật khẩu vào khung chat nghĩa là gửi nó cho dịch vụ AI. Với một dự án học tập thì chấp nhận được. Nếu sau này bạn làm sản phẩm thật có dữ liệu khách hàng, hãy đổi mật khẩu kho sau khi làm xong.

Đường thứ ba: MCP, nối AI vào kho

Đây là đường đáng giá nhất, và để cuối cùng vì nó đòi khởi động lại phần mềm.

Không có đường này, bạn phải làm người chạy vật: AI viết lệnh, bạn chép mang sang Supabase dán, chạy xong lại chép kết quả mang về. Có nó rồi, AI nói chuyện thẳng với kho, tự tạo bảng, tự đổ dữ liệu, tự đọc lại để kiểm. Giống như thay vì bạn cầm chìa khóa chạy ra chạy vào lấy đồ hộ, bạn đưa hản chìa khóa kho cho người thợ.

Bấm thẻ **MCP**. Ở ô **Client**, chọn **Antigravity**. Ô **Read-only** để tắt. Phần **Feature groups** giữ nguyên tất cả các nhóm đang có. Rồi bấm **Copy prompt**.

Connect to your project
Choose how you want to use Supabase

Framework
Use a client library

Server
Build APIs

Direct
Connection string

ORM
Third-party library

MCP
Connect your agent

chọn AI Agent phù hợp

Client
Antigravity
The MCP client you are using.

Read-only
☐
Only allow read operations on your database.

Feature groups
DOCS X ACCOUNT X DATABASE X DEBUGGING X DEVELOPMENT X
FUNCTIONS X BRANCHING X STORAGE X
Which MCP tools to include. Storage is off by default to keep tool counts manageable.

chọn toàn bộ các quyền

Follow these steps

1 Configure MCP
Set up your MCP client.

Add this configuration to ~/.gemini/antigravity/mcp_config.json :

```
1 {  
2   "mcpServers": {  
3     "supabase": {  
4       "serverUrl": "https://mcp.supabase.com/i
```

Copy prompt

Thẻ MCP: chọn Antigravity và giữ đủ các nhóm quyền



Ô **Read-only** đáng dừng một giây. Bật nó thì AI chỉ được đọc, tắt nó thì AI được cả **sửa và xóa** dữ liệu thật. Ta tắt vì lát nữa AI phải dựng bảng cho bạn. Đổi lại, giữ đúng nếp đã học ở bài 3: đọc kế hoạch AI đưa trước khi cho chạy, nhất là khi thấy chữ xóa.

Về Antigravity, dán đoạn vừa chép vào khung Agent, thêm một câu ở cuối rồi nhấn Enter:

<dán đoạn Copy prompt của thẻ MCP vào đây>

Làm giúp tôi phần cấu hình trên. Xong thì nói rõ tôi cần làm gì tiếp.

Khởi động lại Antigravity. Đây là bước hay bị quên nhất. Đường dây mới chỉ chạy sau khi phần mềm khởi động lại: đóng hẳn cửa sổ Antigravity rồi mở lại, mở lại thư mục làm việc như cũ.

Kiểm tra cả ba đường: Gõ vào khung Agent một câu:

Kể tên các bảng đang có trong kho Supabase của tôi.

Kho còn trống nên câu trả lời đúng là "chưa có bảng nào", và đó chính là dấu hiệu tốt: AI đã nhìn thấy kho của bạn. Nếu nó nói không kết nối được, kiểm lại xem đã khởi động lại Antigravity chưa.

Ba đường đã thông. Từ đây trở đi chỉ còn gõ yêu cầu.

5.4 Chặng ba: nhờ AI dựng bảng và đổ dữ liệu

Kho cần các **bảng** bên trong. Một bảng trông y như trang tính Excel: mỗi cột một loại thông tin, mỗi dòng một mục. Cửa hàng cần vài bảng: danh mục, sản phẩm, đơn hàng.

Trước khi gõ câu đúng, xem câu viết vội hổng thế nào.

Yêu cầu chưa tốt

Tạo database cho web của tôi đi.

Chỗ hỏng	AI sẽ làm gì
Không nói dựa trên cái gì	Bịa ra các bảng theo ý nó, không khớp của hàng bạn đang có
Không nói có đổ dữ liệu không	Tạo bảng rỗng, web nối vào chẳng có gì hiện ra
Không nói cho xem trước	Lao vào tạo luôn, sai thì đã nằm trong kho thật
Không có dấu hiệu nghiệm thu	Xong rồi bạn cũng chẳng biết lấy gì kiểm

Yêu cầu tốt

VIỆC: Dựng các bảng cho cửa hàng trong kho Supabase và đổ sẵn dữ liệu
NGUỒN: Danh mục và sản phẩm đang có trong code của dự án mini-shop-next
KẾT QUẢ: Kho có các bảng danh mục, sản phẩm, đơn hàng; bảng sản phẩm chứa đủ các món cửa hàng đang bán, tôi mở Table Editor trên Supabase là thấy



LƯU Ý: Cho tôi xem kế hoạch các bảng trước khi tạo; chưa cần bật khóa an toàn RLS
lúc này, tôi sẽ bật trước khi đưa web lên mạng

AI đưa ra danh sách bảng và cột dự định tạo. Đọc lướt xem có đủ những thứ cửa hàng cần không, thấy ổn thì bảo nó làm. Vài chục giây sau, kho của bạn có bảng và có dữ liệu, không một lần chép dán.

Câu cuối trong phần LƯU Ý là một **lời hẹn**. **RLS** là khóa an toàn gắn trên từng bảng, quy định ai được đọc, ai được ghi. Lúc này cửa hàng còn chạy trong máy bạn, chưa ai ngoài kia vào được, nên để bảng mở cho dễ làm. Nhưng **trước khi đưa web lên mạng ở bài 8, ta sẽ quay lại bật khóa này**. Sách sẽ nhắc đúng lúc.

Kiểm tra: Sang trang Supabase, bấm biểu tượng **Table Editor** ở thanh dọc bên trái, mở bảng sản phẩm. Đủ các món quen thuộc của cửa hàng nằm thành từng dòng:

id	int8	category_id	name	text	description	text	price	numeric
1		1		Ao thun basic	Ao thun cotton đơn giản		120000	
2		1		Ao thun oversize	Ao thun form rộng trẻ trung		150000	
3		2		Túi canvas	Túi canvas đi học đi làm		180000	
4		2		Túi đeo chéo	Túi đeo chéo nhỏ gọn		220000	
5		3		Mũ lưới trai	Mũ lưới trai basic		90000	
6		3		Vỉ da mini	Vỉ nhỏ gọn dễ dùng		160000	

1 Bảng products đã có dữ liệu.

Bảng sản phẩm đã đầy dữ liệu

Khoảnh khắc này đáng dừng lại một giây: dữ liệu cửa hàng của bạn lần đầu nằm trong một cuốn sổ cái thật, trên một máy chủ thật.

5.5 Cho trang web đọc dữ liệu thật

Kho đã đầy, và phần kết nối cho web đã cài sẵn từ mục 5.3. Giờ chỉ việc bảo AI dùng nó:

VIỆC: Sửa trang chủ và trang danh sách sản phẩm để đọc dữ liệu từ Supabase
NGUỒN: Các bảng vừa tạo trong kho; phần kết nối đã cài ở bước trước
KẾT QUẢ: Hai trang hiện sản phẩm lấy từ kho, không còn dùng dữ liệu ghi cứng trong code nữa
LƯU Ý: Giữ nguyên giao diện như hiện tại, chỉ đổi nguồn dữ liệu

Agent sửa code rồi chạy lại trang. Nhìn bề ngoài web không đổi gì, các sản phẩm vẫn hiện như cũ. Khác biệt nằm ở nguồn: giờ chúng chảy ra từ cuốn sổ cái.



5.6 Nghiệm thu: đổi số cái, cửa hàng đổi theo

Kiểm chứng bằng một phép thử sò được.

Sang Supabase, vào **Table Editor**, mở bảng sản phẩm. Bấm đúp vào ô giá của một món bất kỳ, sửa thành một con số khác hẳn, lưu lại. Quay về trang web, tải lại trang.

Giá trên web đổi theo. Bạn vừa chứng kiến đúng cái cảnh "sổ cái ở văn phòng, mọi quầy đọc chung": dữ liệu giờ chỉ có một nguồn sự thật, sửa một chỗ, mọi nơi thấy ngay. Thử xong, sửa giá về như cũ.

Còn một việc khép bài, kiểm tra cái khóa không bị lộ:

```
Kiểm tra giúp tôi: file chứa địa chỉ và khóa Supabase có đang bị đẩy lên GitHub không?  
Nếu có thì gỡ ra và chặn lại.
```

Agent xác nhận file nằm trong danh sách giữ lại ở nhà là ổn. Lưu mốc lên GitHub rồi nghỉ; lần lưu này chỉ chứa code, chìa khóa vẫn ở nhà.

Tóm tắt bài

- Dữ liệu ghi cứng trong code và nằm rải trong máy khách là tạm bợ; cửa hàng cần một sổ cái chung gọi là cơ sở dữ liệu
- Supabase cho thuê kho dữ liệu trên mạng, đăng nhập bằng tài khoản GitHub có sẵn, mức khởi đầu miễn phí
- Lúc tạo dự án: ghi lại mật khẩu kho ngay vì chỉ hiện một lần, và giữ ô Enable Data API được bật
- Ba đường nối, đi từ trái sang phải: Framework nối trang web, Direct nối máy bạn, MCP nối AI Agent
- Cài Framework khi kho còn trống thì phải dặn rõ "chưa sửa trang nào", kéo AI sửa trước lúc có dữ liệu
- Nối MCP xong phải khởi động lại Antigravity, rồi hỏi "kể tên các bảng" để nghiệm thu cả ba đường
- Nối xong hết mới bắt tay làm: AI tự dựng bảng và đổ dữ liệu, bạn chỉ đọc kế hoạch và kiểm bằng Table Editor
- RLS là khóa an toàn trên từng bảng; đang làm trong nhà thì tạm để mở, hẹn bật trước khi lên mạng ở bài 8



BÀI 6: CHO CỬA HÀNG VẬN HÀNH THẬT

Mục tiêu bài học:

- Biết bốn động tác với kho dữ liệu: thêm, đọc, sửa, xóa
- Cho khách đặt hàng và đơn được ghi thẳng vào kho, người bán nhìn thấy
- Thay đăng nhập giả bằng đăng nhập thật của Supabase
- Phân vai người mua và người bán, khu quản trị chỉ chủ cửa hàng vào được
- Cho khu quản trị thêm sửa xóa thật: admin đổi giá, khách thấy ngay

6.1 Từ trưng bày sang vận hành

Sau bài 5, cửa hàng của bạn đọc được sổ cái. Nhưng để ý kỹ, nó mới chỉ **đọc**. Khách đặt hàng, đơn vẫn nằm trong máy khách. Người bán "sửa" sản phẩm trong khu quản trị, thay đổi chỉ hiện trên màn hình rồi mất. Cửa hàng đang giống một gian trưng bày đẹp: xem được, nhưng chưa mua bán thật được.

Làm việc với một kho dữ liệu, xét cho cùng chỉ có bốn động tác: **thêm** một dòng mới, **đọc** các dòng ra, **sửa** một dòng, **xóa** một dòng. Dân trong nghề gộp bốn chữ tiếng Anh thành **CRUD**, bạn chỉ cần nhớ ý: thêm, đọc, sửa, xóa. Bài 5 làm xong động tác đọc. Bài này làm nốt ba động tác còn lại, mỗi động tác gắn vào một việc thật của cửa hàng.

6.2 Khách đặt hàng, đơn vào sổ cái

Bắt đầu từ động tác **thêm**, ở đúng chỗ đau nhất: đơn hàng.

VIỆC: Sửa phần thanh toán để đơn đặt hàng được ghi vào kho Supabase
NGUỒN: Trang thanh toán hiện có; bảng đơn hàng trong kho
KẾT QUẢ: Khách điền form và bấm Đặt hàng thì trong bảng đơn hàng trên Supabase xuất hiện một dòng mới, đủ tên, số điện thoại, địa chỉ, các món và tổng tiền
LƯU Ý: Kho còn thiếu bảng hay cột nào thì tự bổ sung qua đường dây MCP, đừng tự bịa chỗ lưu khác

Kiểm tra: Ra trang web, bỏ hai món vào giỏ, điền form đặt hàng với tên của chính bạn, bấm Đặt hàng. Sang Supabase mở bảng đơn hàng: một dòng mới mang tên bạn vừa nằm đó. Đóng hẳn trình duyệt, mở lại, dòng đó vẫn còn. Đơn hàng đã thoát kiếp tạm bợ.

6.3 Đăng nhập thật

Đăng nhập giả của bài 2 đã xong vai. Giờ thay bằng đăng nhập thật, và đây là chỗ phải cẩn thận nhất cuốn sách, vì nó dính tới mật khẩu của người khác.

Trước khi gõ câu đúng, xem câu viết vội nguy hiểm thế nào.

Yêu cầu chưa tốt

Làm chức năng đăng nhập cho web.



Chỗ hỏng	AI sẽ làm gì
Không nói dùng hệ nào	Có thể tự chế, lưu mật khẩu người dùng vào một bảng thường, lộ là thảm họa
Không tả đủ luồng	Có đăng nhập mà thiếu đăng ký, thiếu đăng xuất, thiếu nhớ phiên
Không nói thay thế bản giả	Bản giả và bản thật chồng lên nhau, loạn
Không có dấu hiệu nghiệm thu	Không biết thế nào là xong

Nguyên tắc an toàn chỉ có một câu, và đáng thuộc lòng: **chuyện mật khẩu không bao giờ tự chế, luôn dùng hệ đăng nhập dựng sẵn của dịch vụ lớn**. Supabase có sẵn hệ đó, đã được người ta lo giùm phần khó nhất.

Yêu cầu tốt

VIỆC: Thay đăng nhập giả bằng đăng nhập thật dùng hệ Auth có sẵn của Supabase
 NGUỒN: Các trang đăng nhập, đăng ký hiện có của mini-shop-next
 KẾT QUẢ: Khách đăng ký được tài khoản mới bằng email và mật khẩu, đăng nhập rồi thấy tên mình trên thanh menu, đăng xuất được; đóng trình duyệt mở lại vẫn còn đăng nhập
 LƯU Ý: Tuyệt đối không tự lưu mật khẩu vào bảng thường, mọi thứ đi qua hệ Auth của Supabase; gỡ hẳn phần đăng nhập giả cũ

Kiểm tra: Đăng ký một tài khoản với email của bạn, đăng nhập, thấy tên trên menu. Đăng xuất, đăng nhập lại. Sang Supabase, vào mục **Authentication**, thấy tài khoản vừa tạo nằm trong danh sách người dùng. Mật khẩu thì không ai xem được, kể cả bạn, và như vậy là đúng.

6.4 Ai là chủ cửa hàng

Đăng nhập thật rồi, nhưng mọi tài khoản đang bình đẳng như nhau. Cửa hàng cần phân vai: **khách** mua hàng, **chủ cửa hàng** (admin) vào khu quản trị. Vai được ghi ngay trong kho, như cột chức danh trong sổ nhân sự.

VIỆC: Thêm phân vai người dùng, mặc định là khách, và chặn khu quản trị
 NGUỒN: Hệ Auth và kho Supabase đang dùng
 KẾT QUẢ: Tài khoản thường vào địa chỉ khu quản trị thì bị đưa về trang đăng nhập; tài khoản có vai admin thì vào được
 LƯU Ý: Cần sửa gì trong kho thì làm qua đường dây MCP; xong thì nâng tài khoản email <ghi email bạn vừa đăng ký> lên vai admin cho tôi

Agent sửa kho và nâng quyền cho tài khoản của bạn. Bạn vừa tự bổ nhiệm mình làm chủ cửa hàng.

Kiểm tra: Đăng nhập bằng tài khoản admin, vào khu quản trị bình thường. Đăng xuất, đăng ký thêm một tài khoản thường bằng email khác, thử vào đúng địa chỉ khu quản trị: bị chặn. Hàng rào đã dựng đúng chỗ.

6.5 Khu quản trị ghi thật

Còn hai động tác **sửa** và **xóa**, giao nốt cho khu quản trị:



VIỆC: Cho khu quản trị làm việc thật với kho Supabase
NGUỒN: Các màn quản trị hiện có: sản phẩm, đơn hàng
KẾT QUẢ: Admin thêm, sửa, xóa sản phẩm thì bảng trên Supabase đổi theo và trang khách hiện đúng như vậy; màn đơn hàng liệt kê đơn thật từ kho, đổi được trạng thái
đơn từ mới sang đang giao, đã giao
LƯU Ý: Sản phẩm mới dùng ảnh có sẵn trong thư mục assets; xóa sản phẩm thì hỏi xác nhận trước khi xóa

Kiểm tra: Đây là màn nghiệm thu vui nhất từ đầu sách. Mở hai cửa sổ trình duyệt cạnh nhau: một bên là khu quản trị đăng nhập admin, một bên là trang khách. Bên quản trị sửa giá Đền tre thủ công, bên khách tải lại trang: giá mới hiện ra. Thêm một sản phẩm mới bên quản trị, bên khách thấy món mới lên kệ. Mở màn đơn hàng, thấy đơn bạn đặt ở mục 6.2, đổi trạng thái nó sang đang giao.

Xong, đừng quên câu quen thuộc:

Lưu mốc lên GitHub, ghi chú: cửa hàng vận hành thật với kho Supabase.

6.6 Nhìn lại

Điểm lại ba chỗ đuôi treo từ bài 2, giờ đã gỡ được mấy?

Chỗ lập menu, bài 3 gỡ bằng component. Chỗ dữ liệu tạm bợ, bài 5 và bài này gỡ nốt: sản phẩm nằm trong sổ cái, đơn hàng ghi về một chỗ, người bán nhìn thấy và xử lý được, người mua có tài khoản thật. Cửa hàng của bạn, chạy trên máy bạn, đã là một phần mềm vận hành đúng nghĩa.

Chỉ còn đúng một chuyện: nó vẫn đang ở localhost, sau cánh cửa nhà bạn. Trước khi mở cửa cho cả thế giới, người cẩn thận sẽ đi khám tổng quát một lượt. Đó là bài 7.

Tóm tắt bài

- Làm việc với kho dữ liệu gói trong bốn động tác thêm, đọc, sửa, xóa; bài 5 lo đọc, bài này lo ba động tác còn lại
- Đơn đặt hàng giờ ghi thẳng vào kho, đóng máy mở lại vẫn còn, người bán nhìn thấy
- Chuyện mật khẩu không bao giờ tự chế; dùng hệ Auth có sẵn của Supabase
- Phân vai bằng một cột trong kho: khách mua hàng, admin vào khu quản trị, rào chặn đặt ở địa chỉ quản trị
- Khu quản trị thêm sửa xóa thật: đổi bên quản trị, trang khách đổi theo
- Cửa hàng đã vận hành thật trên máy bạn; trước khi lên mạng còn một lượt khám tổng quát



BÀI 7: KHÁM TỔNG QUÁT TRƯỚC KHI RA MẮT

Mục tiêu bài học:

- Hiểu vì sao phải tự kiểm thử trước khi cho người khác vào
- Đi hết kịch bản người mua và kịch bản người bán, ghi lỗi cho đúng cách
- Thử web như một người lạ: trên điện thoại, chưa đăng nhập, nhập liệu ẩu
- Đưa danh sách lỗi cho AI sửa an toàn, có mốc để quay lại
- Chốt danh sách sẵn sàng trước khi lên mạng ở bài 8

7.1 Khách không báo lỗi, khách bỏ đi

Cửa hàng chạy tốt dưới tay bạn. Nhưng bạn là người dựng ra nó, bạn bấm đúng chỗ theo thói quen, nhập đúng kiểu dữ liệu nó chờ. Khách thì khác: họ bấm lung tung, nhập thiếu, vào bằng điện thoại, và khi gặp một trang lỗi họ lặng lẽ đóng tab. Rất hiếm ai nhắn cho bạn "web bị hỏng chỗ này". Lỗi trên web bán hàng trả giá bằng đơn hàng mất đi trong im lặng.

Vì vậy trước khi mở cửa, chủ cửa hàng tự đóng vai khách đi mua một vòng từ đầu tới cuối. Bài này làm đúng việc đó, theo ba lượt: đi vai người mua, đi vai người bán, rồi thử như một người lạ. Gặp lỗi thì ghi lại, cuối bài đưa cả danh sách cho AI sửa một thể.

Cách ghi một lỗi cho ra hồn

Ghi "trang giỏ hàng bị lỗi" thì AI cũng bó tay như một người thợ nghe chủ nhà nói "nhà có chỗ hỏng". Một dòng lỗi tốt gồm bốn ý: **ở màn nào, bạn làm gì, thấy gì, đáng lẽ phải thế nào**. Ví dụ: "Ở trang giỏ hàng, tôi bấm nút trừ khi số lượng đang là 1, món hàng biến mất luôn, đáng lẽ phải hỏi tôi có muốn xóa không."

Mở một file ghi chú, mỗi lỗi một dòng theo bốn ý đó. Danh sách này chính là phần NGUỒN cho prompt sửa lỗi ở cuối bài.

7.2 Kịch bản người mua

Đi chậm từng bước như một khách thật, đối chiếu từng dấu hiệu:

Bước	Làm gì	Dấu hiệu đạt
1	Mở trang chủ	Banner, sản phẩm, giá hiện đủ, không ô ảnh vỡ
2	Gõ "đèn" vào ô tìm kiếm	Chỉ còn các sản phẩm đèn
3	Lọc danh mục Đồ thủ công	Chỉ còn nhóm đồ thủ công
4	Mở một sản phẩm	Đúng ảnh, đúng giá, đúng mô tả món đó
5	Thêm vào giỏ hai lần	Số trên biểu tượng giỏ tăng đúng



Bước	Làm gì	Dấu hiệu đạt
6	Vào giỏ, tăng giảm số lượng	Tổng tiền tính lại đúng
7	Thả tim hai món	Trang yêu thích hiện đúng hai món
8	Đăng ký một tài khoản mới	Đăng ký xong, tên hiện trên menu
9	Đặt hàng với giỏ đang có	Báo thành công, giỏ về không
10	Sang Supabase mở bảng đơn hàng	Đơn vừa đặt nằm đó, đủ thông tin

Bước nào lệch, ghi vào danh sách lỗi theo bốn ý, rồi cứ đi tiếp cho hết kịch bản. Đừng dừng lại sửa vặt giữa chừng, khám xong toàn thân rồi chữa một thể.

7.3 Kịch bản người bán

Đăng nhập bằng tài khoản admin, đi tiếp lượt của chủ cửa hàng:

Bước	Làm gì	Dấu hiệu đạt
1	Vào khu quản trị	Vào được, số liệu tổng quan hiện ra
2	Mở màn đơn hàng	Thấy đơn khách vừa đặt ở kịch bản trước
3	Đổi trạng thái đơn sang đang giao	Trạng thái đổi và giữ nguyên sau khi tải lại
4	Thêm một sản phẩm thử	Trang khách thấy món mới
5	Sửa giá món vừa thêm	Trang khách thấy giá mới
6	Xóa món thử đó	Có hỏi xác nhận, xóa xong trang khách hết món đó

7.4 Thử như một người lạ

Hai kịch bản trên là đường đi của người ngoan. Giờ thử kiểu người lạ, ba phép thử:

Thử trên điện thoại. Mở trang web trên điện thoại của bạn (cùng mạng Wi-Fi, Agent sẽ chỉ địa chỉ nếu bạn hỏi), hoặc thu nhỏ cửa sổ trình duyệt về cỡ màn hình dọc. Menu, lưới sản phẩm, giỏ hàng phải xếp lại gọn gàng, chữ không tràn, nút không chồng lên nhau.

Thử khi chưa đăng nhập. Mở một cửa sổ trình duyệt ở chế độ ẩn danh, gõ thẳng địa chỉ khu quản trị. Phải bị chặn và đưa về trang đăng nhập. Nếu vào được, đây là lỗi nặng nhất buổi khám, ghi ngay đầu danh sách.



Thử nhập ầu. Ở form đặt hàng, bỏ trống hết rồi bấm Đặt hàng: web phải nhắc điện, không được tạo đơn rỗng. Nhập số lượng âm hoặc chữ vào ô số: web phải từ chối lịch sự thay vì hiện màn lỗi trắng.

7.5 Đưa danh sách lỗi cho AI sửa

Danh sách lỗi trong tay, giờ chữa. Trước khi gõ câu đúng, xem câu viết vội hổng ra sao.

Yêu cầu chưa tốt

Kiểm tra web giúp tôi xem có lỗi gì không rồi sửa hết đi.

Chỗ hổng	AI sẽ làm gì
Không đưa danh sách lỗi của bạn	Đọc lướt code, khen "nhìn chung ổn", bỏ sót lỗi thật bạn đã thấy
Không giới hạn phạm vi	Nhân tiện "cải thiện" lung tung, hổng cả chỗ đang tốt
Không có mốc quay lại	Sửa hổng nặng hơn thì hết đường lùi
Không nói cách báo kết quả	Sửa xong bạn chẳng biết thử lại chỗ nào

Yêu cầu tốt

Lưu mốc trước, rồi giao đúng danh sách:

VIỆC: Sửa các lỗi theo danh sách tôi liệt kê dưới đây
 NGUỒN: Danh sách lỗi, mỗi dòng gồm: ở màn nào, tôi làm gì, thấy gì, đáng lẽ thế nào
 KẾT QUẢ: Từng lỗi được sửa, báo lại đã sửa ra sao để tôi thử lại đúng chỗ đó
 LƯU Ý: Trước khi sửa hãy lưu mốc lên GitHub; chỉ sửa các lỗi trong danh sách, không tự ý đổi giao diện hay thêm tính năng khác
 <dán danh sách lỗi vào đây>

Agent sửa xong lỗi nào, bạn mở đúng chỗ đó thử lại theo kịch bản. Hết danh sách thì chạy lại trọn hai kịch bản một lần chót. Vòng lặp khám rồi chữa này lặp tới khi đi hết kịch bản mà danh sách lỗi trống trơn.

Muốn kỹ thêm một nấc, nhờ AI tự khám phần của nó:

Tự rà toàn bộ dự án, liệt kê những chỗ dễ gây lỗi hoặc còn thiếu sót, chưa cần sửa.
 Tôi sẽ chọn chỗ nào đáng sửa.

Nó sẽ liệt ra một danh sách, thường lẫn cả những góp ý kiểu làm màu. Bạn đọc và chỉ chọn những mục thấy đúng là vấn đề, quyền quyết định luôn ở người chủ cửa hàng.

7.6 Danh sách sẵn sàng ra mắt

Khép bài bằng bốn câu hỏi, trả lời đủ "rồi" thì cửa hàng được phép lên mạng:

- Hai kịch bản người mua, người bán đi trọn không còn lỗi?



- Cửa sổ ẩn danh vào khu quản trị đã bị chặn?
- Khóa Supabase vẫn nằm ngoài GitHub (đã kiểm ở bài 5)?
- Bản mới nhất đã lưu mốc lên GitHub?

Bốn chữ "rời" trong tay, hẹn bạn ở bài cuối: mở cửa cho cả thế giới.

Tóm tắt bài

- Khách gặp lỗi thì bỏ đi trong im lặng, nên chủ cửa hàng phải tự khám trước khi mở cửa
- Một dòng lỗi tốt gồm bốn ý: màn nào, làm gì, thấy gì, đáng lẽ thế nào
- Khám theo kịch bản tròn vòng: người mua mười bước, người bán sáu bước, lỗi ghi lại chữa một thể
- Thử như người lạ: trên điện thoại, ẩn danh vào khu quản trị phải bị chặn, nhập ầu phải bị từ chối lịch sự
- Giao AI sửa theo đúng danh sách, lưu mốc trước khi sửa, sửa xong thử lại từng chỗ
- Đủ bốn chữ "rời" trong danh sách sẵn sàng thì mới lên mạng



BÀI 8: ĐƯA CỬA HÀNG LÊN MẠNG

Mục tiêu bài học:

- Bật khóa an toàn RLS cho kho dữ liệu trước khi mở cửa, trả lời hện của bài 5
- Đưa cửa hàng lên mạng bằng Vercel, ai có link cũng vào được
- Nghiệm thu trên web thật: tự đặt hàng bằng điện thoại, nhờ người thân mua thử
- Nắm nhịp làm việc về sau: sửa, lưu mốc, web tự cập nhật
- Biết cần cất giữ những gì để cửa hàng sống lâu dài

8.1 Giờ mở cửa đã tới

Từ bài 1, bạn đã biết ranh giới giữa hai thế giới: web chạy localhost trong máy mình, và web trên mạng ai cũng vào được. Suốt bảy bài, cửa hàng của bạn sống ở thế giới thứ nhất. Hôm nay nó bước sang thế giới thứ hai.

Dịch vụ đưa nó sang là **Vercel**, hiểu đơn giản là một bãi đỗ trên mạng cho các web dựng bằng Next.js, mức khởi đầu miễn phí. Vercel có một điểm ăn khớp tuyệt vời với thứ bạn đã có: nó **nối thẳng vào GitHub**. Bạn chỉ nó tới kho mini-shop, nó tự lấy code về, tự dựng, tự phát hành. Và từ đó về sau, mỗi lần bạn lưu mốc phiên bản mới lên GitHub, Vercel tự động cập nhật web trên mạng. Thói quen lưu mốc của bài 4 giờ trả lại.

Nhưng khoan bấm nút. Còn một lời hện phải trả trước đã.

8.2 Khóa kho trước, mở cửa sau

Bài 5 có một lời hện: các bảng trong kho đang **mở**, vì lúc đó cửa hàng còn chạy trong nhà. Đưa web lên mạng khi kho còn mở giống như khai trương cửa hàng mà cửa kho không khóa: ai tình ý đều tự vào đọc, thậm chí sửa sổ sách của bạn. Vậy nên thứ tự bắt buộc là **khóa kho trước, mở cửa sau**.

Khóa ở đây là **RLS**, khóa an toàn trên từng bảng mà bạn đã gặp. Bật khóa nghĩa là đặt luật cho từng bảng: ai được đọc, ai được ghi. Trước khi gõ câu đúng, xem câu viết vội hồng ra sao.

Yêu cầu chưa tốt

Bật bảo mật cho database đi.

Chỗ hỏng	AI sẽ làm gì
Không nói luật cho từng bảng	Khóa tuốt mọi thứ, web trắng trơn vì chính nó cũng hết đọc được
Không cho xem luật trước	Khóa xong mới biết sai, mà lúc đó kho đã khóa rồi
Không nghiệm thu sau khóa	Khóa xong hồng chỗ nào không ai biết



Chỗ hỏng	AI sẽ làm gì
Không có móc quay lại	Lỡ khóa sai không biết đường lùi

Yêu cầu tốt

Luật của cửa hàng nói bằng tiếng người như sau: ai cũng được **xem** sản phẩm; người mua **tạo được đơn của mình**; chỉ **admin** được thêm sửa xóa sản phẩm và xử lý mọi đơn. Đưa nguyên văn cho AI, và nhớ đường dây MCP từ bài 5 vẫn còn đó nên nó tự làm được:

VIỆC: Bật khóa an toàn RLS cho toàn bộ các bảng trong kho, trả lời hện ở bài 5
NGUỒN: Các bảng danh mục, sản phẩm, đơn hàng, người dùng trên Supabase, làm qua MCP

KẾT QUẢ: Các bảng đều có khóa theo luật: ai cũng xem được sản phẩm và danh mục; người đăng nhập tạo được đơn của mình; chỉ admin được thêm sửa xóa sản phẩm và xem, xử lý mọi đơn

LƯU Ý: Lưu mốc lên GitHub trước; cho tôi xem luật định đặt trước khi bật; bật xong nhắc tôi đi lại kịch bản bài 7 để chắc không chỗ nào bị khóa nhầm

AI trình bày luật định đặt cho từng bảng. Đọc xem có đúng bốn ý bạn vừa nói không, thấy ổn thì cho chạy. Kho của bạn giờ đã khóa đàng hoàng, đúng nghĩa một cái kho trước giờ khai trương.

Muốn tận mắt nhìn các khóa vừa đặt, sang Supabase, ở thanh dọc bên trái tìm mục **Authentication** rồi vào **Policies**. Mỗi bảng hiện ra kèm danh sách luật đang áp cho nó:



The screenshot shows the Supabase Studio interface for managing Auth Policies. The left sidebar contains navigation links for Database, Authentication, Policies, Storage, API, Edge Functions, and Settings. The main content area is titled 'Auth Policies' and includes a search bar and an 'Add new policy' button. Below this, there are two sections: 'categories' and 'products'. Each section contains a table of policies.

Policy Name	Operation	Roles	Definition	Actions
Public read active categories	SELECT	anon , authenticated	Allows public read access to categories that are marked as active.	Edit Delete
Admins can read/insert/update/delete categories	ALL (or SELECT, INSERT, UPDATE, DELETE)	authenticated	Allows authenticated administrators full access to categories.	Edit Delete

Policy Name	Operation	Roles	Definition	Actions
Public read active products	SELECT	anon , authenticated	Allows public read access to products that are marked as active.	Edit Delete
Admins can read/insert/update/delete products	ALL (or SELECT, INSERT, UPDATE, DELETE)	authenticated	Allows authenticated administrators full access to products.	Edit Delete

Danh sách luật đang áp cho từng bảng

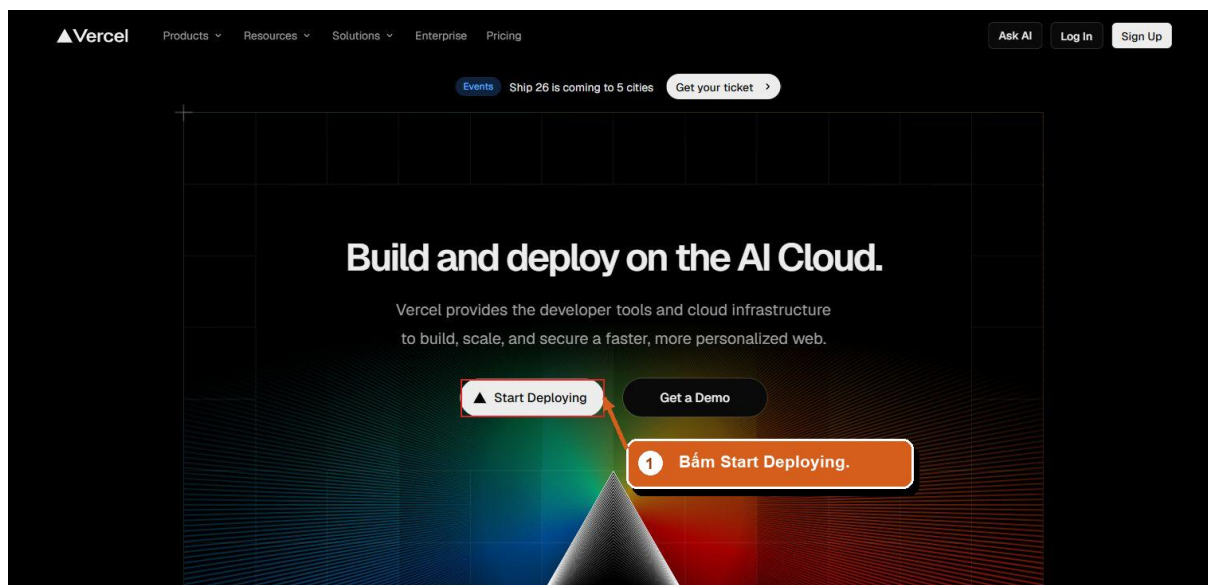
Kiểm tra: Đi nhanh lại kịch bản bài 7: khách xem hàng, đặt một đơn; admin vào quản trị sửa giá. Tất cả phải chạy như trước. Nếu chỗ nào bỗng trắng trơn hay báo không có quyền, tả đúng hiện tượng cho Agent, nó chỉnh lại luật là xong.

8.3 Đưa lên Vercel

Tới nghi thức chính. Phần này là việc tay trên trang Vercel, có ảnh dẫn từng chặng.

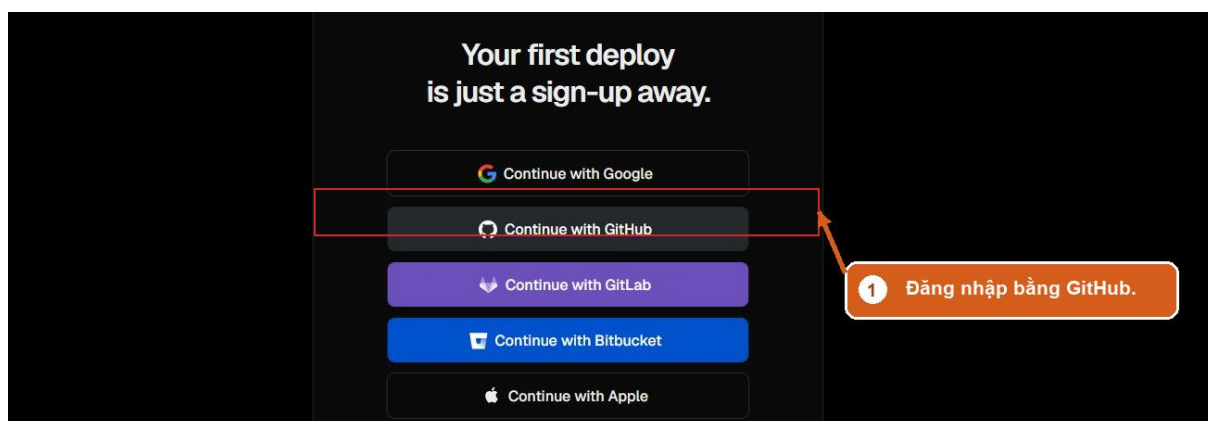
Bước 1. Tạo tài khoản Vercel. Vào vercel.com, bấm nút bắt đầu ở trang chủ.





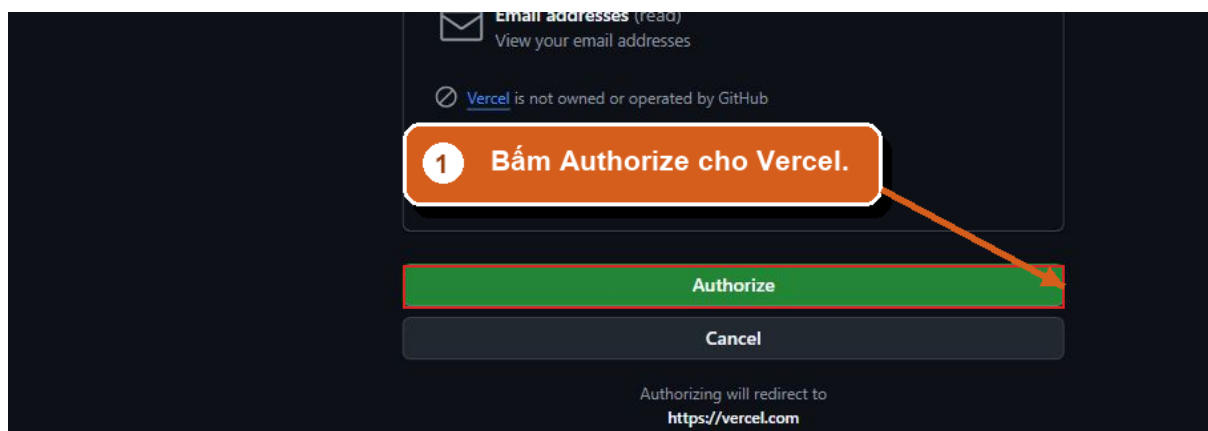
Trang chủ Vercel

Ở màn chọn cách đăng nhập, chọn **Continue with GitHub**. Vẫn là tài khoản GitHub quen thuộc, lần thứ ba nó mở cửa cho bạn.



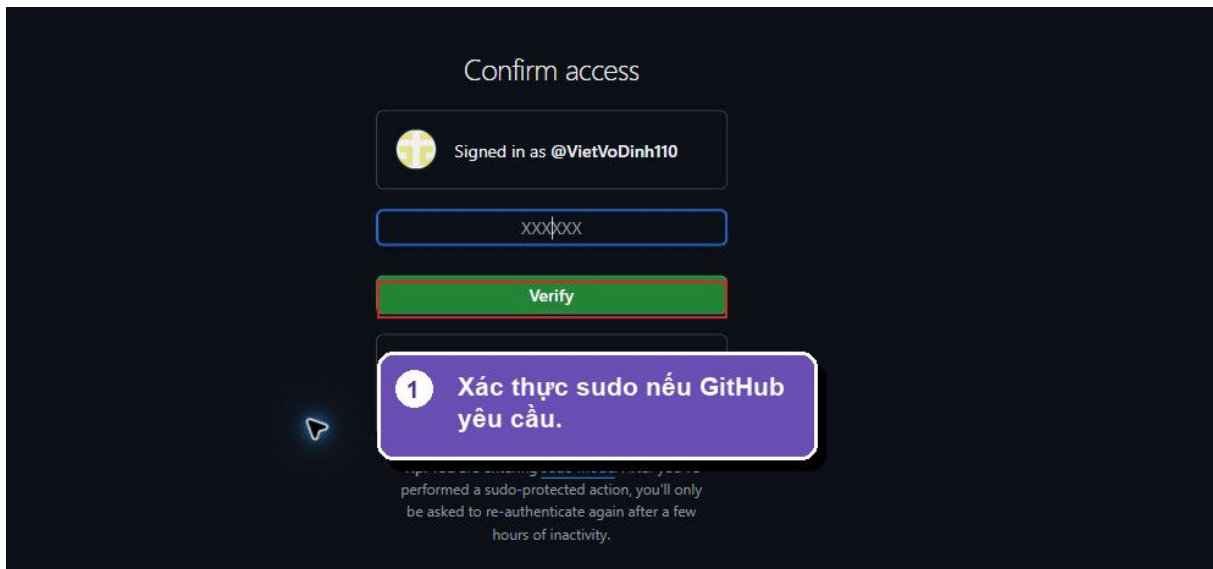
Đăng ký Vercel bằng tài khoản GitHub

GitHub hỏi có cho phép Vercel không, bấm **Authorize Vercel**:



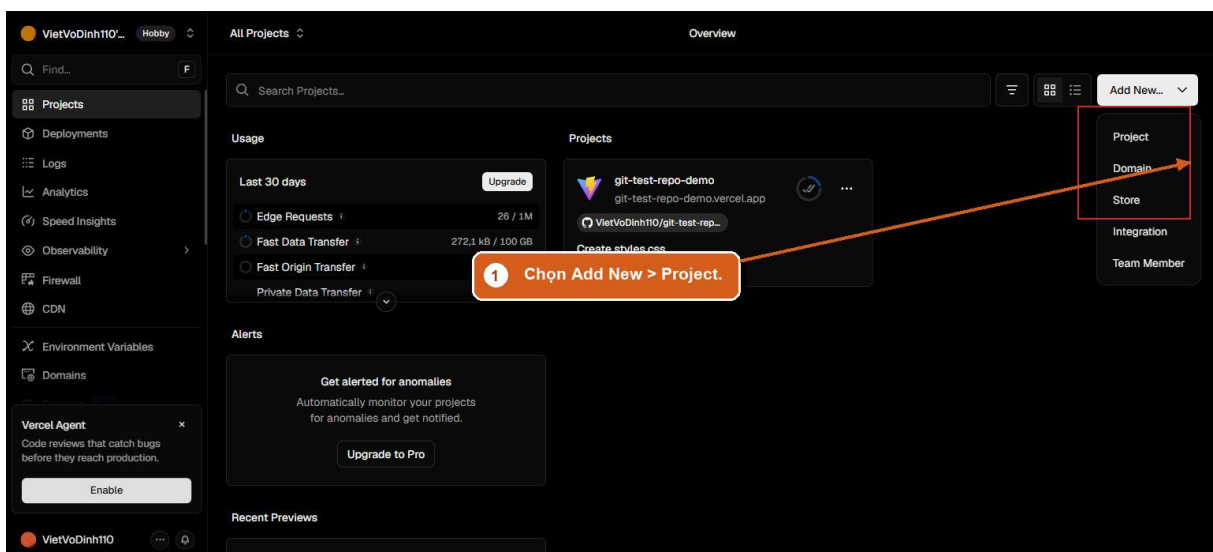
Cho phép Vercel dùng tài khoản GitHub

Có thể có thêm một màn hỏi Vercel được xem những kho nào. Chọn cho phép tất cả các kho, hoặc chọn riêng kho **mini-shop** cũng được, rồi bấm nút xác nhận.



Xác nhận Vercel được xem kho nào

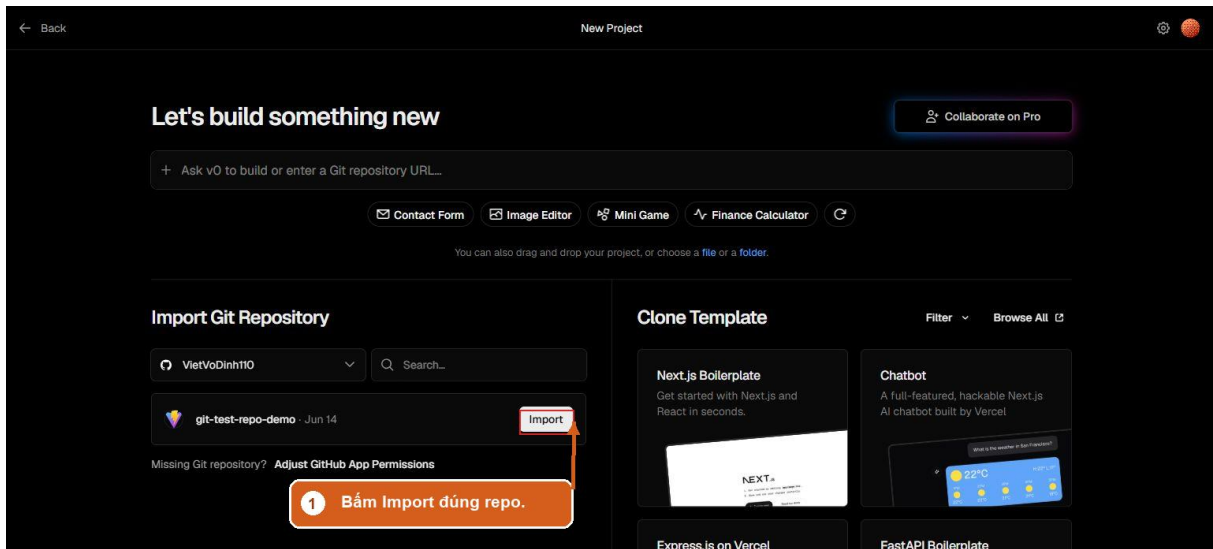
Bước 2. Tạo dự án từ kho GitHub. Vào trang quản lý của Vercel, bấm **Add New** rồi chọn **Project**:



Bấm Add New rồi chọn Project

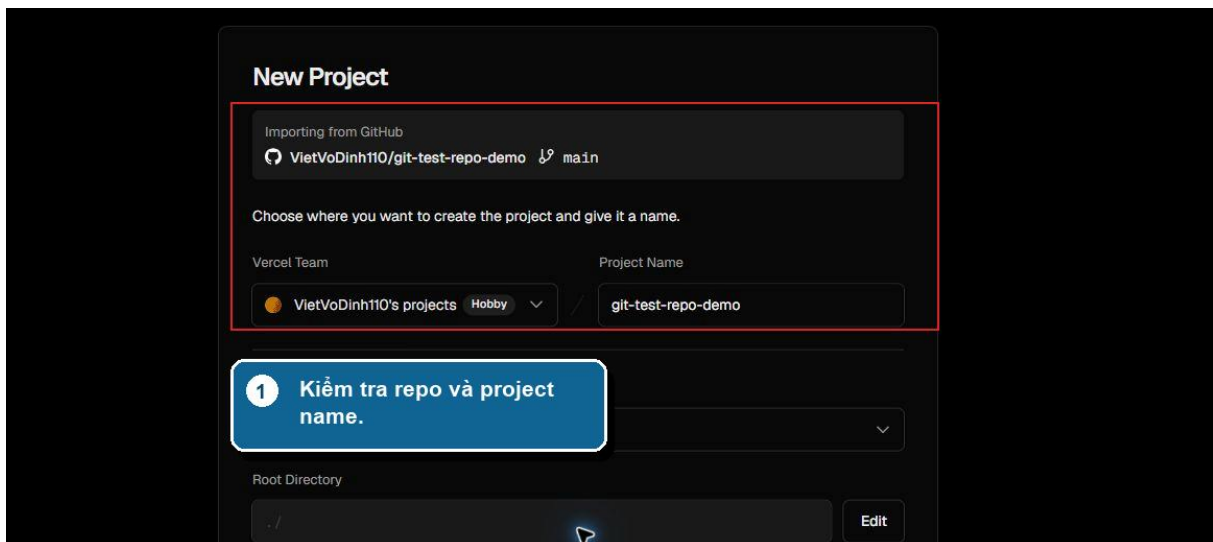
Danh sách kho GitHub của bạn hiện ra. Tìm dòng **mini-shop**, bấm **Import**:





Import đúng kho mini-shop

Bước 3. Kiểm màn cấu hình. Màn **New Project** hiện ra. Nhìn ba chỗ:



Màn cấu hình dự án

- **Importing from GitHub:** kiểm đúng kho **mini-shop** của bạn.
- **Framework Preset:** Vercel tự nhận ra đây là dự án Next.js, để nguyên như nó chọn.
- **Root Directory:** chỗ này để dấu chấm gạch chéo, nghĩa là "lấy từ gốc kho". Ở bài 4 bạn đã đẩy đúng thư mục mini-shop-next lên nên cứ để nguyên. Nếu chẳng may lúc đó đẩy nhầm cả thư mục cha, bấm **Edit** rồi chọn thư mục mini-shop-next là chữa được.

Bước 4. Khai hai chuỗi bí mật. Kéo xuống mục **Environment Variables**. Đây là chỗ duy nhất cần tay bạn gõ.

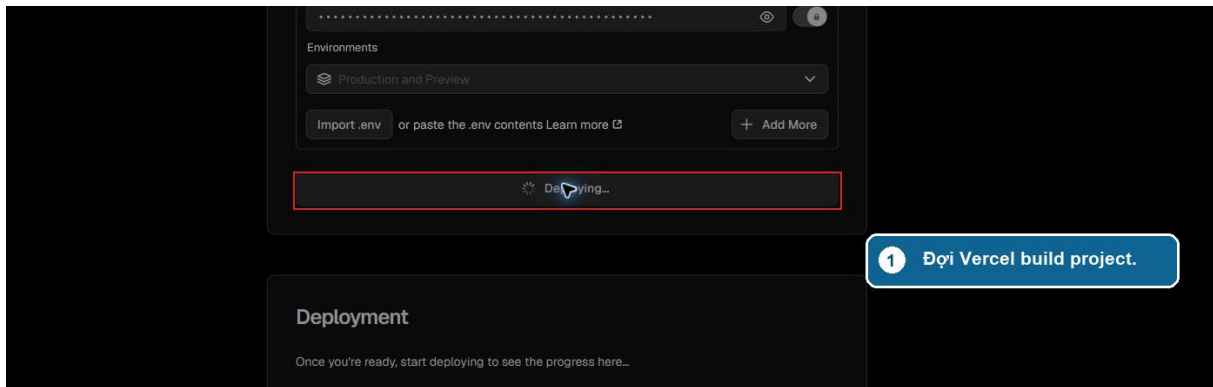
Nhớ lại bài 5: địa chỉ kho và chìa khóa nằm trong file môi trường, và file đó cố tình **không** được đẩy lên GitHub. Vercel vì vậy chưa biết hai chuỗi này, bạn phải trao tận tay. Mở file môi trường trong dự án (khung Explorer của Antigravity, file tên bắt đầu



bằng .env), với **từng dòng** trong đó: chép phần tên biến bên trái dấu bằng dán vào ô Name, chép phần giá trị bên phải dấu bằng dán vào ô Value, bấm **Add**. Làm đủ các dòng.

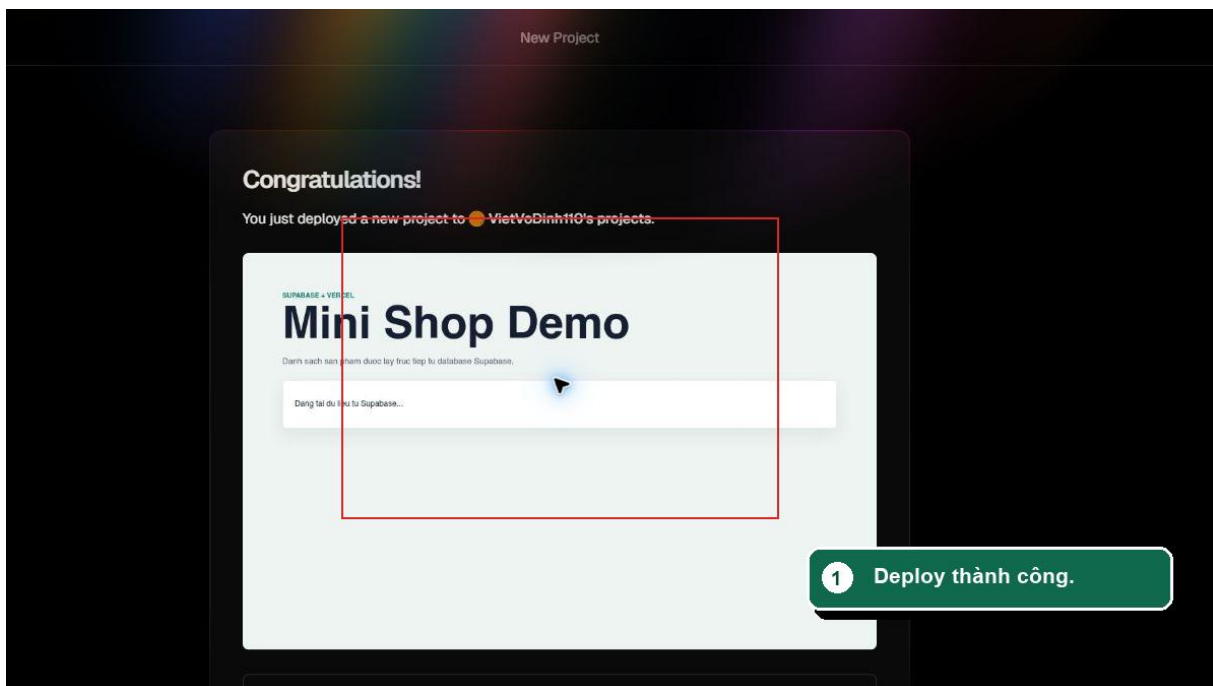
Đây chính là cách chuyên nghiệp để đem bí mật lên dịch vụ: đi thẳng từ tay bạn vào kết của Vercel, không đi qua chỗ công khai nào.

Bước 5. Deploy. Bấm nút **Deploy**. Vercel lấy code từ GitHub về và dựng, mất chừng một hai phút:



Vercel đang dựng web

Dựng xong, màn hình chúc mừng hiện ra kèm ảnh chụp trang web của bạn:



Deploy thành công

Bấm vào ảnh chụp đó, trình duyệt mở cửa hàng của bạn ở một địa chỉ dạng mini-shop-xxxx.vercel.app. Nhìn kỹ thanh địa chỉ: không còn localhost. Đây là web thật, trên mạng thật.

8.4 Đi một vòng trên web thật

Nghiệm thu lần này có nghi thức riêng, vì nó xứng đáng.

Rút điện thoại, **tắt Wi-Fi đi cho dùng mạng di động**, gõ địa chỉ cửa hàng. Trang chủ hiện ra trên một thiết bị chẳng dây dợ gì với máy tính của bạn. Đặt thử một đơn ngay trên điện thoại.

Rồi làm điều mà cả tám bài hướng tới: **gửi đường link cho một người thân**, nhờ họ chọn một món và đặt thử. Vài phút sau, mở khu quản trị: đơn của họ nằm đó, tên thật, món thật. Người cách bạn cả thành phố vừa dùng phần mềm bạn dựng. Cảm giác này, người làm nghề lâu năm vẫn nhớ lần đầu của họ.

Điện thoại có chỗ hiển thị lệch, form có chỗ khó bấm? Ghi lại theo đúng kiểu bài 7, về Antigravity sửa, rồi xem mục tiếp theo để hiểu vì sao bản sửa tự lên mạng.

8.5 Nhịp làm việc từ nay về sau

Cửa hàng đã sống trên mạng, và nhịp chăm nó gói trong một vòng bốn nhịp:

1. **Sửa** trên máy bằng prompt, như vẫn làm suốt tám bài
2. **Nghiệm thu** bản local, tự tay bấm thử
3. **Lưu mốc** lên GitHub bằng một câu
4. **Chờ vài phút**, Vercel tự dựng và web trên mạng thành bản mới

Không phải làm lại thao tác deploy nào nữa. Đường ống từ máy bạn ra thế giới đã nối xong, van tự động. Mọi tính năng bạn thêm về sau, từ đổi banner tới làm chương trình giảm giá, đều chảy qua đúng vòng bốn nhịp này.

8.6 Giữ gìn và đi tiếp

Cửa hàng sống lâu dài dựa trên một ít của cải cần cất kỹ. Ghi vào chỗ an toàn của bạn:

Thứ cần giữ	Là gì
Ba tài khoản	GitHub, Supabase, Vercel, và mật khẩu của chúng
Mật khẩu kho dữ liệu	Đặt ra ở bài 5 khi tạo dự án Supabase
File môi trường	File .env trong dự án, thứ duy nhất không nằm trên GitHub
Đường link cửa hàng	Địa chỉ vercel.app của bạn

Muốn địa chỉ đẹp kiểu cuahangcuaban.com? Mua một tên miền rồi nhờ Agent hướng dẫn trở về Vercel, việc chừng mười lăm phút, làm lúc nào cũng được.

Còn cánh cửa lớn hơn thì bạn đã tự mở rồi: một ý tưởng bất kỳ, một bộ ảnh, và tám bài vừa rồi làm khuôn. Vòng lặp là như nhau cho mọi sản phẩm.



Tóm tắt bài

- Thứ tự bắt buộc: khóa kho trước, mở cửa sau; nói luật bằng tiếng người rồi để AI bật RLS qua đường dây MCP
- Khóa xong phải đi lại kịch bản bài 7 để chắc không khóa nhầm chỗ
- Vercel lấy code từ GitHub, tự dựng và phát hành; tài khoản đăng nhập bằng GitHub
- Hai chuỗi bí mật trao tận tay Vercel qua mục Environment Variables, không đi qua GitHub
- Nghiệm thu web thật bằng điện thoại dùng mạng di động và một đơn hàng do người thân đặt
- Nhịp về sau: sửa, nghiệm thu, lưu mốc, web tự cập nhật; cất kỹ ba tài khoản, mật khẩu kho, file môi trường và đường link



LỜI KẾT

Hãy quay lại bức tường ở đầu cuốn sách.

Bạn có một ý tưởng, hình dung được từng màn hình, và dừng lại ở câu "tôi không biết viết code". Tám bài trước, bức tường đó còn đứng nguyên. Bây giờ, trong điện thoại của bạn có một đường link. Bấm vào là một cửa hàng thật, chạy trên mạng thật, có kho dữ liệu thật, và một đơn hàng do người thân của bạn đặt.

Bạn vẫn chưa viết một dòng code nào. Hóa ra cánh cửa chưa bao giờ đòi hỏi điều đó.

Điều thật sự ở lại

Tám bài có rất nhiều thao tác: kéo ảnh, gõ prompt, dán SQL, bấm deploy. Những thao tác rồi sẽ cũ. Công cụ đổi giao diện, dịch vụ đổi tên nút, vài năm nữa có khi cả cái tên Antigravity cũng thành kỷ niệm.

Thứ ở lại là cách bạn làm việc với AI, gói trong bốn thói quen:

- **Mô tả có khuôn:** việc gì, dựa vào đâu, ra cái gì nhìn thấy được, ràng buộc nào. Có ảnh mẫu thì đưa ảnh.
- **Việc lớn thì kiểm kê trước, bắt lên kế hoạch, duyệt rồi mới cho chạy.**
- **File là trí nhớ:** dự án nằm trong thư mục và trên GitHub, ảnh mẫu nằm cạnh code, khóa bí mật nằm ngoài chỗ công khai.
- **Nghiem thu bằng mắt và tay:** tự bấm thử theo kịch bản trước khi tin là xong, và lưu mốc trước mỗi lần sửa lớn.

Bốn thói quen này không thuộc về công cụ nào. Chúng đi theo bạn sang mọi AI, mọi nền tảng về sau.

Sản phẩm tiếp theo là của riêng bạn

Mini Shop là sản phẩm để học. Sản phẩm đáng giá nhất là cái ý tưởng vẫn nằm trong đầu bạn từ trước khi mở cuốn sách này.

Đừng chọn ý tưởng to nhất. Chọn cái nhỏ mà bạn thật sự cần: trang giới thiệu cho cửa hàng nhà mình, công cụ ghi lịch hẹn, bảng theo dõi thu chi của đội nhóm. Kiểm vài tấm ảnh mẫu ưng mắt, mở thư mục mới, và đi lại đúng con đường tám bài: dựng thô, cho chạy thật, khám, rồi mở cửa. Con đường giờ đã quen chân.

Sản phẩm đầu tiên chạy được sẽ dạy bạn nhiều hơn mọi trang sách, kể cả những trang này.

Chia lại cho người bên cạnh

Điều bạn vừa học quý nhất khi được chia. Chỉ cho đồng nghiệp cách viết một yêu cầu bốn phần. Gửi người bạn đang ấp ủ ý tưởng cái link cửa hàng của bạn kèm câu



"tôi tự làm đấy". Ngồi cạnh con em mình trong buổi đầu nó dựng thử một trang web bằng lời.

Bức tường "phải biết code mới làm được phần mềm" đứng vững bao năm vì ai cũng tin nó. Mỗi người bước qua và quay lại kể, bức tường thấp đi một chút.

Học cùng Sao Việt

Trung tâm Đào tạo AI Sao Việt đồng hành cùng bạn trên chặng đường này. Ngoài hành trình bạn vừa hoàn thành, chúng tôi còn những chương trình khác cho người muốn đưa AI vào công việc và vận hành hàng ngày.

Liên hệ: tinhocsaoviet.com

Cảm ơn bạn đã đi cùng tới trang cuối. Chúc sản phẩm tiếp theo của bạn, cái của riêng bạn, sớm có đường link của nó.

